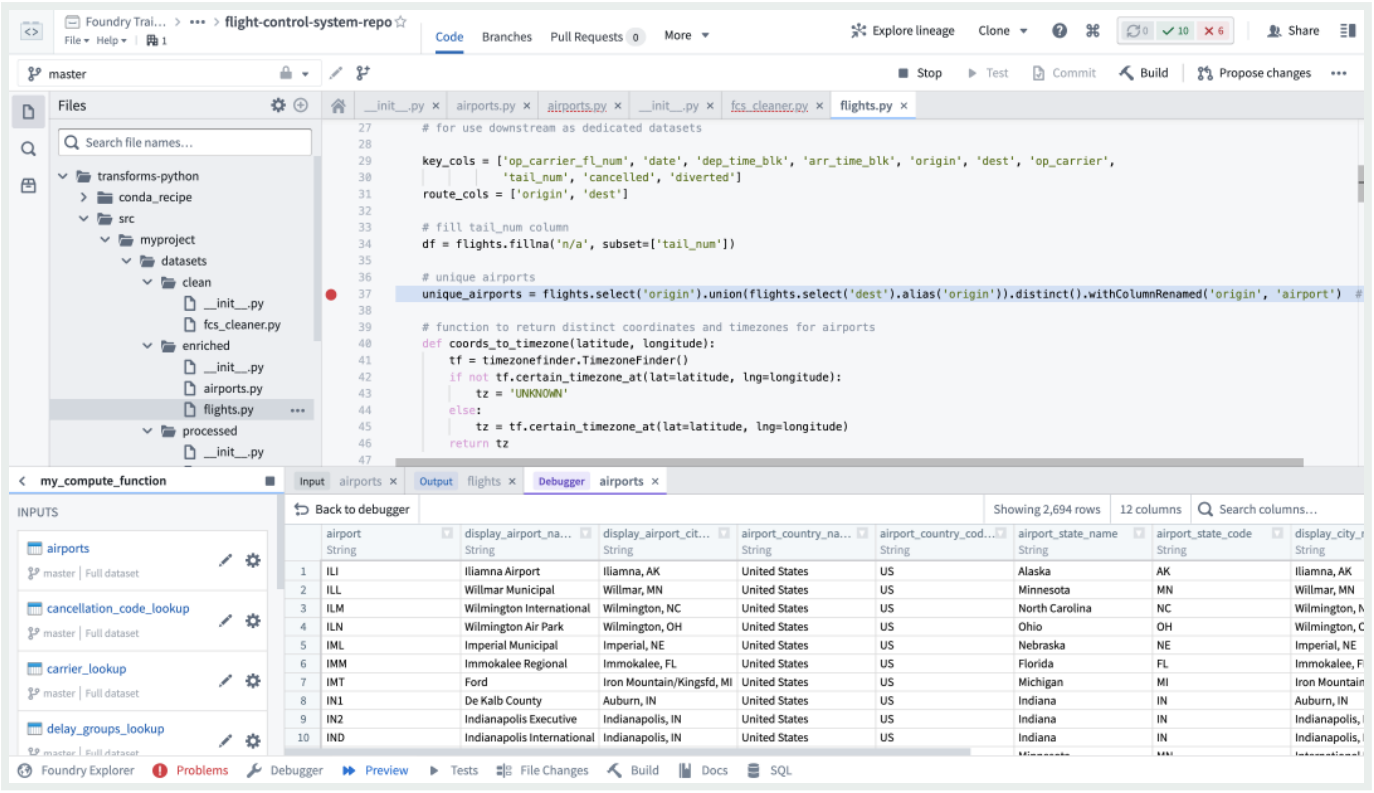
Table of Contents

Code Repositories

> To edit and manage your code in a familiar IDE, try VS Code workspaces and the Palantir extension for Visual Studio Code.

Code Repositories provides a web-based integrated development environment (IDE) for writing and collaborating on production-ready code in Foundry. The application provides a user-friendly way to interact with the underlying Git repository, and provides a range of additional features:

- All common Git version control tasks, including branching, committing, and tagging releases can be performed through the web UI
- Repositories have integrated support for code review and collaboration through *pull requests*, including support for highly configurable permissions to ensure the codebase is high-quality
- Every repository type includes integrated features to aid with the code authoring experience, including IntelliSense, code linting and error checking, and rich help dialogs



Repository types

Code Repositories support creating repositories of many types. The most common repository types are described below.

- **Transforms** repositories support authoring data transformation logic, and include features to enable previewing and debugging transforms. Supported languages include Python, Java, and SQL
- **Functions** repositories enable writing business logic that can be executed with low latency in an operational context, and include native support for accessing data from the Foundry Ontology. The Code Repositories environment supports autocomplete based on Ontology data types, and enables code authors to preview Functions while authoring them. Functions can be written in TypeScript or Python.
- **Model development** is supported in Code Repositories. Learn more about how to develop models in Code Repositories.

← API Reference ↗ ransforms / Window

NEXT
Python transforms [Legacy] →

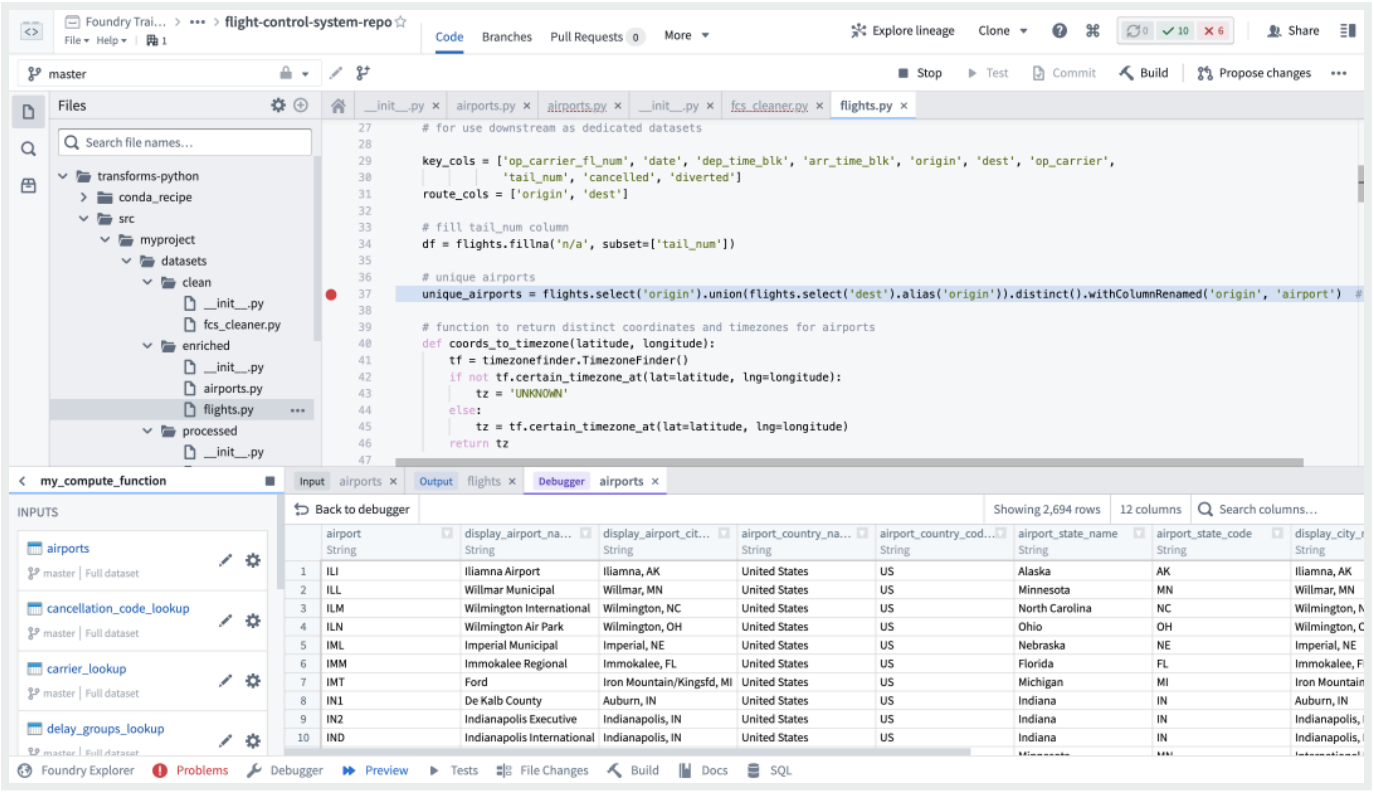
Send feedback

Code Repositories

> To edit and manage your code in a familiar IDE, try VS Code workspaces and the Palantir extension for Visual Studio Code.

Code Repositories provides a web-based integrated development environment (IDE) for writing and collaborating on production-ready code in Foundry. The application provides a user-friendly way to interact with the underlying Git repository, and provides a range of additional features:

- All common Git version control tasks, including branching, committing, and tagging releases can be performed through the web UI
- Repositories have integrated support for code review and collaboration through *pull requests*, including support for highly configurable permissions to ensure the codebase is high-quality
- Every repository type includes integrated features to aid with the code authoring experience, including IntelliSense, code linting and error checking, and rich help dialogs



Repository types

Code Repositories support creating repositories of many types. The most common repository types are described below.

- **Transforms** repositories support authoring data transformation logic, and include features to enable previewing and debugging transforms. Supported languages include Python, Java, and SQL
- **Functions** repositories enable writing business logic that can be executed with low latency in an operational context, and include native support for accessing data from the Foundry Ontology. The Code Repositories environment supports autocomplete based on Ontology data types, and enables code authors to preview Functions while authoring them. Functions can be written in TypeScript or Python.
- **Model development** is supported in Code Repositories. Learn more about how to develop models in Code Repositories.

← API Reference ↗ ransforms / Window

NEXT
Python transforms [Legacy] →

Send feedback

Python transforms in Code Repositories [Legacy]

⚠ Python transforms support in Code Repositories is in the legacy phase of development. The Code Repositories editor will remain supported and available, though future feature development will focus on VS Code workspaces.

VS Code workspace is the recommended environment for working with Python transforms and includes the current features of Code Repositories along with AI-powered coding assistance, full dataset preview, an optimized language server, and an integrated terminal.

Review a feature comparison between VS Code workspaces and Code Repositories.
Learn more about the benefits of VS Code workspaces.

Python transforms support in Code Repositories is in the legacy phase of development. Workflows for additional repository types supported in VS Code workspaces will move to the legacy phase over time.

If VS Code workspaces are unavailable on your enrollment, Code Repositories remains the recommended way to author Python transforms.

A[illegible]ansforms in Code Repositories

API Reference ↗

C[illegible]itories remains the recommended editor for other repos

Send feedback

Java and SQL transforms.

PREVIOUS
← Overview

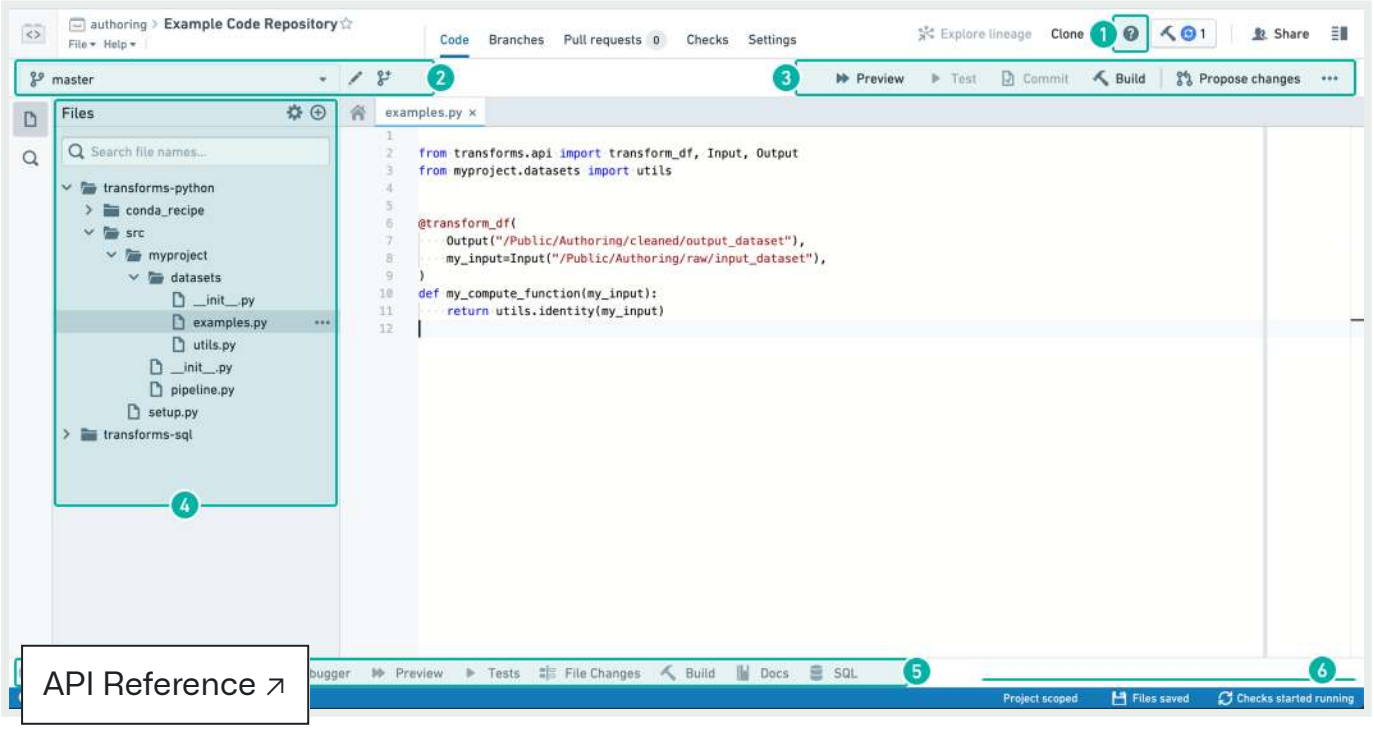
NEXT
Navigation →

Navigation

There are five different tabs that you can select at the top of the Code Repositories interface:

1. Code tab
2. Branches tab
3. Pull requests tab
4. Checks tab
5. Settings tab

Code tab



API Reference ↗

Help

Send feedback

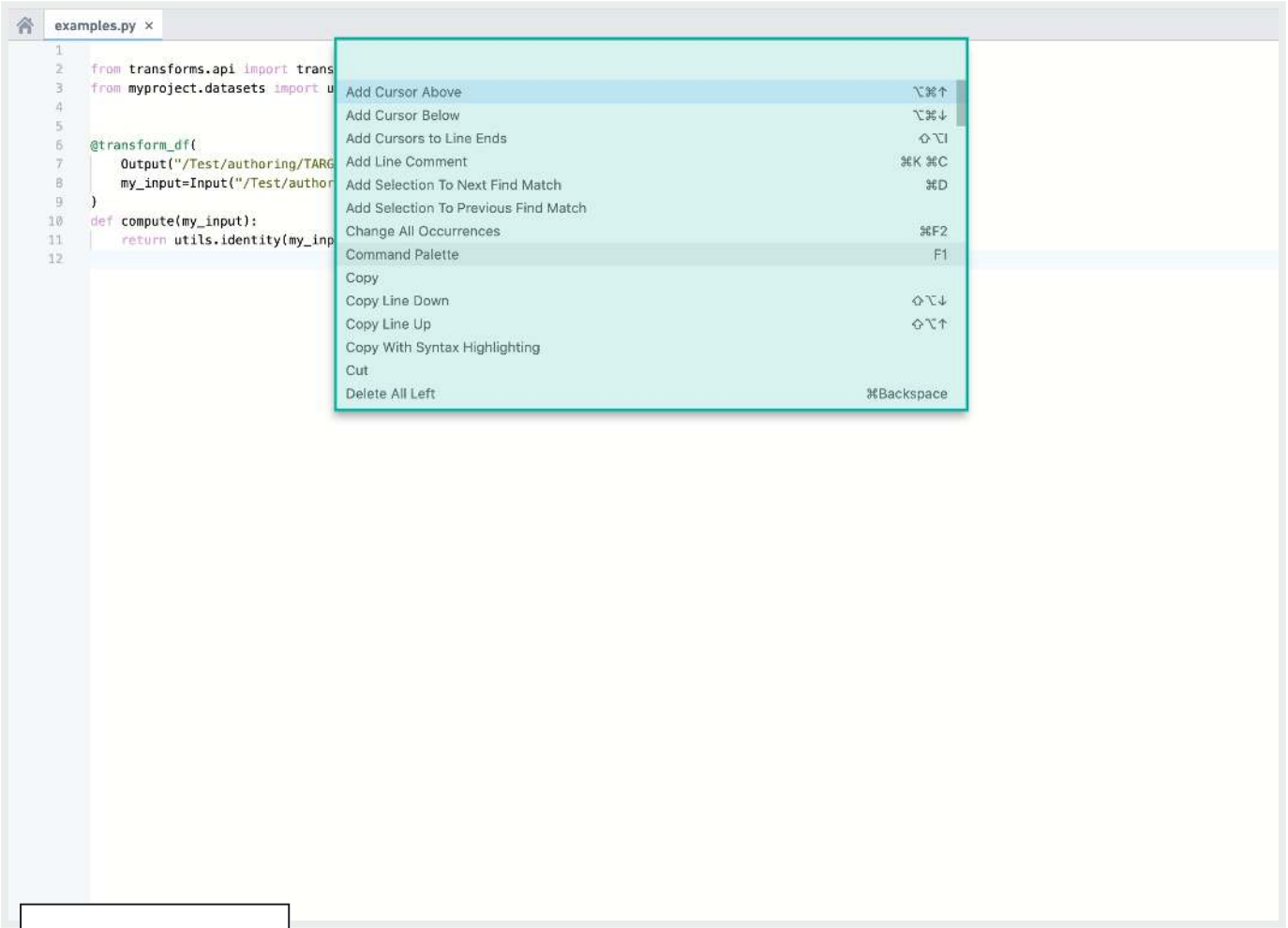
1. Help
2. Branch Options

3. Code Editor Options
4. File Editor
5. Helper Panels
6. Status bar

In-App help

In the **Code** tab, you can click the ? button to start a step-by-step walkthrough that guides you through the core functionalities available in your Code Repository. The in-app help is currently only available in the **Code** view.

To expose keyboard shortcuts via the command palette, use the F1 key in Windows or Fn+F1 on macOS:



API Reference ↗

Send feedback

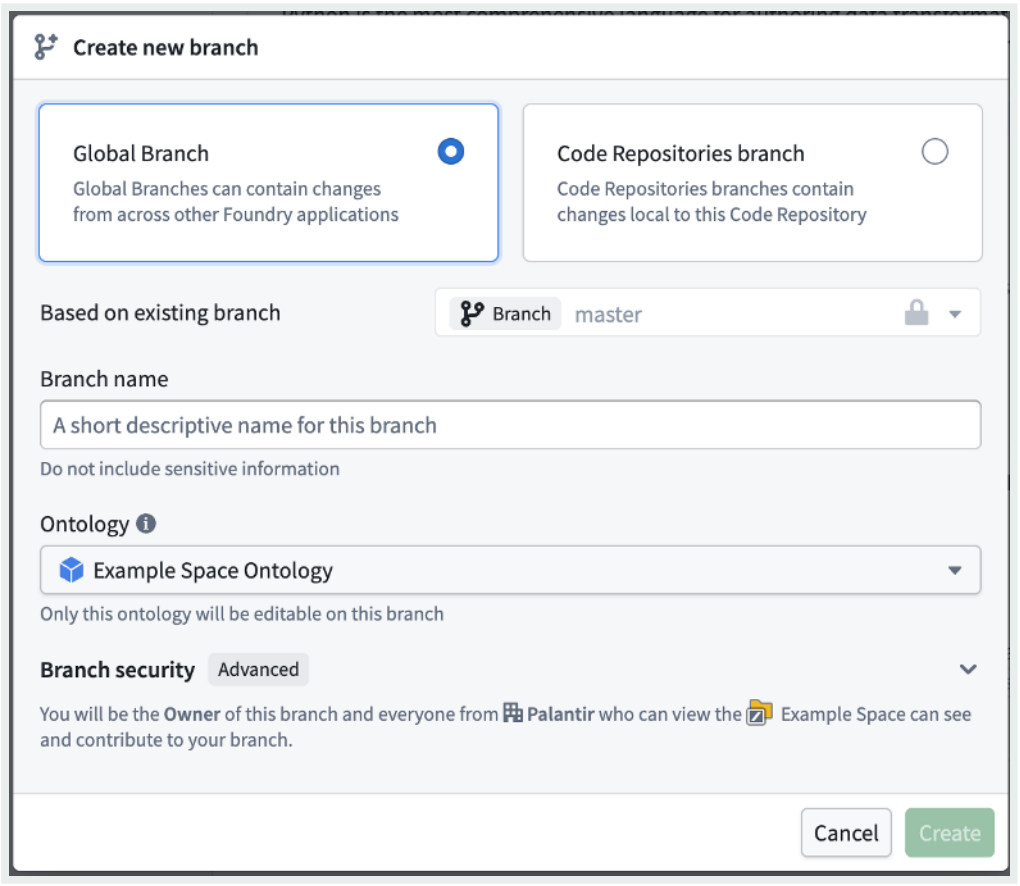
Branch options

Use the dropdown branch menu to select one of your existing sandbox branches to work in.

Alternatively, click the  icon to create a new sandbox branch which contains a copy of the code on an existing branch. To edit code in your repository, you must work in a sandbox branch — protected branches cannot be directly edited.

You can choose between a **Code Repositories branch** and, for supported repository types, a **Global branch**. Global branches are supported in **Python Transforms** and **TypeScript v1** repositories, and can contain changes across multiple Palantir applications. Learn more about global branches.

Global branches must be based on the main branch of the repository, while Code Repositories branches can be based on any branch.



API Reference ↗

Code editor options

Send feedback

As you write and edit code in your repository, you can do the following:

- Click the Preview button to run the Transform on a sample of the input datasets. This is a quick way to preview your code changes on real data.
- Click the Test button to run all unit tests defined in the current file. See Python Tests or Java Tests for information about how to add unit tests to your repository.
- Click the Commit button to commit any changes in your sandbox branch. Once you commit changes, automatic checks run on your code.
- If you select a dataset source file (a file that defines a transformation, see e.g. Python Transforms), you can click the Build button to build a new version of your output dataset after running automatic checks on your code. Clicking the button will trigger a build on *all output datasets of the current file*; if the current file does not generate any datasets, no build is triggered.
- Click the Propose changes button to create a new *Pull request* containing your changes. This allows others to review and comment on your code before merging it into the main code.
- Click on the **...** button to access several additional actions:
  - **Merge:** Merge another branch into your current branch.
  - **Reset:** Reset the contents of all files to match the latest commit on your remote branch. This will clear any changes that have not yet been committed on your branch.
  - **Upgrade:** Upgrade your branch to the latest language versions.

File editor

Click the  icon to create a new file, folder, or sub-project. Select the “New sub-project” option if you want to add another language-specific sub-project to your Code Repository.

Helper panels

Foundry Explorer Helper

API Reference ↗

T**r** helper is a file navigation interface that lets you quickly browse all files and folders. Once you select a specific dataset, you can click “O

Send feedback

dataset.

Problems Helper

The **Problems** helper tells you about any issues detected in your code. Click on a specific issue listed here to open up the problematic code.

Debugger Helper

The **Debugger** allows you to examine your transform behavior while it runs (see Debug Transforms).

Preview Helper

The **Preview** helper lets you run your code on a limited sample of the input datasets to quickly preview the code without committing your changes (see Preview Transforms).

Tests Helper

When your repository contains unit tests (see Python Tests and Java Tests), the **Tests Helper** lets you run those tests and displays their results.

File Changes Helper

The **File Changes** helper can be used to view any uncommitted changes to the current file, as well as compare previous versions of the file.

Build Helper

The **Build** helper lets you trigger dataset builds and view the progress for your builds. Once you select a dataset source file, you can click the build button to build a new version of your output dataset as well as run automatic checks on your code. You can then view the progress of the running tasks in the **Build** helper.

> Clicking the **Build** button at the top right corner of the Code Repositories interface is equivalent to triggering a build from the **Build** helper.

API Reference ↗

Docs Helper

The **Docs** helper contains references for the available languages that you can write code in. For more detailed language-specific documentation that isn't available in the in-product documentation, refer to the supported languages.

SQL Scratchpad

The **SQL** helper lets you quickly test out SQL queries. Write a SQL query and click ▶ Run to preview the results of your query. You can also test out an existing query you've written in your repository by selecting the appropriate `.sql` file and clicking Preview File .

To view queries marked as favorites, go to the ★ tab. To view a history of queries ran in the **SQL** helper, go to the 🕒 tab.

> ⓘ To access a specific branch of an input dataset in SQL Scratchpad, you prepend the name of the branch to the query: e.g. `SELECT * FROM` `` `branch_A`.`/path/to/dataset` ``. If no branch is specified it will default to `master`.

Status bar

The status bar provides information on the state of the environment and checks results. Information you can find in the status bar includes:

- **Code Assist state** - Code Assist is essential for detecting problems in your code and running previews. You can write code without Code Assist, but will need it to be up and running to complete your work. Hover over the Code Assist status you can get details on the initialization progress. If Code Assist fails to initialize, raise a support request.
- **Problems** - When problems are detected in your code, an indication appears on the left side of the status bar. Click on the indication to open the Problems helper.
- **Checks status** - The checks status is displayed on the right side of the status bar. More details of checks are in the Checks tab.
- **File saving** - After any change, the file saving status displays when the automatic save progress.

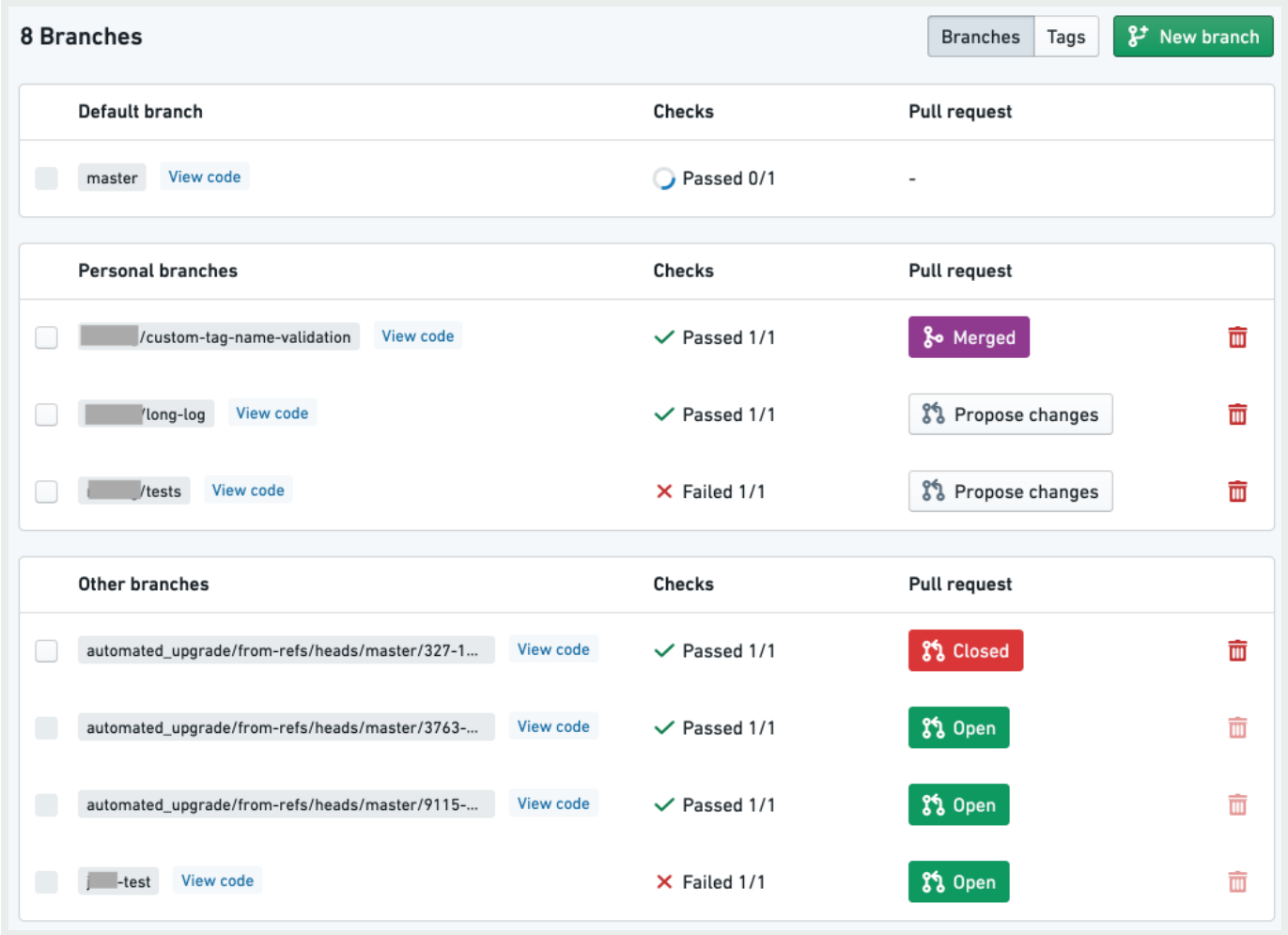
API Reference ↗

Branches tab

Send feedback

> This section summarizes how to create new branches and create new Pull requests in the **Branches** tab. These functionalities are also available in the Code tab.

In the **Branches** tab, you can see a list of branches existing in your Code Repository — this includes your own branches as well as other users' branches. Click the New branch button to create a new sandbox branch which contains a copy of the code on a specific branch.



Each listed branch contains a summary with the following available functionalities:

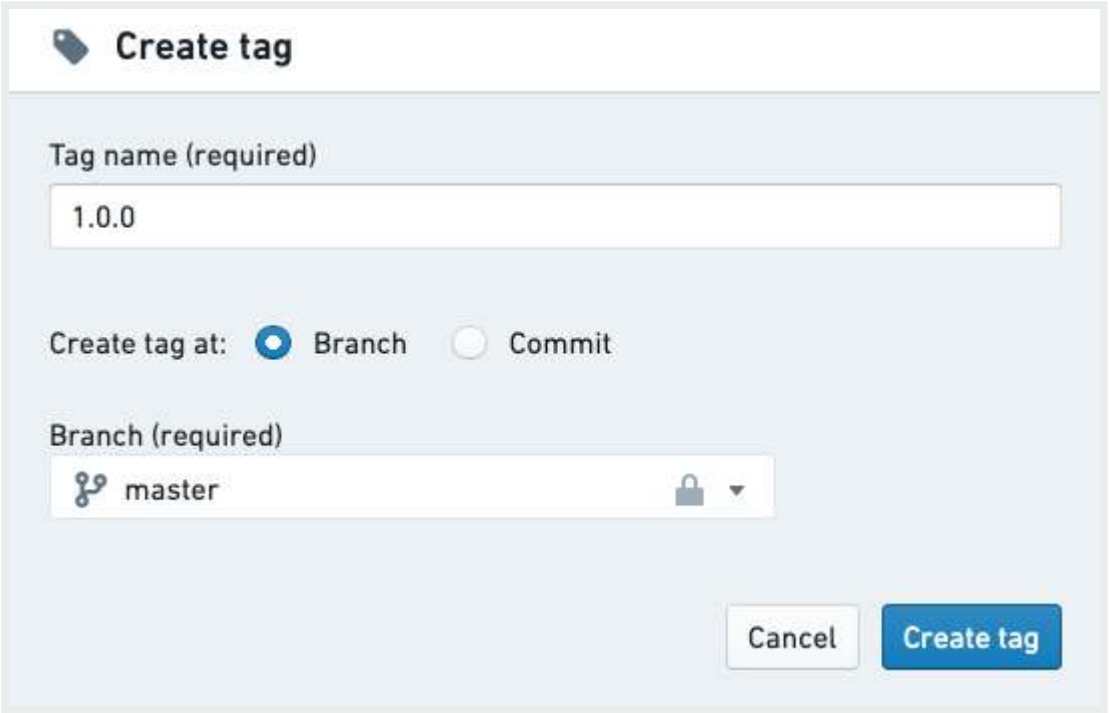
- The "Checks" column indicates whether or not the automatic code checks have passed for a branch.
- The "Pull request" column tells you about any existing *Pull requests* in a branch and lets you create new *Pull requests*. To create a new *Pull request* that contains the changes on e "Propose changes" button. This will create a new *Pull request* for merging your changes into the *master* branch by default. If you want to merge your changes into a branch other than *master*, select a different branc menu. If you don't see the button to create a new *Pull request*, it means that a *Pull*

request already exists for a branch. Click on the “Open” / “Closed” / “Merged” button to open the full *Pull request*.

- Click “View code” next to a branch name to view the code on that branch.
- To delete a branch, click the icon. **You should not delete any branches that you did not create. This can result in lost work for others.**

Tags

The branches tab also lets you access a list of **tags**, which are like immutable branches. A tag can be used to mark a significant version of the code for future reference by giving it a version number or name. To create a new tag, navigate to the tags section of the branches tab and click the "New Tag" button. A tag can be created from the current version of a branch, or from any arbitrary commit.



To enforce that all tag names follow a specific naming convention, you can add a `tagNameValidation` configuration block to a file called `repoSettings.json` at the root of the repository, e.g.: "tagNameValidation": { "regex": "^(0|[1-9]\\d*)\\.(0|[1-9]\\d*)\\.(0|[1-9]\\d*)(-rc\\d+)?$", "errorMessage": "Tag name must have the format '.' }

API Reference ↗

Send feedback

For more information about the recommended git development workflow, refer to the Developer Best Practices.

Pull requests tab

> This section summarizes how to create new Pull requests in the **Pull requests** tab. This functionality is also available in the Code tab.

In the **Pull requests** tab, you can find information about *Pull requests* in your Code Repository. A *Pull request* lets users view a history of the changes on your branch and review your code on a line-by-line basis before merging your changes. Any time you want to merge the changes on your branch into the main code, you should create a new *Pull request*.

Click the + New pull request button to create a new *Pull request*. By default, the new *Pull request* created will merge your changes into the main branch of your repository (this is usually the *master* branch). Select which branch you want to base your new *Pull request* off of.

Filtering pull requests

You can switch between a list of open and closed *Pull requests* by clicking the "Open" / "Closed" button at the top of the pull requests list, and use the search bar to further filter the list based on title or author.

API Reference ↗

Reviewing pull requests

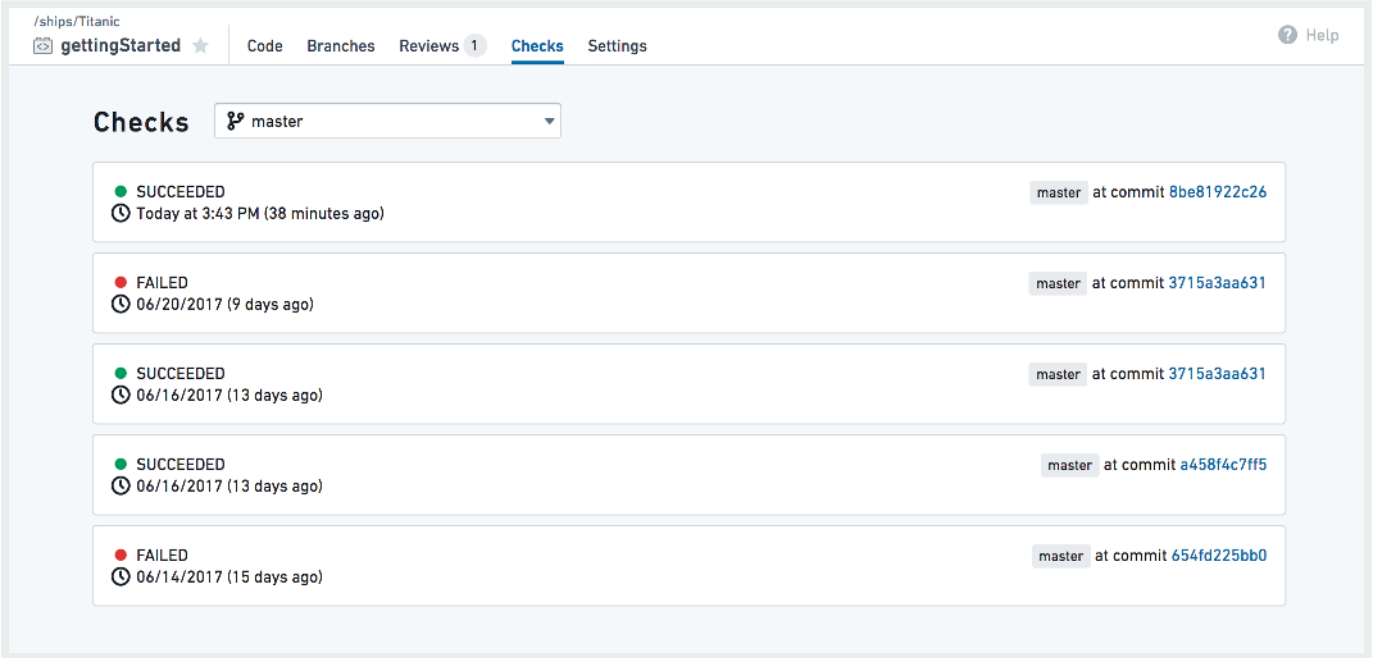
Send feedback

Click on one of the *Pull requests* to review the proposed code changes line-by-line and add comments. Depending on the repository settings, each *Pull request* may require at least one approving review before they can be merged.

When reviewing changes to Transforms code, you can also check how these changes affect your datasets.

Checks tab

In the **Checks** tab, you can view a summary of running and completed checks on each branch. Use the dropdown branch menu to select a different branch. Click on a specific check to view more detailed information.

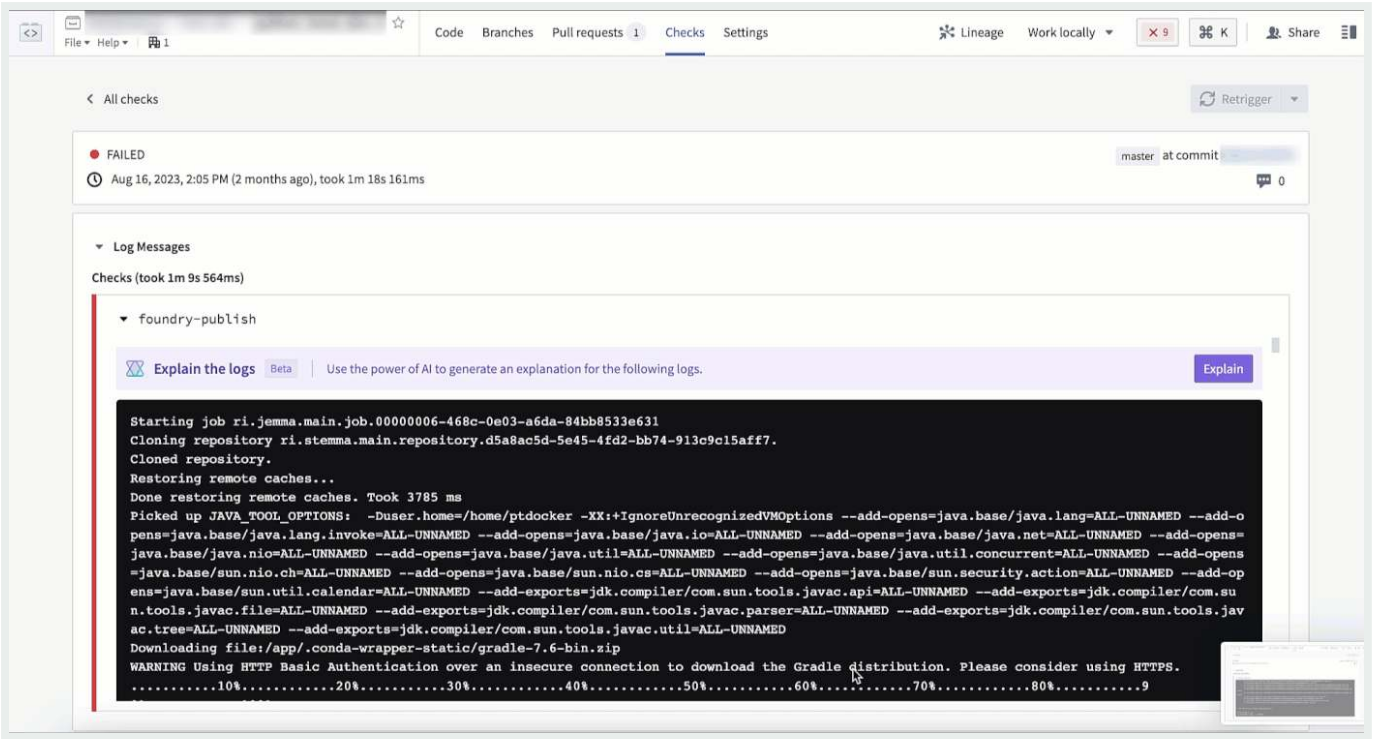


The Checks tab will also include the output of any unit tests that have been defined for your repo. You can define unit tests for Python and Java.

> If AIP is enabled on your stack, the error enhancer widget complements the detail view of a failed check to help you better understand and resolve issues that arise.

API Reference ↗

Send feedback



Settings tab

In the Settings tab, code authors can configure their personal editor preferences and repository administrators can control the repository's behavior and policies. To learn more about the Settings tab, see Administering Code Repositories.

> Most options in the **Settings** tab are aimed for administrators and will be available only to users with the appropriate permissions (by default, repository owners).

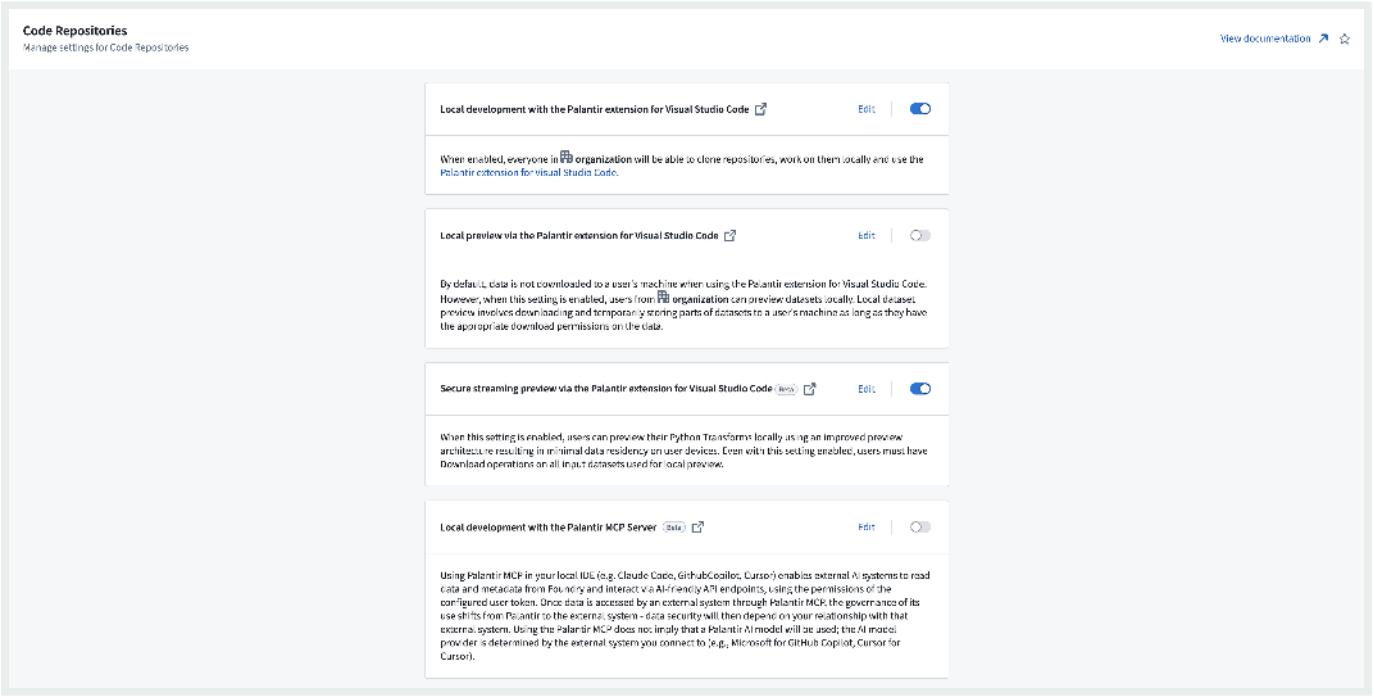
PREVIOUS
← Python transforms [Legacy]

NEXT
Configuration / Configure Code Repositories settings in Control Panel →

API Reference ↗
Privacy Statement ↗
Terms of Use ↗
Send feedback

Configure Code Repositories settings in Control Panel

You can configure many Organization-wide Code Repositories settings within Control Panel. To modify these settings, you will need the `User Experience Administrator` role.



Available settings

- **Local development with the Palantir extension for Visual Studio Code:** When enabled, organization will be able to clone code repositories and work on them locally using the Palantir extension for VS Code. This setting is enabled by default.
- **eview through the Palantir extension for Visual Studio C** g is enabled, users from your organization can preview datasets locally. Local dataset

preview involves downloading and temporarily storing parts of datasets to a user's machine. This setting is disabled by default. This setting is deprecated and is superseded by secure streaming preview via the Palantir extension for Visual Studio Code.

- **Secure streaming preview via the Palantir extension for Visual Studio Code:** When this setting is enabled, users from your organization can preview datasets locally using the secure streaming preview implementation. Local dataset preview involves streaming parts of datasets to the system memory of a user's machine. This setting is enabled by default. This setting does not yet take effect and it will become effective after a platform intervention.
- **Local development with the Palantir MCP Server:** Using Palantir MCP in your local IDE, such as Claude Code, GitHub Copilot, or Cursor, enables external AI systems to read data and metadata from Foundry and interact via AI-friendly API endpoints, using the permissions of the configured user token. Once data is accessed by an external system through Palantir MCP, the governance of its use shifts from Palantir to the external system. Data security will depend on your relationship with that external system. Using the Palantir MCP does not imply that a Palantir AI model will be used; the AI model provider is determined by the external system you connect to (for example, Microsoft for GitHub Copilot, Cursor for Cursor).

> ⚠ To preview locally, users must be allowed to do so through the above local preview settings and have `Download` operations on the previewed inputs. These are privileged operations that allow users to download entire datasets locally, and should be granted with caution. In cases where users cannot be granted this permission, the VS Code developer experience is still available through Code Workspaces, which guarantees stricter data control. For more details, see the security considerations for local preview.

You can learn more about local development, previewing and debugging Python transforms, and using the Palantir extension in our documentation.

Code Repositories FAQ

The following are some frequently asked questions about Code Repositories.

For general information, view the Code Repositories documentation.

Can I publish a Python package from the latest commit on a branch without requiring tagging?

Yes. You can publish the latest commit of a Python package by modifying your package's root `build.gradle` file to publish the branch. For example, to publish the latest commit on the master branch, modify the `build.gradle` file as follows:

Return to top

How do I restore previously deleted code in my code repository?

If these transforms were built into a dataset, you can use the **Compare** feature of the resulting Dataset Preview to view the code at that time. From there, you can copy-paste the relevant transforms. Alternatively, you can navigate to **Branches** in your code repository, open the specific branch, and review the full history of changes there.

Return to top

Can I duplicate my code repository?

There is no built-in capability to copy code repositories in the platform. You can however, clone the repository to your machine and then push that code to a new code repository. If you do this, remember to add all the inputs as references to the new project. Learn how to clone a repository.

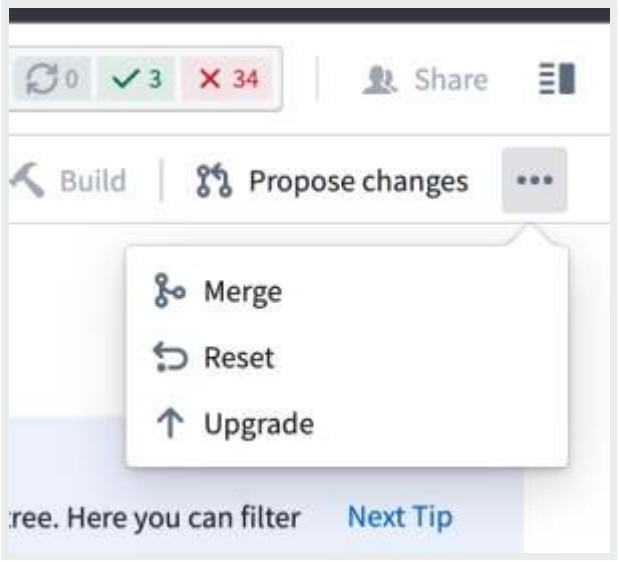
Return to top

How do I know if I need to upgrade my branch?

You can confirm your code repository is up-to-date by selecting ... in the top-right corner of the code repository and confirming whether **Upgrade** appears as an option. If the **Upgrade** option is not available, then the repository is already up-to-date.

API Reference ↗

Send feedback



Return to top

Can my transform dynamically select inputs and outputs whenever a build starts?

This is not supported. Continuous integration (CI) checks define the set of inputs and outputs whenever a new commit is added in the code repository.

Return to top

In Code Preview, code works but builds fail

Your Code Preview succeeds but your build fails. Code Previews run on a subset of data, which likely means there are data values not in the subset breaking your code when the full build runs.

To troubleshoot, perform the following steps:

1. API Reference ↗ error from the failed build. Look for lines starting with “Caused by” and read them carefully. Sometimes they will mention explicit lines of code in your transformation file.

2. Try upgrading your code repository. To do this, in the top right corner of your repository, select **... > Upgrade**. This will create a PR to upgrade your branch. Be sure to merge in the upgrade PR so that your branch actually gets upgraded. You can upgrade any branch, but merging the upgrade commit into a protected branch requires a review and approval.

Return to top

Code running in Code Workbook but not in Code Repositories

Sometimes, porting code from a code workbook to a code repository will not work without modifying the code to run in a code repository.

To troubleshoot, perform the following steps:

1. Have you verified your transform decorator is correct? That is, `@transform` vs. `@transform_df`?
2. Have you verified your inputs are all declared and passed as inputs to your compute function?
3. Are the names of your dataframes the same as your workbook? Is your code repository actually returning the dataframe?
4. Check that the libraries being used in the code workbook code are also available in the code repository.
5. Verify the inputs and branches to the code workbook cell are in fact the same datasets used in the code repository.

Review our FAQ on builds and checks errors for more detail.

Return to top

C failing due to a missing Python library, in a code repository where that library was

working

Sometimes, a repository that has been working starts encountering a problem where repository checks begin failing with an error indicating that Conda packages could not be obtained. This may be a `PackageNotFoundError`, or a `MD5MismatchError` due to a Conda cache getting corrupted.

To troubleshoot, perform the following steps:

1. In most cases, the Conda cache can be unstuck by simply creating a new commit in your code repository. Open the repository and make any change (even just an empty new line), and press **Commit**.
2. If the symptoms still appear after performing a new line commit, it is possible that the Conda cache has made a higher-level cache also become corrupt. The following steps will clear your code repository's Gradle cache.
3. In your code repository, press the settings (cog wheel) icon, and enable **Hidden files**.
4. Locate `conda-versions.run.linux-64.lock` in your Python subproject, delete it, and press **Commit**.
5. If the symptoms still appear after clearing both of these caches, contact Palantir Support.

Return to top

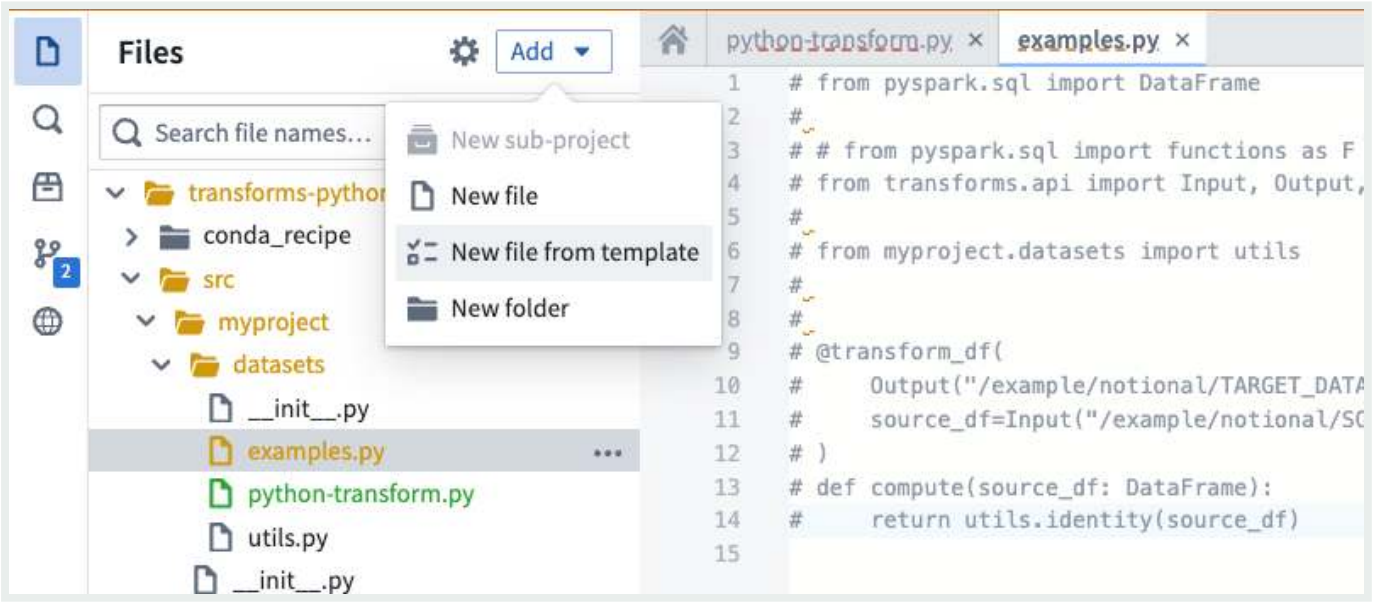
PREVIOUS
← Configuration / Configure Code Repositories settings in Control Panel

NEXT
Transforms / Create transforms →

Create transforms

Transforms can be created in Code Repositories using the file template configuration wizard. This wizard enables you to select a transform type and then bootstrap a minimal example by providing values for the variables required by the transform, such as input or output datasets.

To get started using the wizard, create new Python transforms repository. The wizard will be automatically opened. In existing repositories, the wizard can be opened by selecting **Add** and then **New file from template** in the Files side panel.



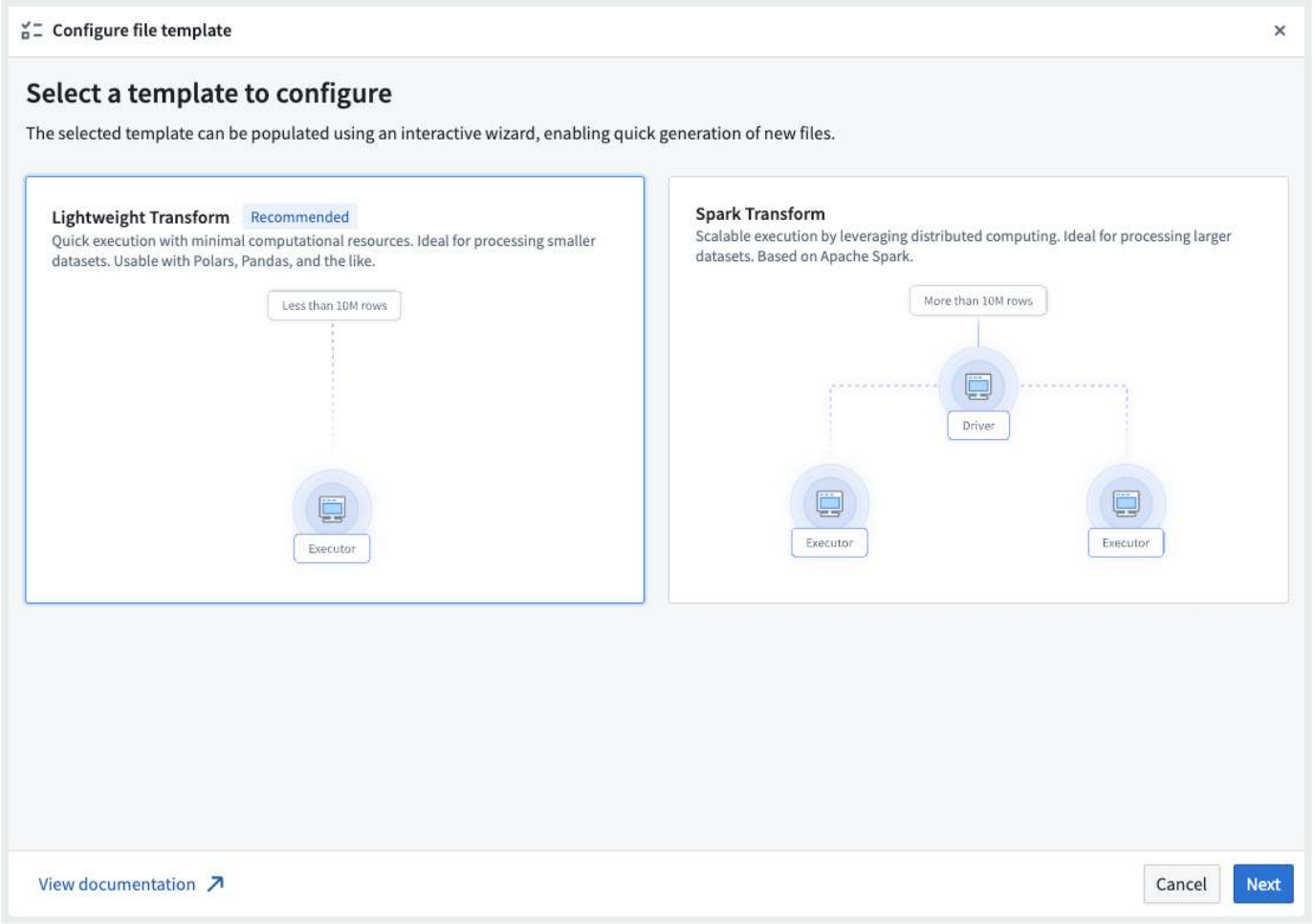
After opening the wizard, you will be prompted to select a template. Each template represents a transform type. Illustrations and descriptions are provided to help contrast the different options and identify the correct choice for your use-case. Available templates in

API Reference ↗

- **ght transforms:** Generally recommended for transforms datasets of less than 10 million rows.

Send feedback

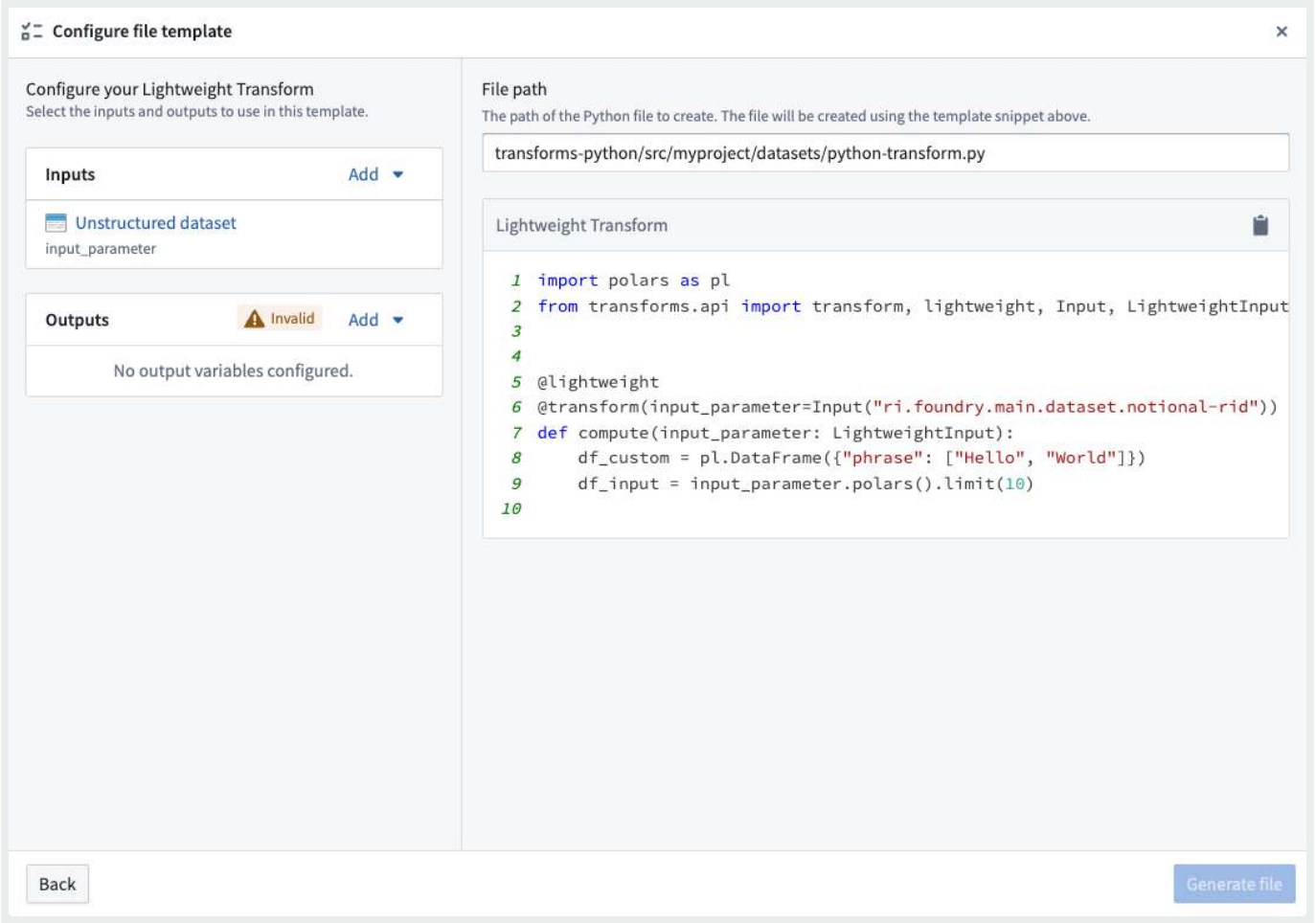
- **Spark transforms:** Leverage distributed computing to enable better performance for transforms on datasets of more than 10 million rows.



After choosing a template, select **Next** to open the template configuration page. Here, you can configure your template by supplying the required values and selecting resources from across Foundry. This page displays a live preview of your transform based on the current configuration; modifying any input or output variables will update the preview. Your configuration is automatically validated at this stage, and any errors must be addressed before the transform can be created.

Common validation errors include:

- **Using resources from different projects:** Input and output resources must be contained within the same project as the repository.
- **Duplicate parameter names:** All parameter names in your transform must be unique.
- **quired parameters:** Some templates require at least one input or output a valid variable value to resolve this error.



Once your configuration is valid, you will be able to create your transform by selecting **Generate file**. This will create a new file in your repository with the contents shown in the preview of the configuration page. If the template requires specific backing dependencies or libraries, these will be automatically configured so that your transform runs correctly.

← PREVIOUS
FAQ

NEXT
Preview transforms →

Preview transforms

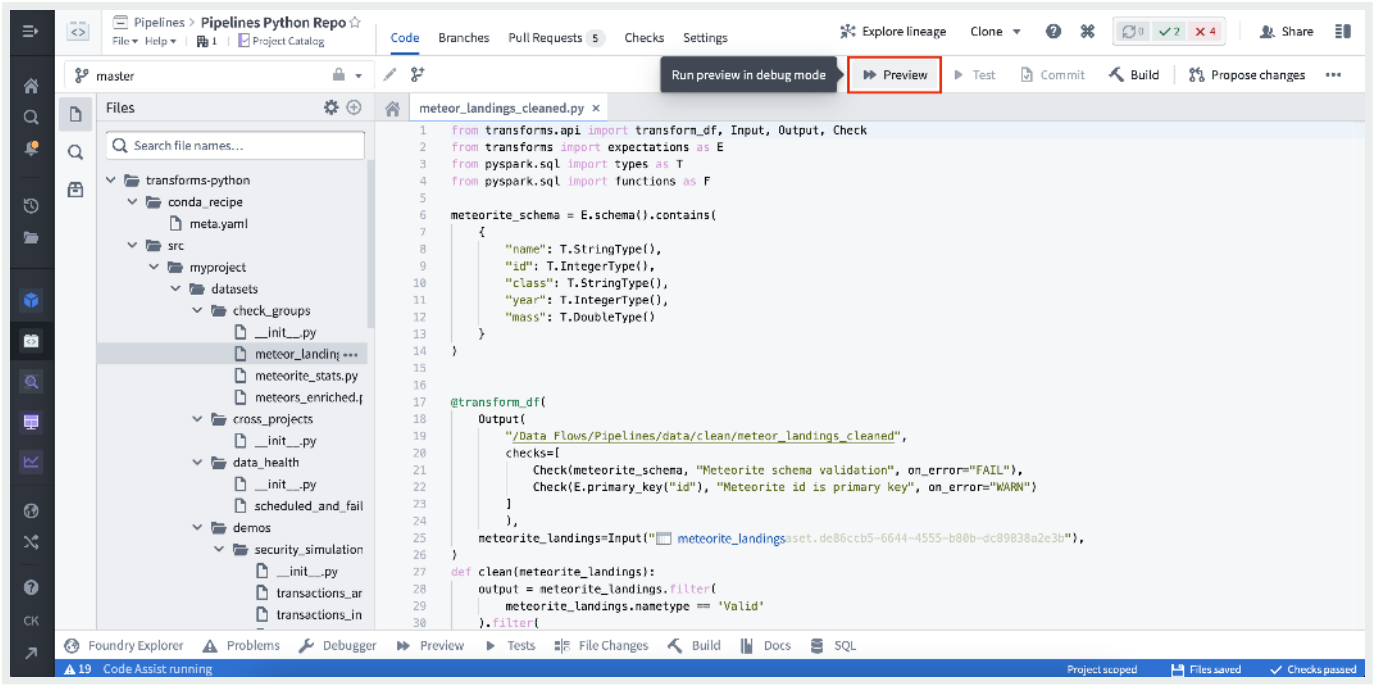
Use the Preview tool in Code Repositories to run your code on a limited sample of the input datasets to quickly preview the output. Preview produces a sample output without committing changes, running checks, or materializing any datasets in Foundry. Preview can accelerate the development cycle, removing the need to trigger a build to test code changes.

> **Tip**
>
> Preview works on all Foundry datasets, including datasets with files and models.

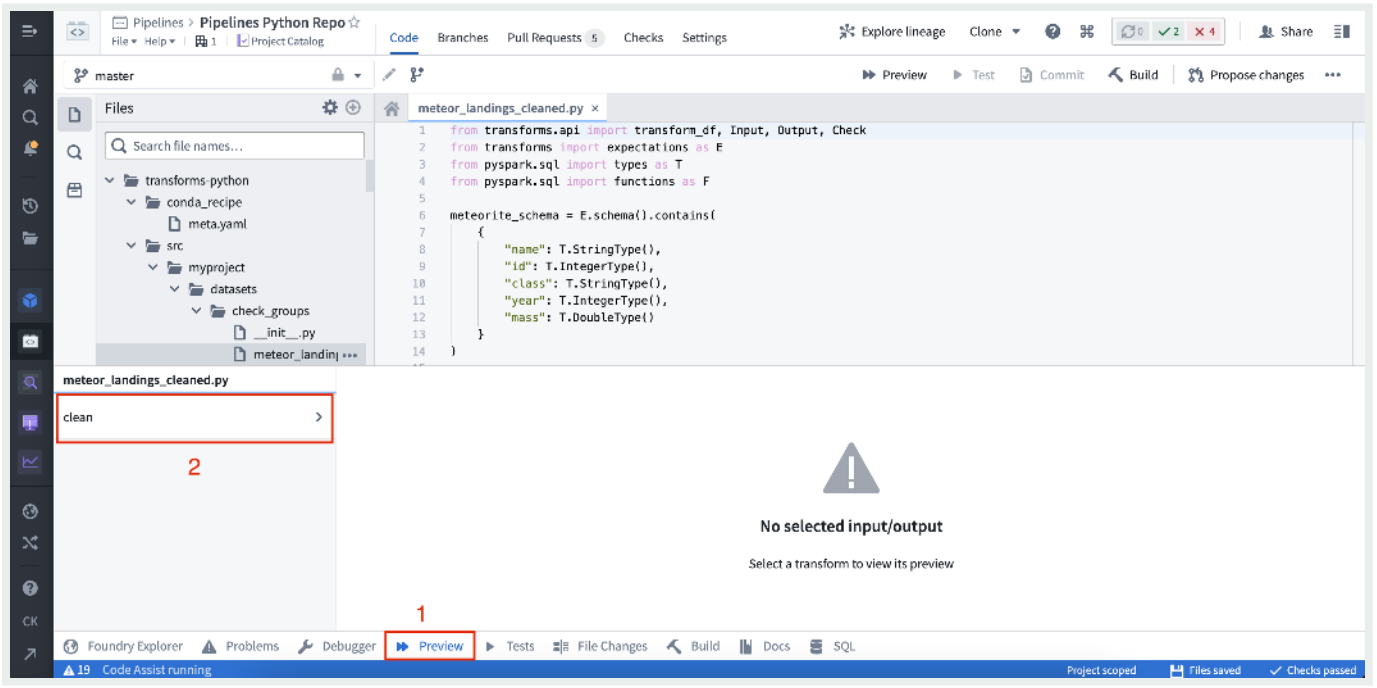
Running Preview

Preview can be triggered from two places within Code Repositories.

(1) By selecting Preview in the code editor options panel:

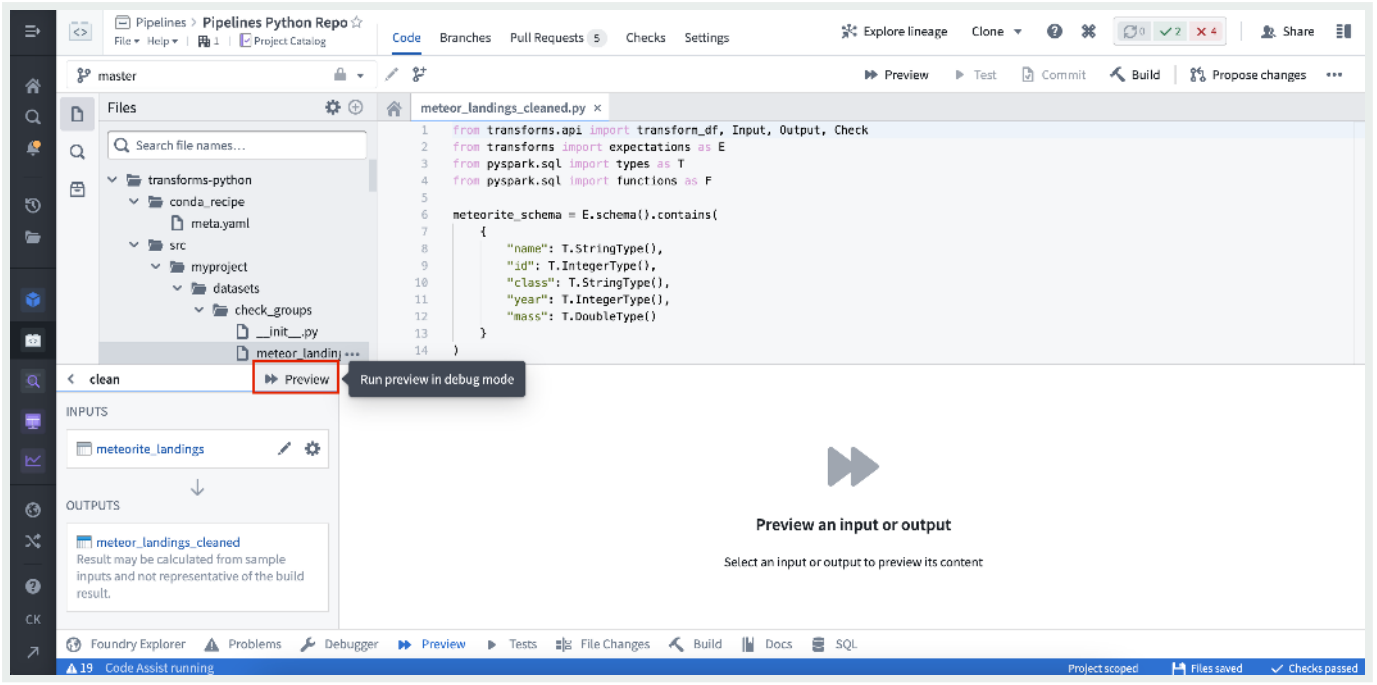


(2) By selecting Preview in the helper panel:

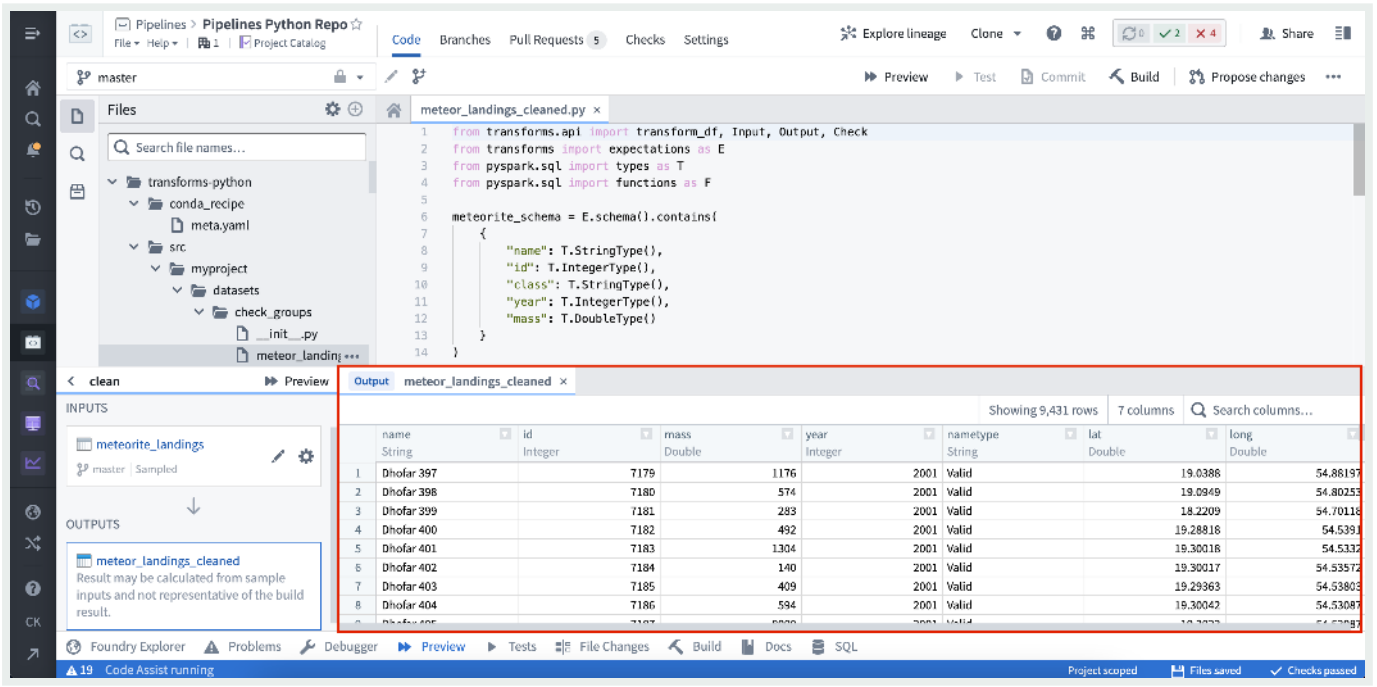


API Reference ↗

Send feedback



Once the Preview has executed, the output is displayed:

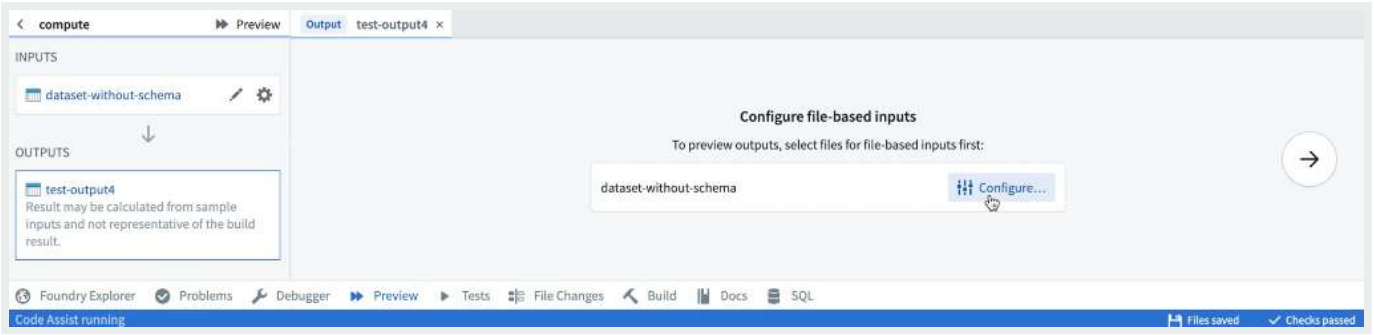


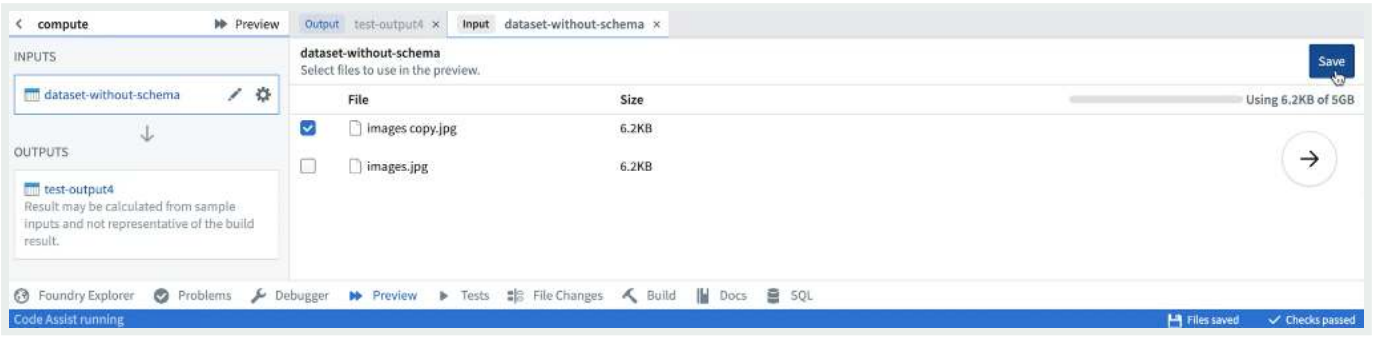
Configuring Preview with files

Preview can be used on datasets that contain unstructured files. When running Preview for th taset containing files, you must configure the files that will be used w

API Reference ↗

Send feedback





Once the sample files have been selected, they can be reconfigured by selecting the relevant input from the list of inputs. After saving the configuration, Preview will execute the code on the chosen sample of files. When running Preview again, there will be no need to reconfigure input files. Once Preview has executed, you can view the sample output as rows or files. If you have the required permissions, you can also choose to download the output files.

Configuring Preview with models

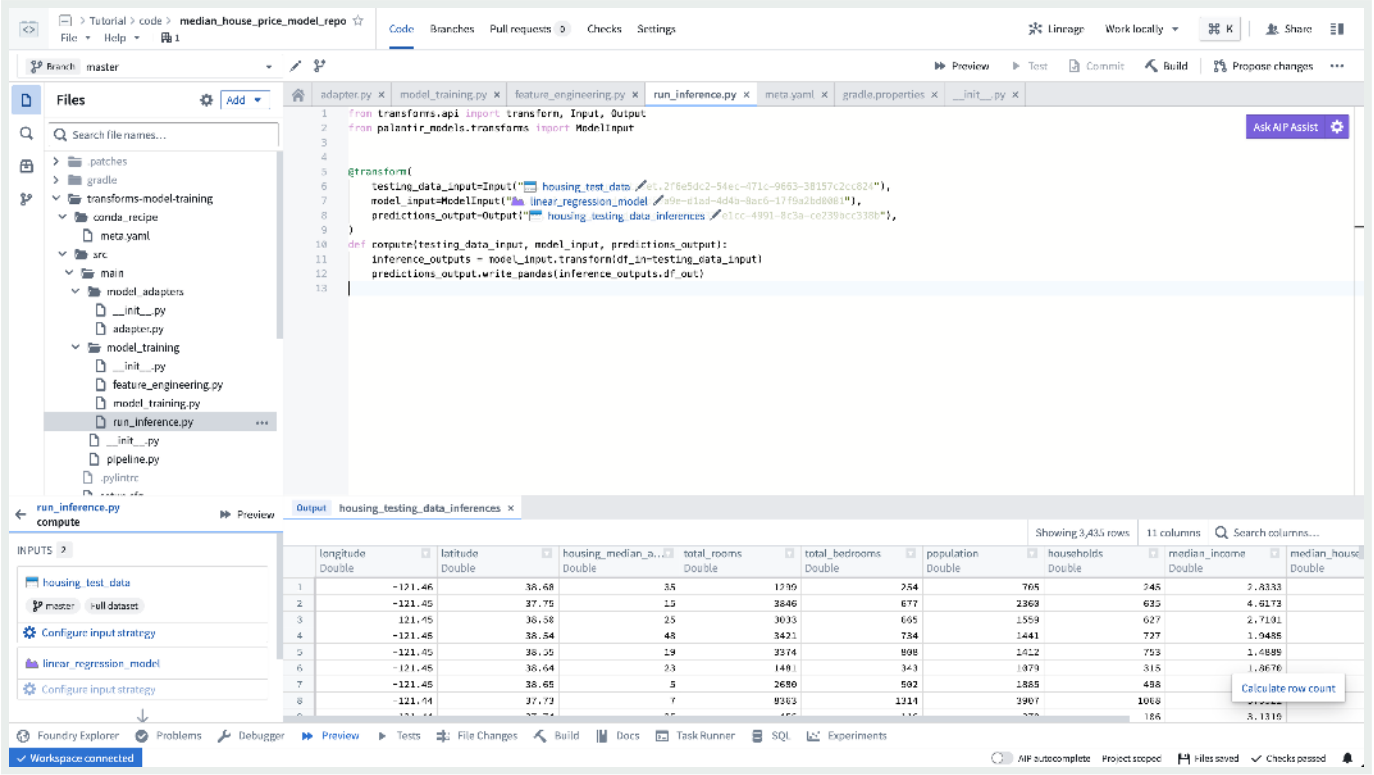
Model Assets

Preview, without the requirement of additional configuration, is supported for model assets that are trained in Foundry or backed by pre-trained files.

Container backed models and externally hosted models do not currently support preview.

API Reference ↗

Send feedback



Previewing transforms created in transforms generator

Transforms created in a transforms generator share the function's name; to make it easier to select the intended transform for preview, change the `__name__` attribute of generated transforms to produce meaningful names. For example:

```python
from transforms.api import transform_df, Output


def generate_transforms():
    transforms = []
    for output_dataset_name in ["One", "Two", "Three"]:
        @transform_df(
            Output(f"/output/path/{output_dataset_name}"))
        def my_transform(ctx, output_dataset_name=output_dataset_name):
            # by default, generated transforms would be named `my_transfo
            cols = ['id', 'value']
            vals = [
                (0, f'{output_dataset_name}'),
                (1, f'{output_dataset_name}'),
                (2, f'{output_dataset_name}')
            ]
```

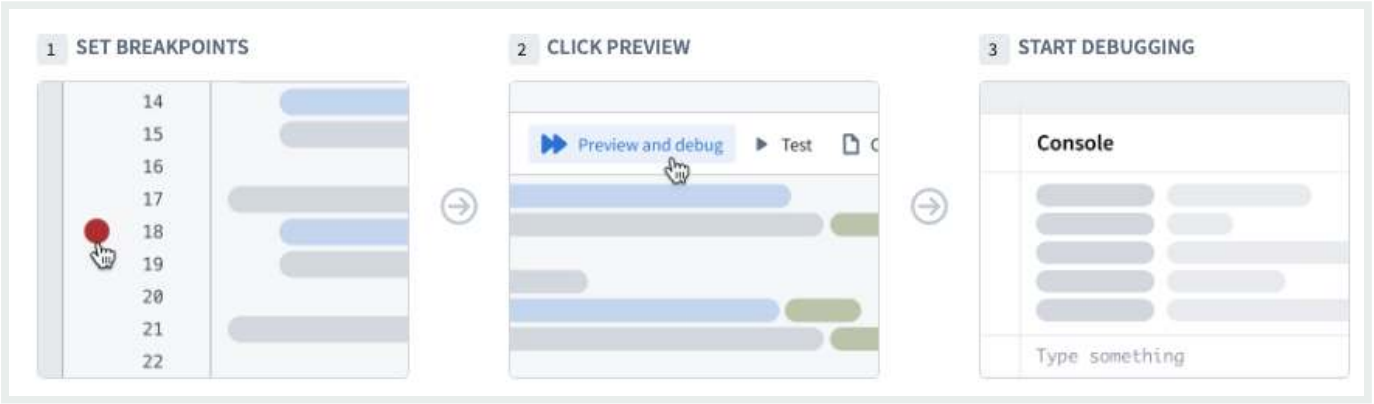
```
            df = ctx.spark_session.createDataFrame(vals, cols)
            return df
        transforms.append(my_transform)
        transforms[-1].__name__ = f'{output_dataset_name}_{transforms[-1]
    return transforms


TRANSFORMS = generate_transforms()
```

PREVIOUS
← Create transforms

NEXT
Debug transforms →

Debug transforms

Use the debugger tool in Code Repositories to examine your data transformation behavior while it runs. Set breakpoints to pause the execution of the transform in order to examine variables, view dataframes, and understand functions and libraries.



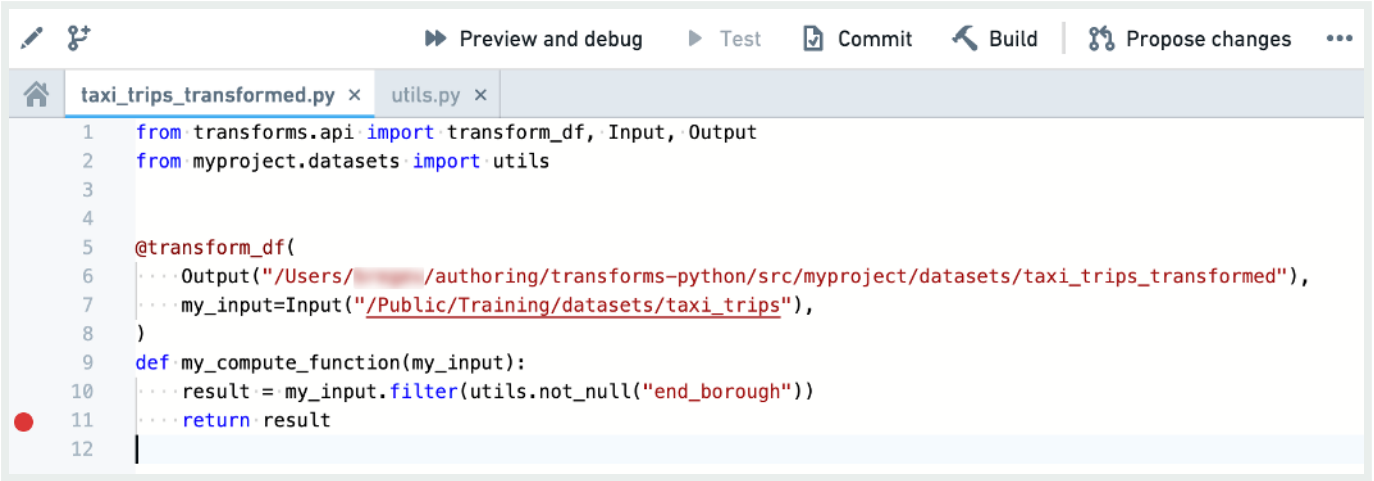
> The debugger is only available for Python.

Setting breakpoints

To use the debugger, you need to set *breakpoints*. Breakpoints signal to the debugger the points where it should pause the execution of the code and allow you to interact with variables and dataframes.

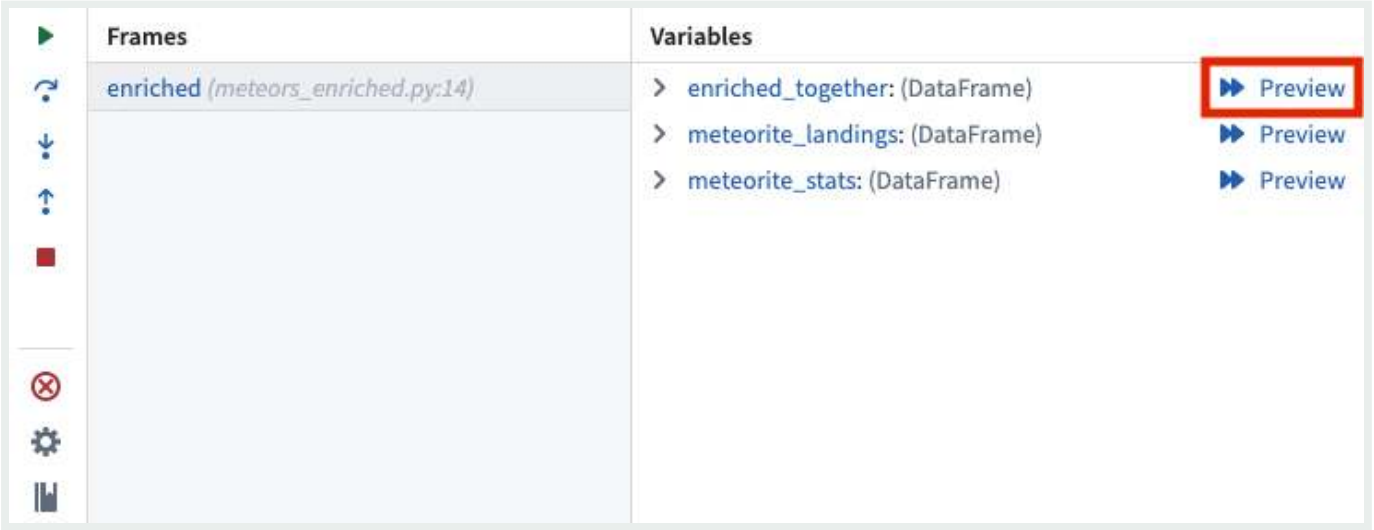
You can set a breakpoint by clicking on the faded red dot in the margins of each line of code. This suspends the execution **before** the marked line runs. You can set multiple breakpoints across several files if needed.

> The console functionality might be limited when using internal library breakpoints. In these cases the breakpoint is colored in grey and the debugger offers to either ignore the breakpoint or use limited console functionality.



Running the debugger

After adding breakpoints in your code, click on **Preview and debug** in the code editor actions bar. The debugger panel opens and pauses on the first breakpoint it encounters. The left bar of the debugger allows you to navigate the code, remove breakpoints, and finish/stop the debugging session.



Y API Reference ↗ bility to navigate internal libraries. Locate the **Internal libraries debugging is disabled** section, and select **Enable internal libraries d**

As you navigate in the code, the editor highlights the line of code to be executed next. Use the following buttons to advance the debugger:

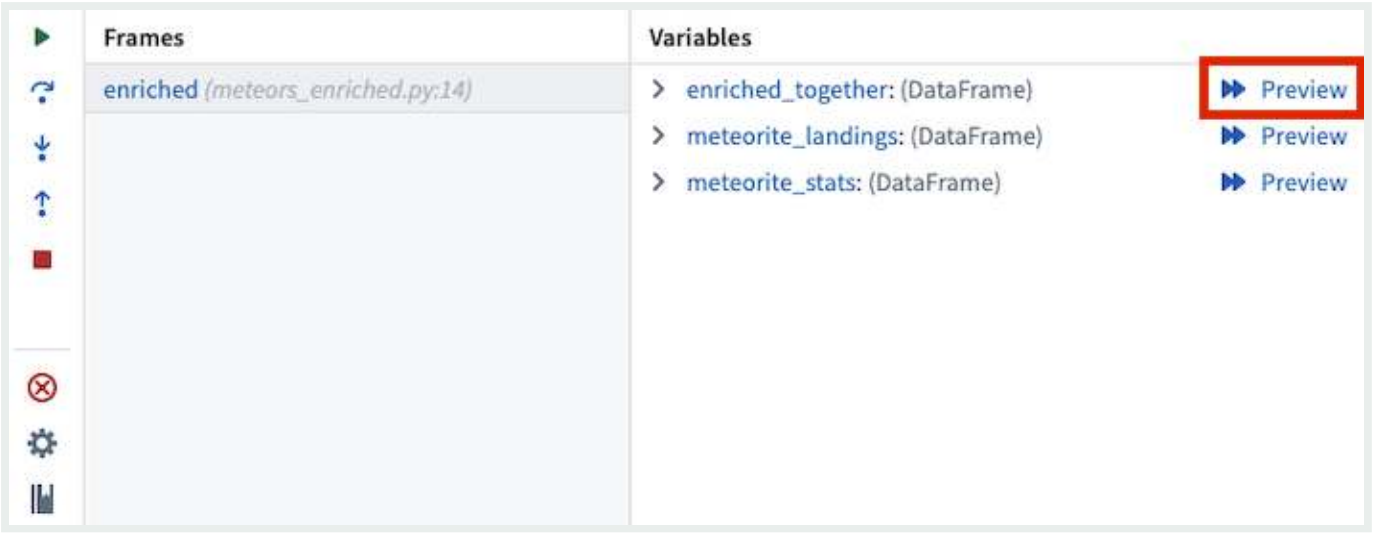
1. **Resume execution:** Continue execution until completion or until paused by the next breakpoint.
2. **Step over:** Execute the line of code without stepping into internal functions.
3. **Step into:** Navigate into internal functions if they exist in that line of code.
4. **Step out:** Navigate out of an internal function and advance the debugger.
5. **Stop execution:** Stop the debugger completely.
6. **Remove breakpoints:** Remove all breakpoints from the repository and run the preview without pausing the execution.
7. **Settings:** Toggle the debugger on/off (without clearing the breakpoints).
8. **Documentation:** Open the documentation for additional details.

API Reference ↗

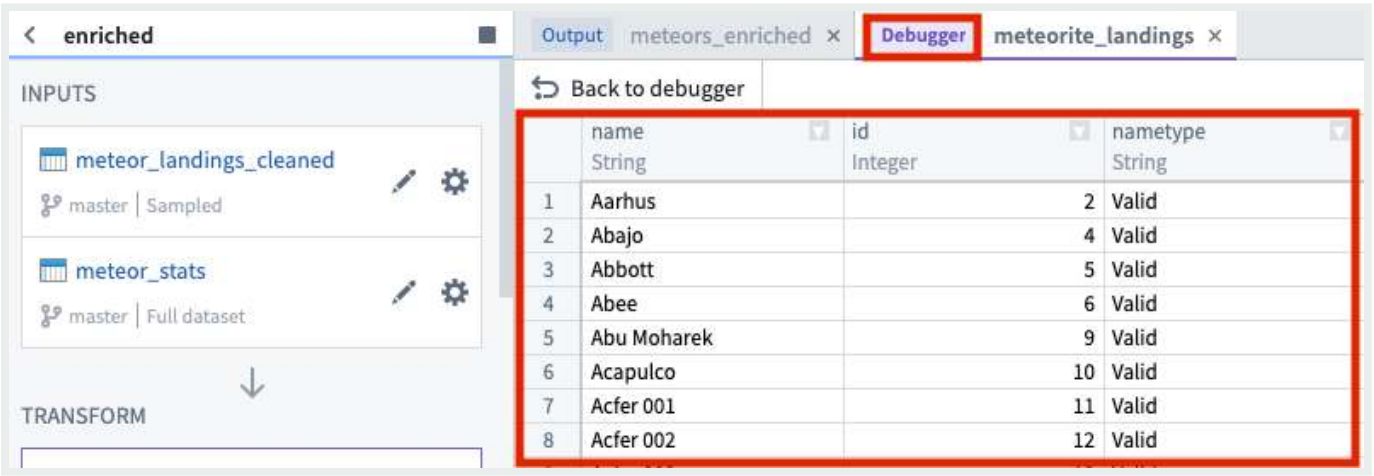
Previewing dataframes

Send feedback

When running the debugger, you can also preview an intermediate dataframe at each breakpoint. To do this, select **Preview** in the variables view:

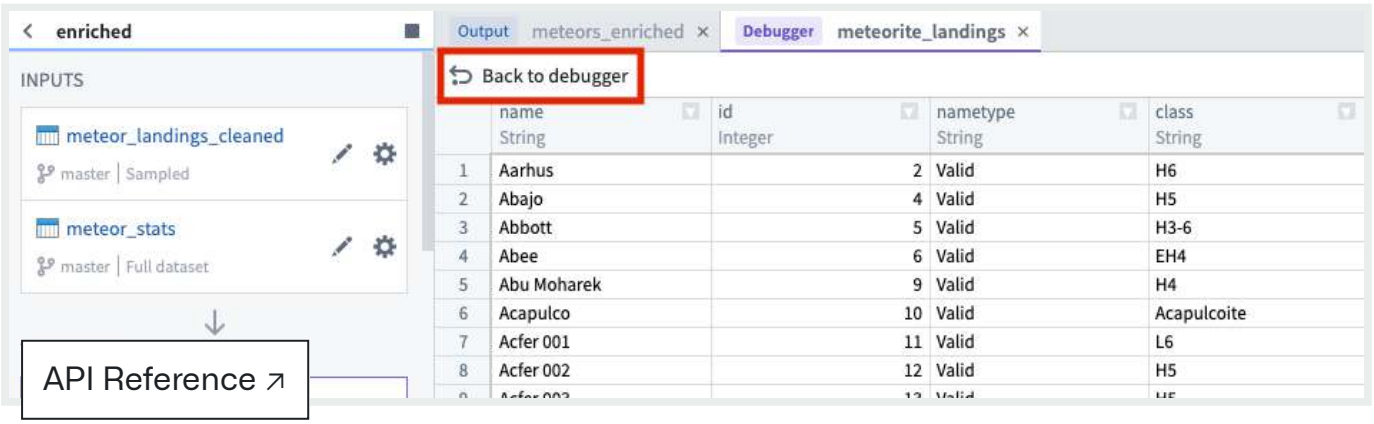


Selecting **Preview** will open a debugger preview panel for the selected dataframe:



	name String	id Integer	nametype String
1	Aarhus	2	Valid
2	Abajo	4	Valid
3	Abbott	5	Valid
4	Abee	6	Valid
5	Abu Moharek	9	Valid
6	Acapulco	10	Valid
7	Acfer 001	11	Valid
8	Acfer 002	12	Valid

To return to the debugger, select **Back to debugger**:



	name String	id Integer	nametype String	class String
1	Aarhus	2	Valid	H6
2	Abajo	4	Valid	H5
3	Abbott	5	Valid	H3-6
4	Abee	6	Valid	EH4
5	Abu Moharek	9	Valid	H4
6	Acapulco	10	Valid	Acapulcoite
7	Acfer 001	11	Valid	L6
8	Acfer 002	12	Valid	H5

Examining variables

While the debugger is running you can examine the variables and data at the exact point of code execution.

Frames

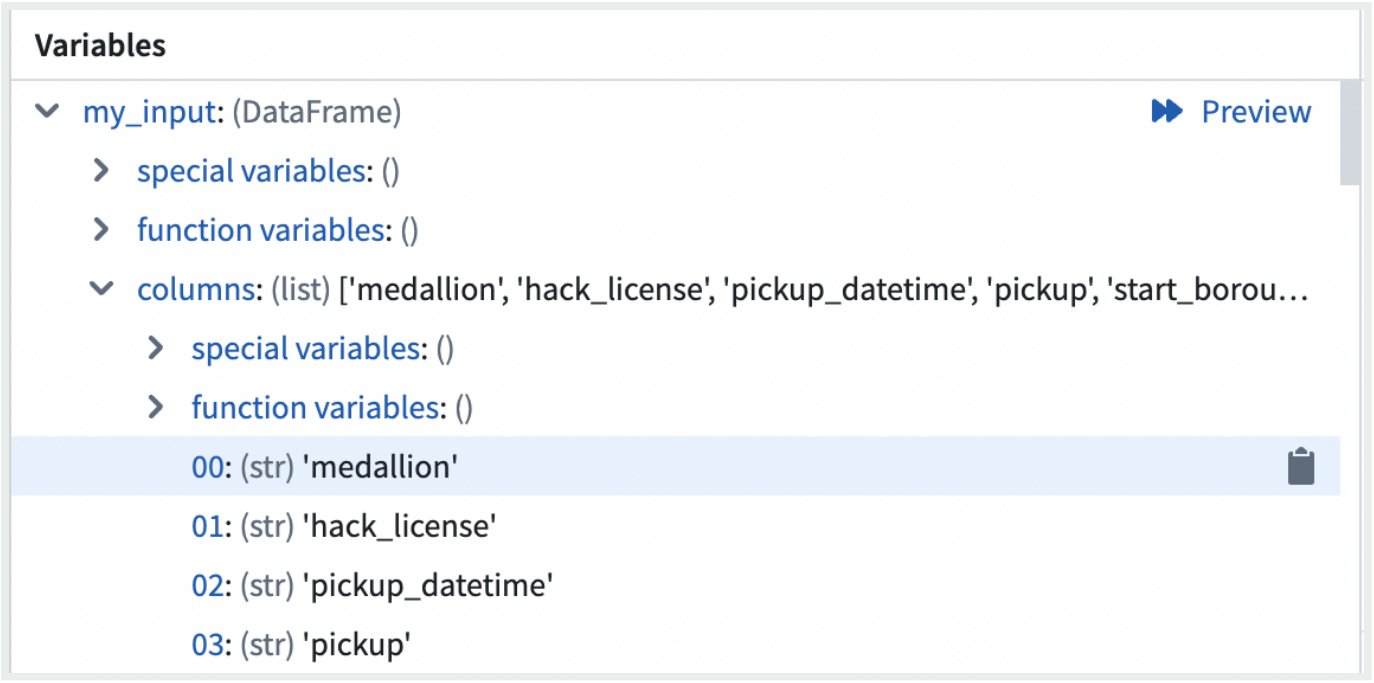
Frames represent the functions in which the debugger is active or breakpoints exist. Each frame indicates the name of the function followed by the name of the file and the line number in which the function is written.

Select a frame to examine the variables within that frame and run console commands against it.

Variables

The *variables* section shows the values stored in both local and global variables while the transform is executed.

> Dataframe values are based on the preview sample and may not represent the full dataset. Use them to understand and debug your code but not as an indication for the transform output.

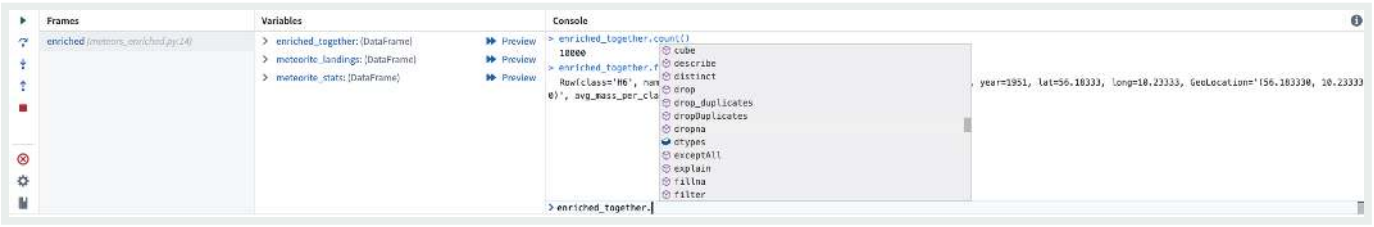


Console

The console allows you to interact with your data using PySpark commands while running the debugger. There are two commonly used patterns in the console:

- Running commands against dataframes and variables directly in the command line at the bottom of the console tab while initiating commands with the Enter or Return key.
- Calling the `print` function in the transform code to send indicative information to the console.

> Notice that the console runs against the selected frame. Trying to execute commands on variables local to a different frame will result in a NameError.



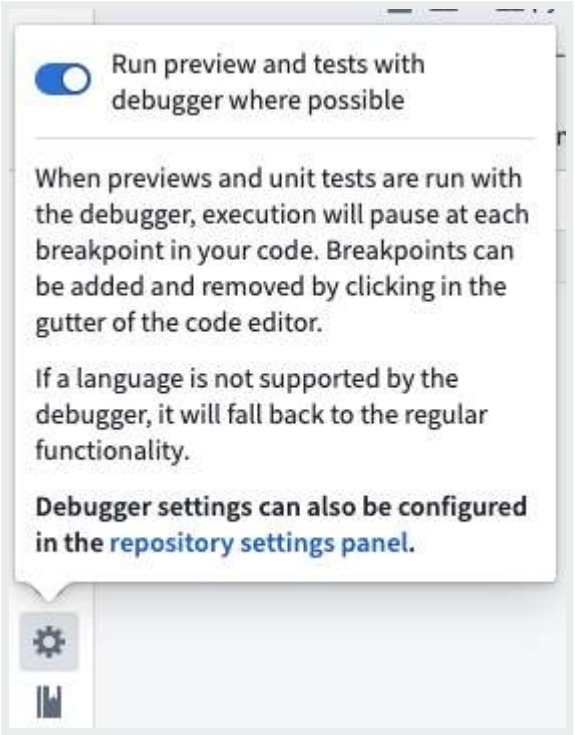
API Reference ↗

Configuring the debugger

Send feedback

Toggle the debugger functionality on and off by navigating to the debugger tab and clicking on the settings cog. Turn the debugger off if you want to run previews without stopping on breakpoints.

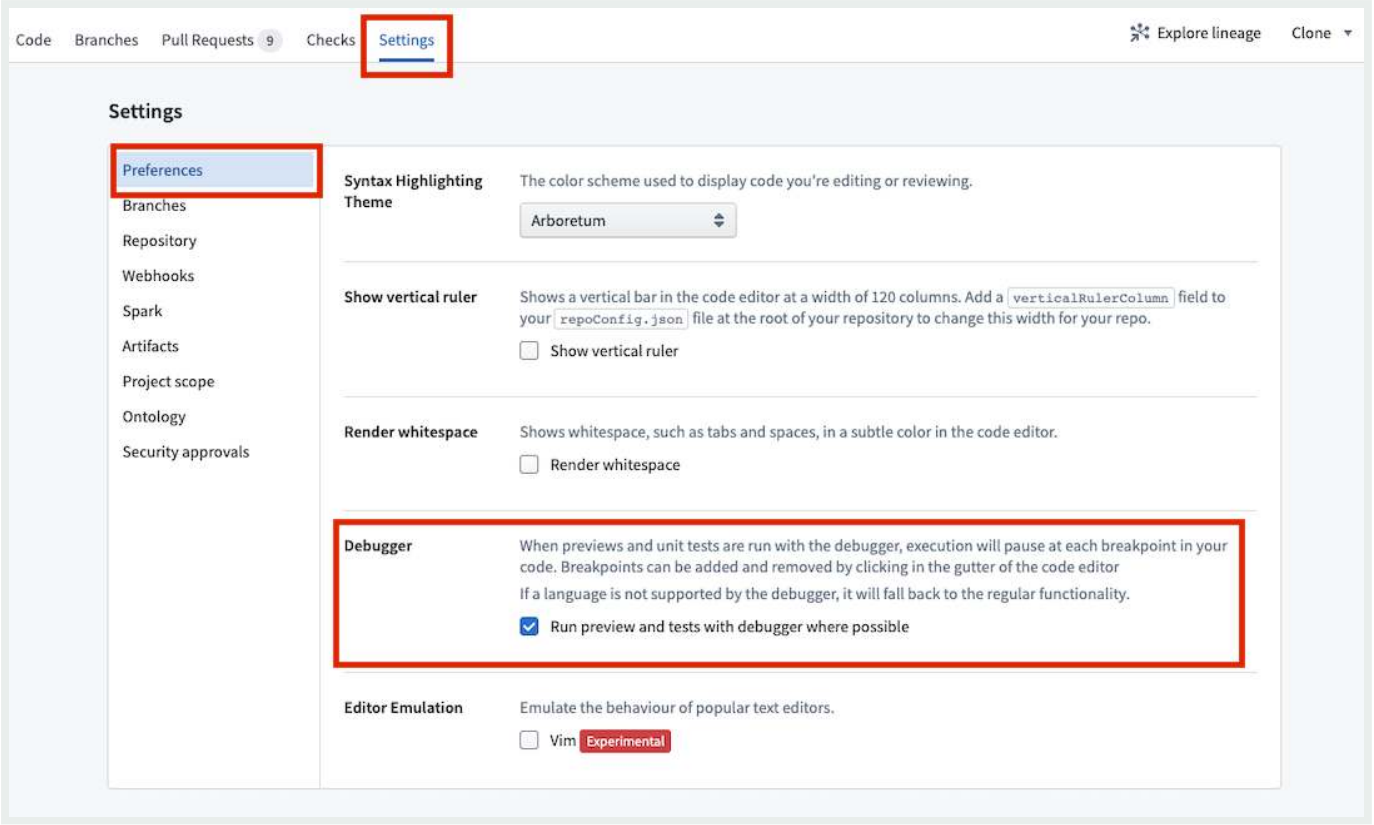
> While the debugger configuration applies to the entire repository, there might be languages in the repository that are not supported by it. If a language is not supported by the debugger, previews will continue to function normally regardless of the debugger setting.



You can also configure the debugger in the **Settings** tab under **Preferences** > **Debugger**.

API Reference ↗

Send feedback



← PREVIOUS
Preview transforms

NEXT
Use project references →

Use Project references

Project references provide a mechanism for users with higher permissions - typically the owners of a dataset or pipeline - to allow the discovery and use of their data in other Projects. Intermediate Projects can be used as permission boundaries or discovery hubs, wherein the users who own one part of a pipeline can declassify, curate, or otherwise distribute shareable versions of their datasets and make them available to downstream consumers.

Project references add an increased layer of scrutiny for moving data between Projects by explicitly acknowledging when a dataset is exported from or imported to a Project.

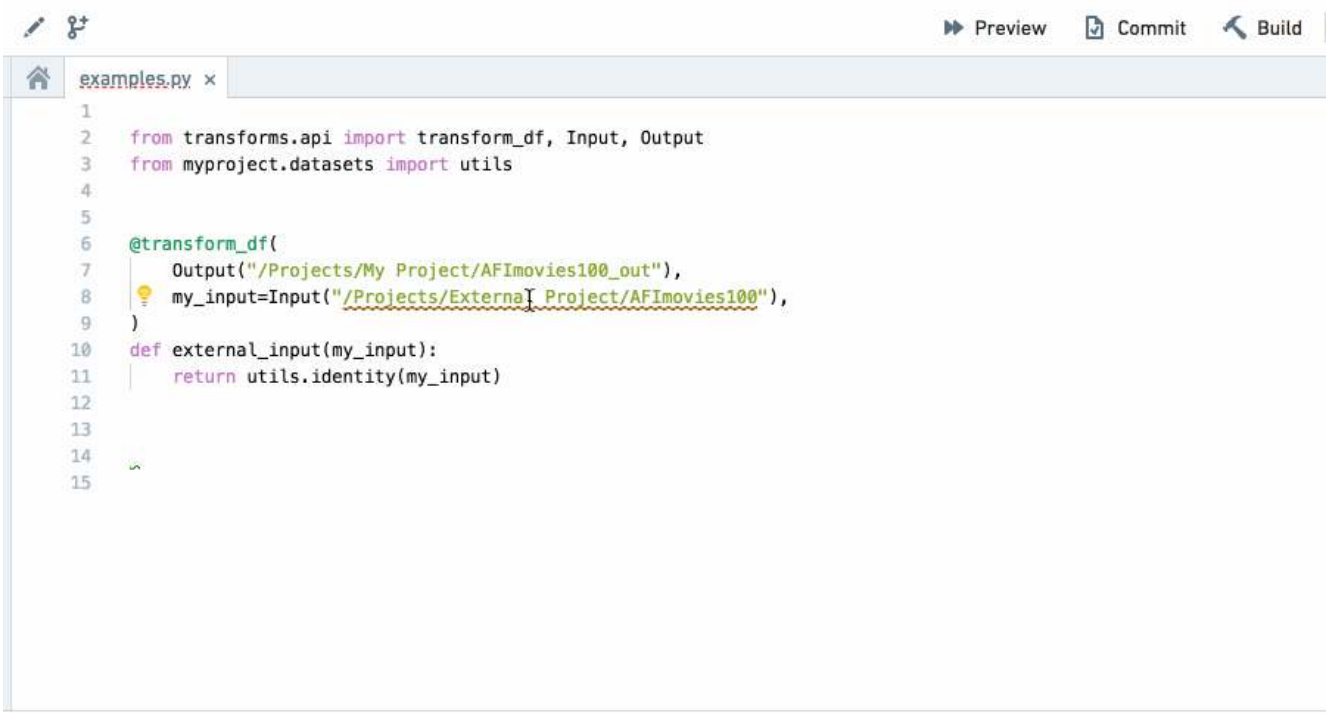
Instructions

1. The code editor informs users that outputs generated from transforms must be within the Project scope of the code repository. For example, if your code repository is in a project called "Data Cleaning Project", the outputs from your code repository can only be in "Data Cleaning Project", not any other project.

```
5
6   @transform_df(
7       Output("/Projects/External Project/AFImovies100_out"),
8       my_input=Input("/Projects/My Project/AFImovies100"),
9   )
10  def external_input(my_input):
11      return utils.identity(my_input)
12
```

2. input dataset from outside the Project scope, the code editor flags that the dataset must be added as a Project reference. By clicking the light bulb icon, the

input dataset can be added as a Project reference.



3. References to datasets external to the Project can also be added from **Projects & files**.
4. If the repository's language packages are out of date, it will not be possible to enforce Project scope until they have been updated. Code Repositories will flag that these need to updated and clicking the update button that appears will resolve this.
5. You can only add a Project reference to a dataset from outside the Project scope as an input datast, not as an output dataset. Trying to output to a dataset outside of the Project scope will trigger a AccessOutsideProjectDenied error.

Project references and permissions

To reference a resource, you must have `compass:import-resource-from` on the resource (usually expanded from the Viewer role) and `compass:import-resource-to` on the destination Project (usually expanded from the Editor role). These permissions can be customized using custom roles.

A user needs access to all Markings related to datasets referenced in a code repository in order to modify code in that repository.

API Reference ↗

← Debug transforms

NEXT

Analyze th →

Analyze the impact of changes

Code edits may result in unexpected changes to dataset content, permissions, and structure. We recommend protecting production branches and requiring a review of proposed changes before merging; these options can be found in the branch settings. The pull request page will then offer various ways to analyze the impact of code changes on the affected datasets.

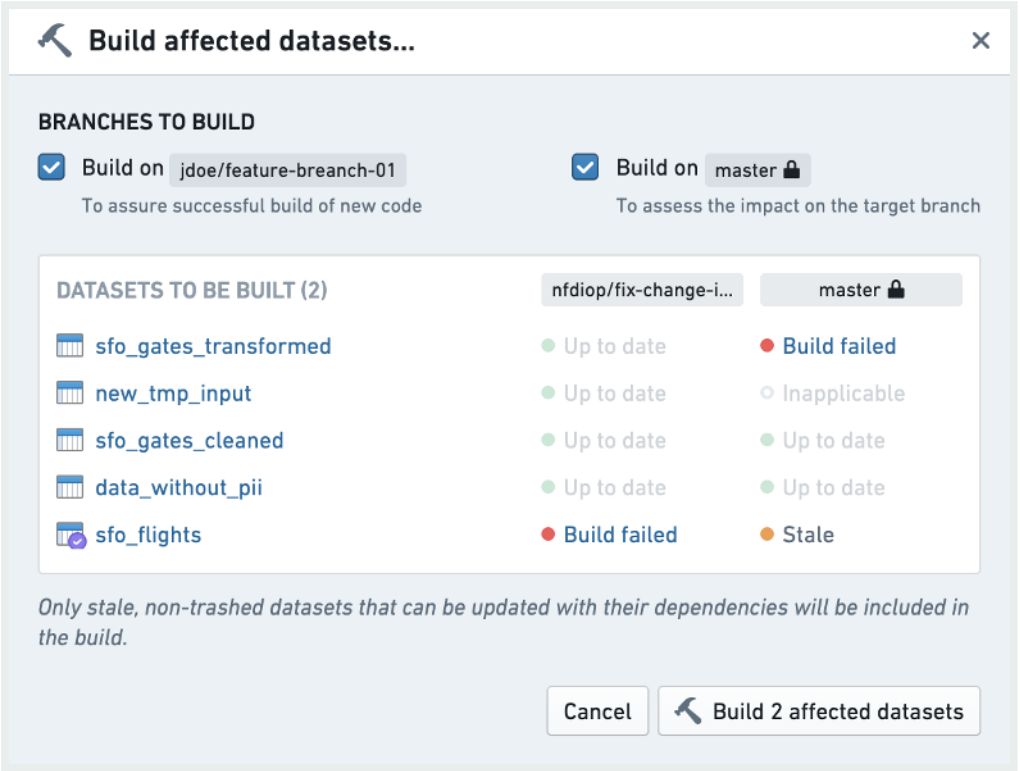
Stale datasets

Evaluating the impact of changes requires the affected datasets to be built with the most recent code changes. This build is required on both the head branch (development) and the base branch (target):

- **Build on head branch** (**development**) to validate that the code builds properly, the outputs appear as expected, and that all Data Expectations are met.
- **Build on base branch** (**target**) to compare the outputs to the latest version of the target.

The pull request page will warn you when affected datasets are stale. Clicking on **Configure and build** will allow you to review the stale datasets and build them on head and base branches.

> **Warning**
>
> The pull request page's staleness warning only covers affected datasets within a repository. The pull request page does not warn about stale parent datasets outside of the repository or about uncommitted changes.



Impact analysis

The **Impact analysis** tab provides information on datasets affected by the pull request. By default, it will only show directly affected datasets, excluding derived datasets that may be impacted. You can inspect the impact on additional tables by clicking on **Add datasets to analysis**.

> Reviewing affected datasets requires access to the data. Inaccessible datasets will be indicated in the UI as inaccessible.

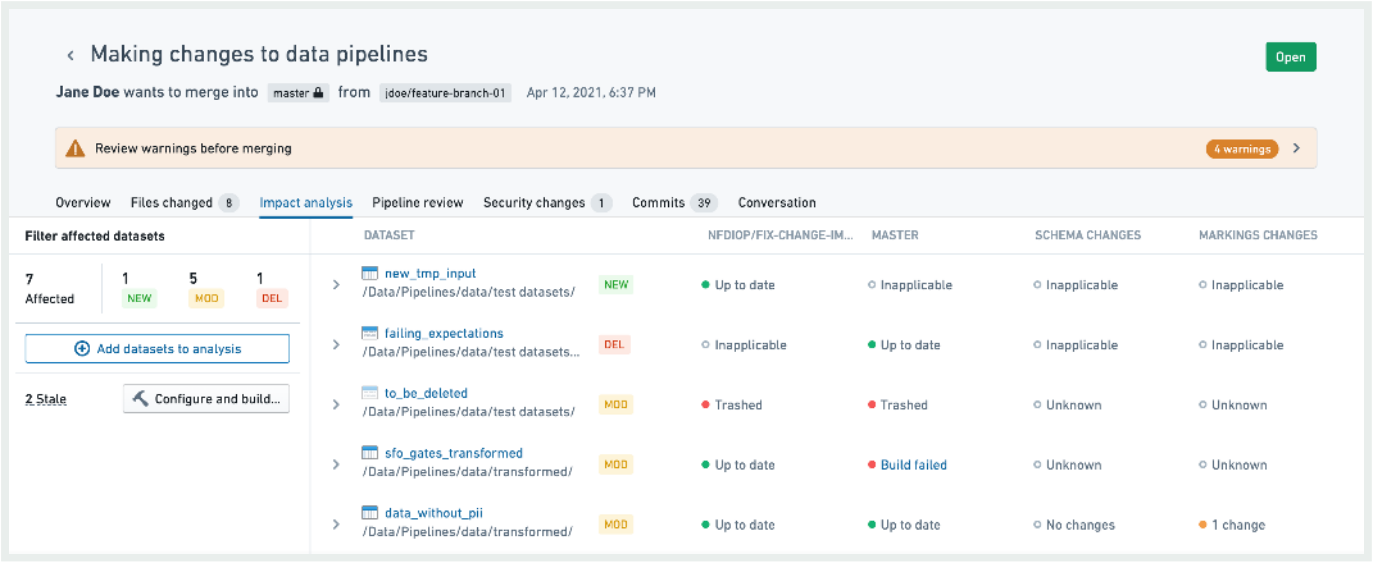
> The way a repository determines the list of affected datasets can differ depending on the language you use. Python repositories use Transforms Level Logic Versioning (TLLV) to generate the list. In Java transforms, datasets are considered directly affected if their source file is changed by the pull request.

API Reference ↗

Send feedback

The impact analysis tab shows the following information:

- **Code** - View changes to the source file (it will not include other files that may be referenced or used).
- **Schema** - View changes to dataset columns.
- **Security** - View changes to markings applied to the output dataset.
- **Expectations** - View the data expectations on the head branch.
- **Trashed datasets** - Trashed datasets that are updated by the PR will appear faded.



View impact on derived datasets

Clicking on **Add datasets to analysis** will analyze the pull request's impact on derived datasets. All intermediate datasets between the selected dataset and the affected datasets will be added as well.

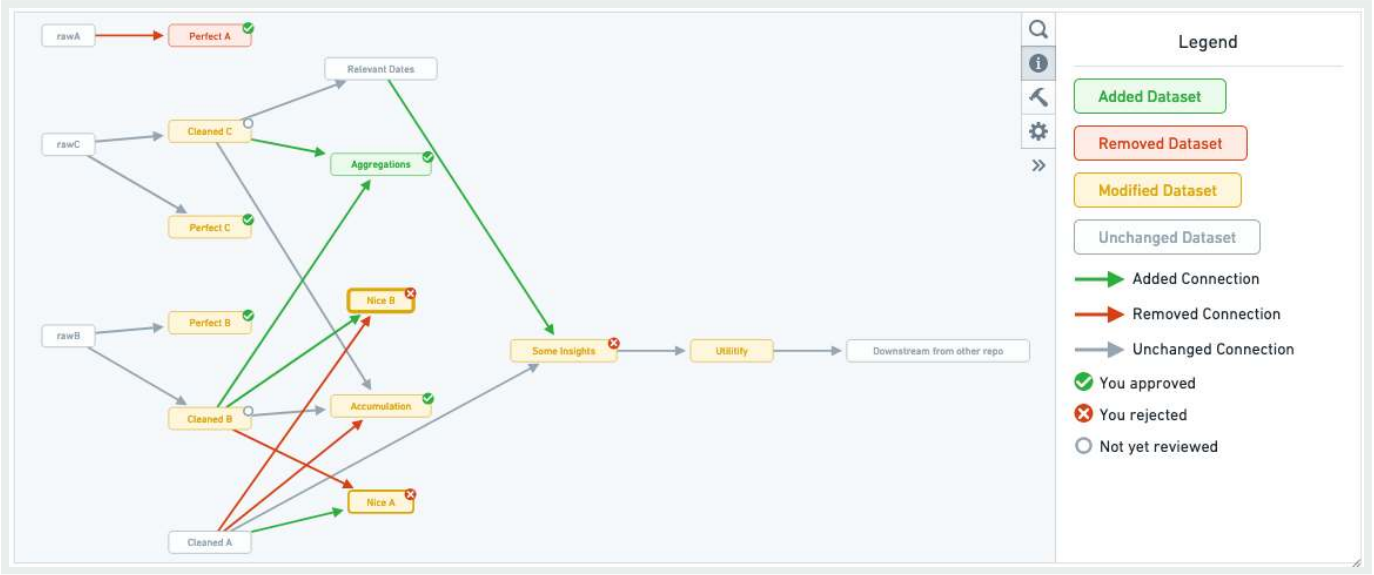
The added datasets must not be stale in order to show impact information. Click on **Configure and build** to view an updated list of stale datasets and trigger a build.

Pipeline review

The pipeline review tab offers a lineage view of the datasets affected by the pull request.

API Reference ↗

Send feedback



When one of the dataset nodes is selected, the changes to the transforms code file that generates this dataset is displayed along with the schema changes for each dataset generated by that code file. Navigating through the affected datasets in the order of data flow can help you understand how changes to different files and datasets correspond.

To keep track of your progress while reviewing the changes in a pull request, you can approve or reject each file individually. For transform source files, this review status is shown in the pipeline graph as an indicator on the corresponding output dataset nodes.

PREVIOUS
← Use project references

NEXT
Unit tests →

Unit tests

Code Repositories support discovering and running unit tests through an integrated helper that supports most languages and repository types. For more details, visit the unit testing documentation for each language:

- Python transforms unit tests
- Java transforms unit tests
- TypeScript functions unit tests

PREVIOUS

← Transforms / Analyze the impact of changes

NEXT

Pin Spark modules in-platform →

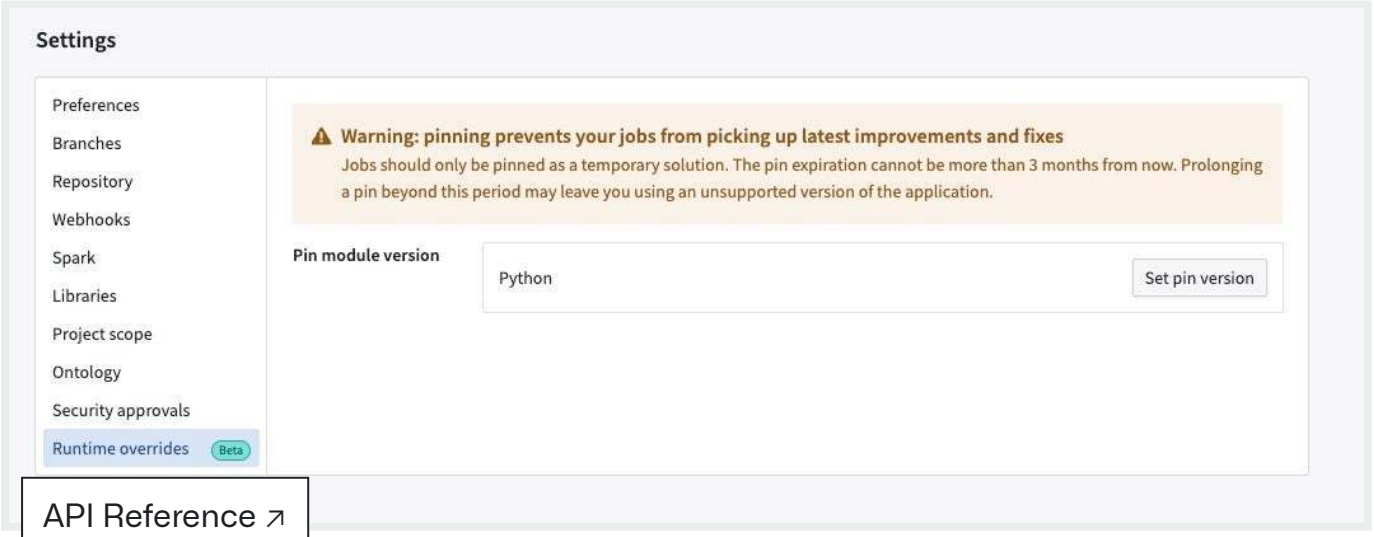
Pin Spark modules in-platform

Code Repositories allows you to pin a Spark module for a repository. This enforces the usage of a specific version of Spark. You can pin a Spark module for both new and existing builds.

> We recommend that you always use the newest version of Spark for the most up-to-date performance and security features. Pinning a Spark module should be a temporary measure.

How to pin a Spark module

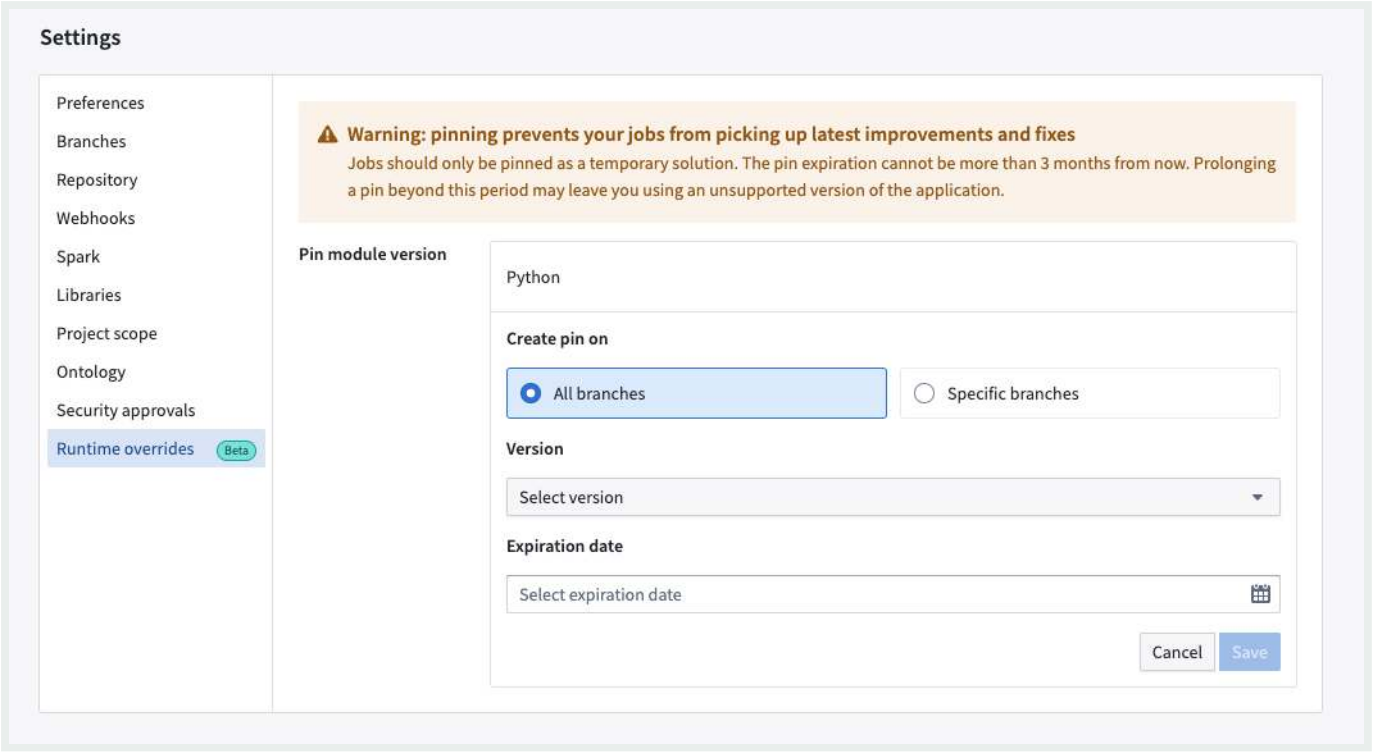
Navigate to the **Settings** tab in Code Repositories, then select **Runtime overrides**.



API Reference ↗

Create a pin

From the **Runtime overrides** tab, select **Set pin**.



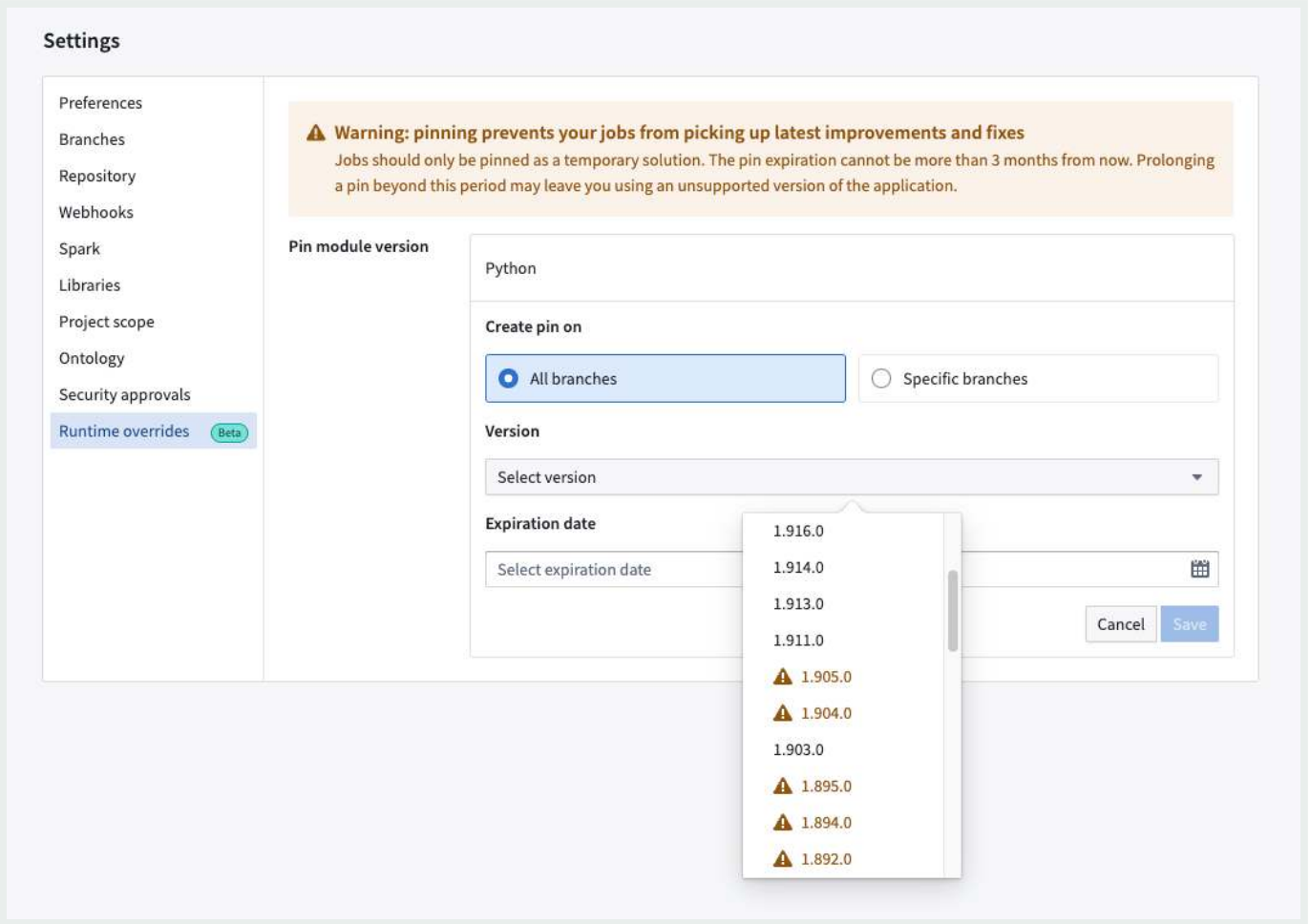
You can create pins on all branches or on specific branches. You can select a Spark module version (for each module type available in the given repository) you want to pin and also specify an expiration date. The expiration date cannot be more than 90 days beyond the current date. The pin expires at the end of the 90-day period or the expiration date you specify, whichever is earlier. After the expiration date, your builds may fail.

Create a pin on all branches

First specify the Spark version you want to pin and the expiration date.

API Reference ↗

Send feedback



Unstable and non-recommended versions have a warning sign prefixed as we do not recommend using them unless absolutely necessary.

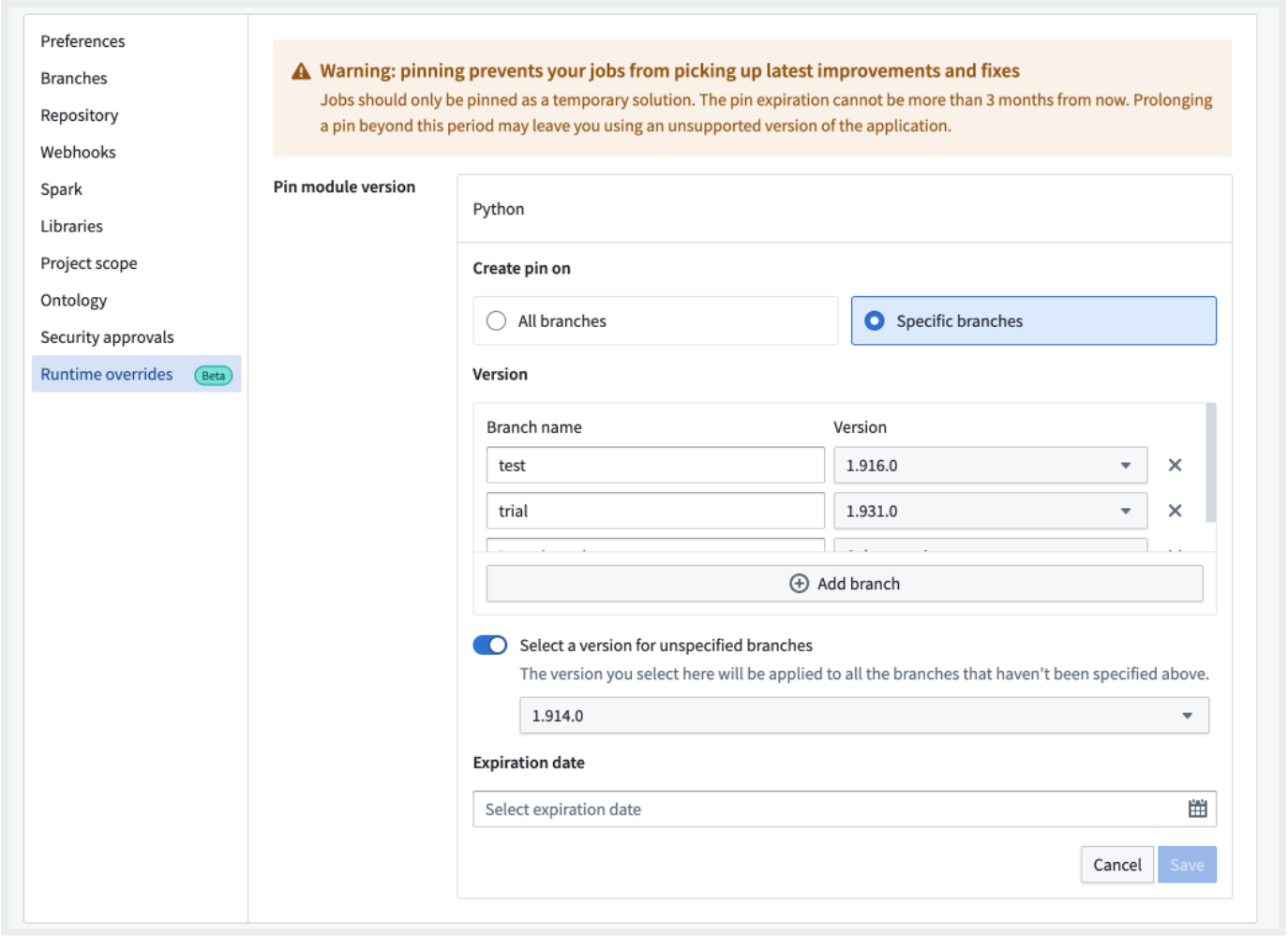
When you are finished, select **Save** to set the pin.

Create a pin on specific branches

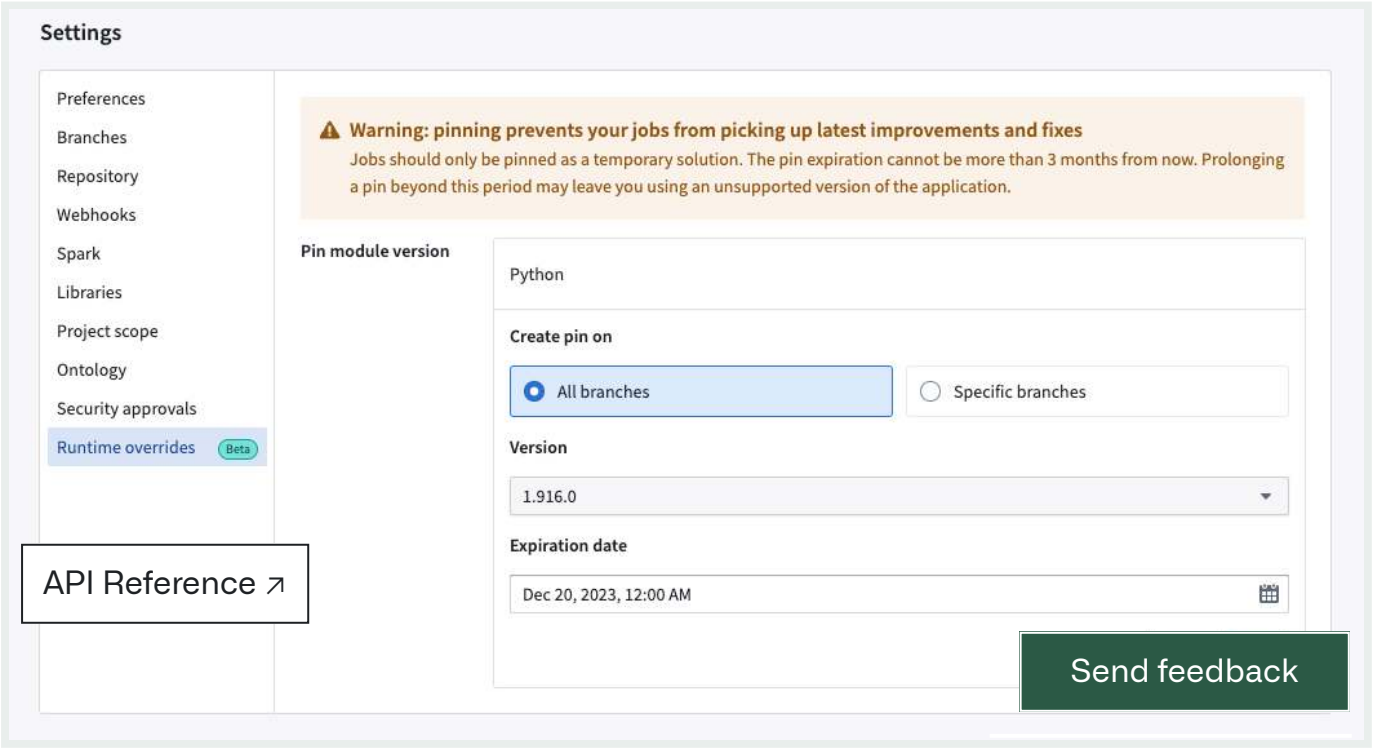
You can pin versions for each branch. You can also select versions for unspecified branches.

API Reference ↗

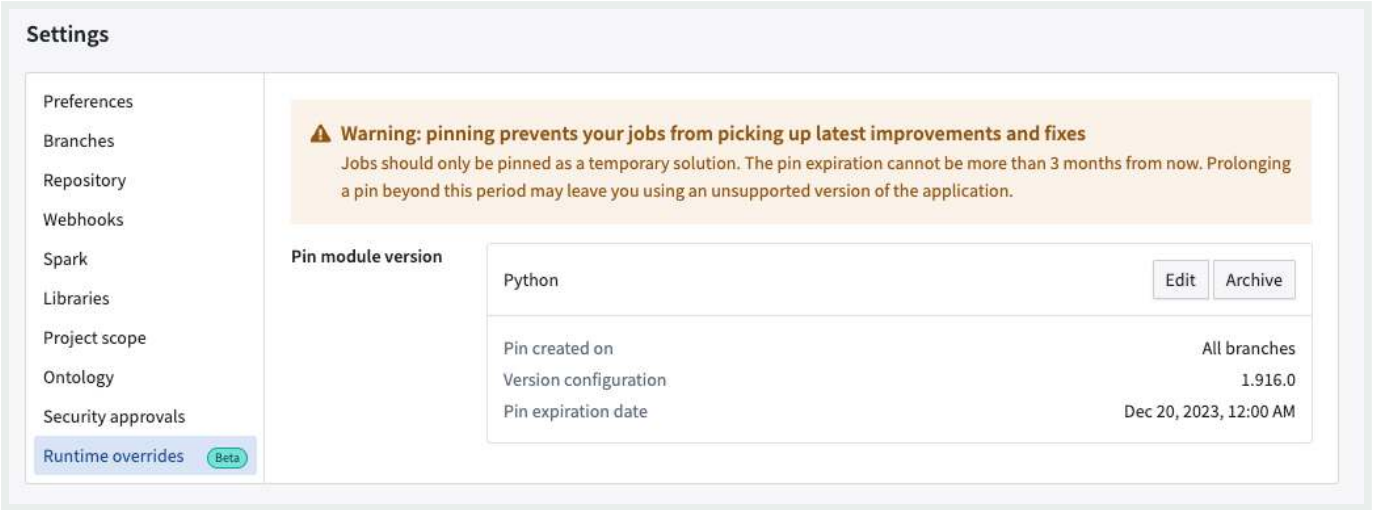
Send feedback



When you are finished, select **Save** to set the pin. In the below sample screenshot, we create a pin on all branches for version 1.916.0, which is set to expire on December 20, 2023 at 12:00 AM.

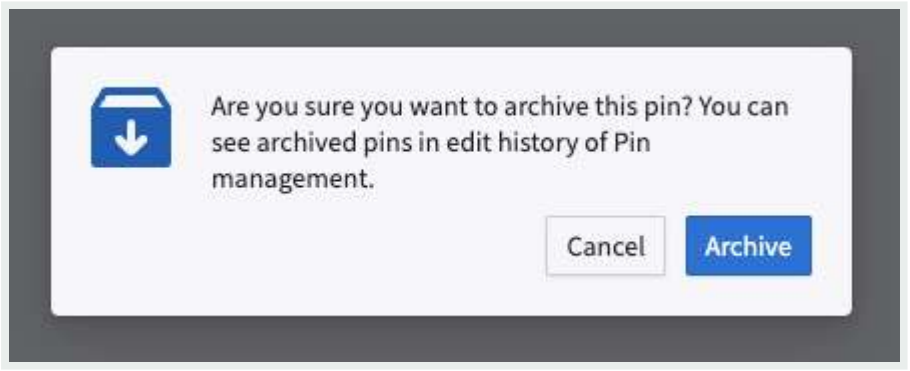


After setting the pin, you can view a confirmation that the pin was created.



Edit existing pins

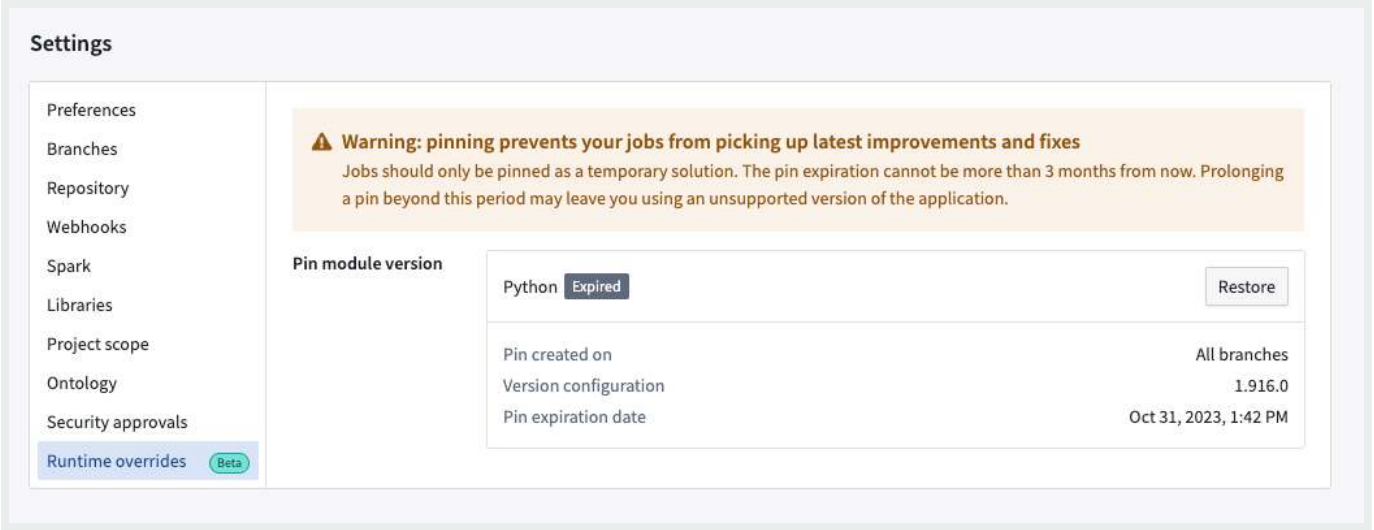
Selecting **Edit** lets you modify a pin. You can change the branch, version, and expiration date using the same workflow for creating a pin. Selecting **Archive** will remove the pin and add the `Expired` label to the pins you previously set.



You can select **Restore** to re-create the pins you archived.

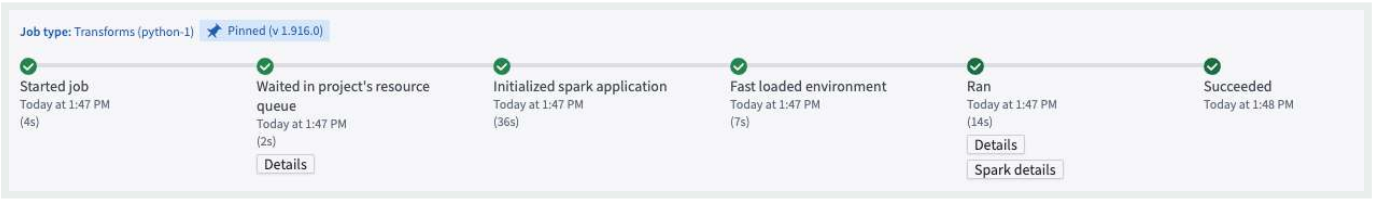
API Reference ↗

Send feedback



View your pin

On the **Build Preview > view details** page, you can view the version you pinned for the repository.



Use pins to control transforms versions in Code Repositories CI

When running continuous integration (CI) for a build in Code Repositories, a version of the transforms library is chosen based on the current version of the repository template. If a pin is applied, CI will instead use the pin's version of the library if it is higher than the minimum version declared by the template. Users can leverage pins for faster access to new versions of the API and new features for code assist and local development.

FAQ

API Reference ↗

Send feedback

1. Why does the pin expire in 90 days? Why can I not use it for longer periods?

We recommend that you always use the newest version of Spark. Not doing so means that you may be missing out on performance and security fixes and improvements which may have adverse impact on your builds.

That said, we understand that there may be specific use-cases that make pinning necessary and therefore offering this functionality with the caveat that it should be used on a temporary basis. Prior to the expiration date, we recommend that you make changes required on your end to be compatible with the newest version of Spark.

2. Can I get notifications via the UI when my pin is about to expire?

This feature is in development.

3. Can I pin individual builds via the Job Tracker UI?

This feature is in development.

4. Do I have to upgrade or make changes on my side to use this feature?

No upgrades or actions are needed from your side.

5. Is this feature available for new builds, or can I apply it to existing builds?

Y API Reference ↗ ure for both new and existing builds.

Send feedback

6. What happens if my job is already pinned in cdconfig?

The in-platform pin will always take precedence.

PREVIOUS
← Unit tests

NEXT
Libraries →

Libraries

Code Repositories allow you to use local and external libraries, as well as share your own code within your Foundry environment. Different languages require different steps to share and use libraries:

Python

- Share Python libraries
- Discover and use shared Python libraries

Java

- Share Java Code & Macros

PREVIOUS
← Pin Spark modules in-platform

NEXT
Documentation →

Documentation

You can provide users with documentation on projects in Code Repositories by adding a README file. README files in Code Repositories support Markdown for flexible, easy-to-use formatting and styling.

Edit or add a README.md file to your repository to get started. This page contains information on additional features that can be used to customize a README file.

If a README file is insufficient for your documentation needs, you can create custom documentation using Code Repositories that can be published as in-platform documentation.

Features

Code Repositories provides a number of formatting options for READMEs.

Inline image previews

You can display an image from your repository inline with the text of a Markdown file by using the following syntax:

```
![File Name](/transforms-python/path/to/my/file.jpeg)
```

In order to upload images to a code repository, you will need to clone your repository locally, a[illegible] the local repository, and then push the changes to the server.

Mentioning Foundry users

To mention a Foundry user in your Markdown files, enter their username with the `@` symbol as a prefix. This will create a reference to the mentioned user and a direct link to their profile.

`@username`

Referencing Foundry resources

You can reference any Foundry resource by pasting its Resource ID directly into the Markdown file for the README. Resources referenced like this will automatically be named and linked to the corresponding resource in platform.

`This repository will be deployed to ri.foundry.main.deployed-app.a00000aa-a000-000a-0000-000a0aa0a00a`

Linking to files in the repository

To create a link to a file within your repository, use the `repo://` protocol followed by the file path; for example, `repo://transforms-python/src/myproject/datasets/examples.py`. Files referenced like this will automatically open when clicked. This allows you to easily reference and navigate to other files within your repository.

Syntax highlighting

README files support language syntax highlighting in code blocks in order to improve code legibility. To use syntax highlighting, specify the language after the opening code block delimiter as follows:

```
def hello_world():
    print("Hello, World!")
```

T API Reference ↗

You can also create tables using standard Markdown table syntax:

```
| Header 1 | Header 2 |
| -------- | -------- |
| Cell 1   | Cell 2   |
```

Links

URLs and email addresses in a README will be automatically converted into clickable links.

PREVIOUS
← Libraries

NEXT
AIP features →

AIP features in Code Repositories

The following is a list of AIP-powered features in Code Repositories.

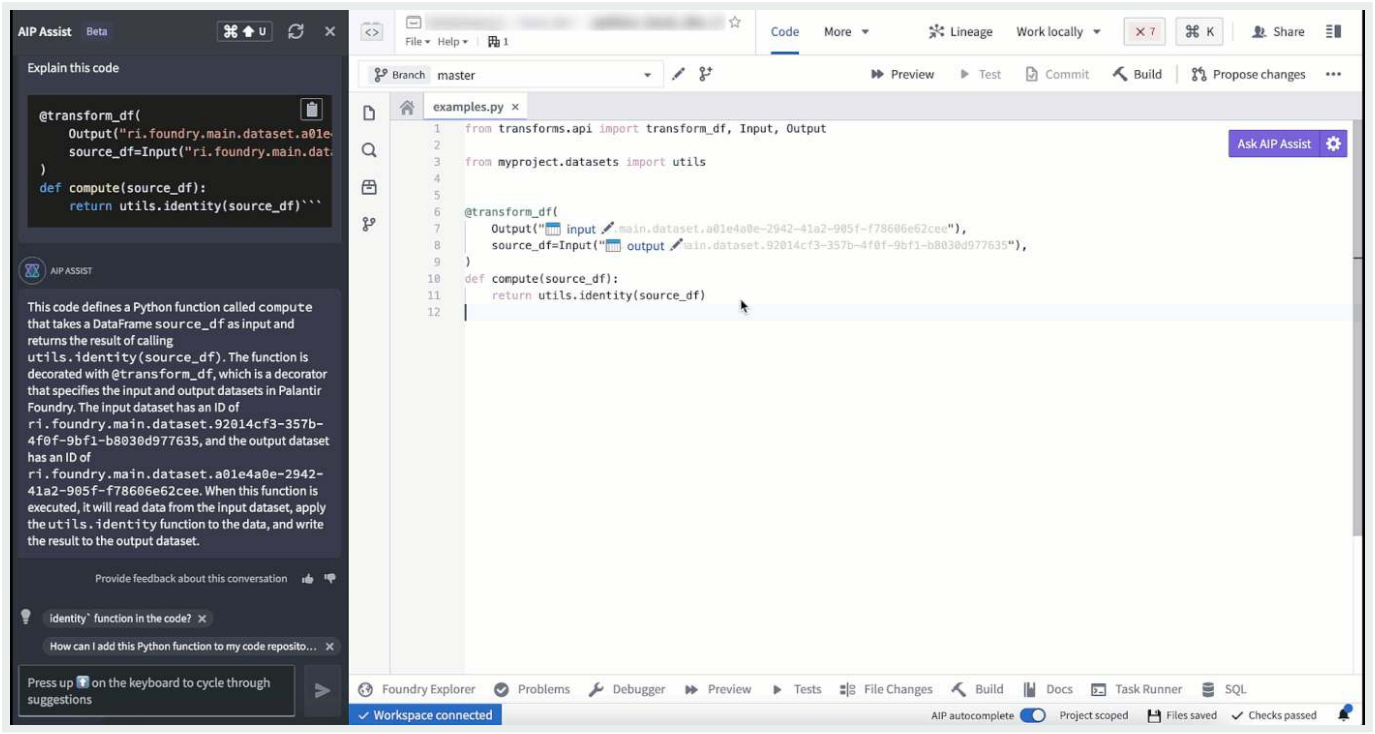
AIP Assist

Access AIP Assist features through the **Ask AIP Assist** option, through the editor's right-select menu, or through options inline and enhance your code comprehension using AIP Assist.

The AIP Assist features in Code Repositories can be configured by navigating to **Ask AIP Assist > Configure AIP Settings > AIP features**.

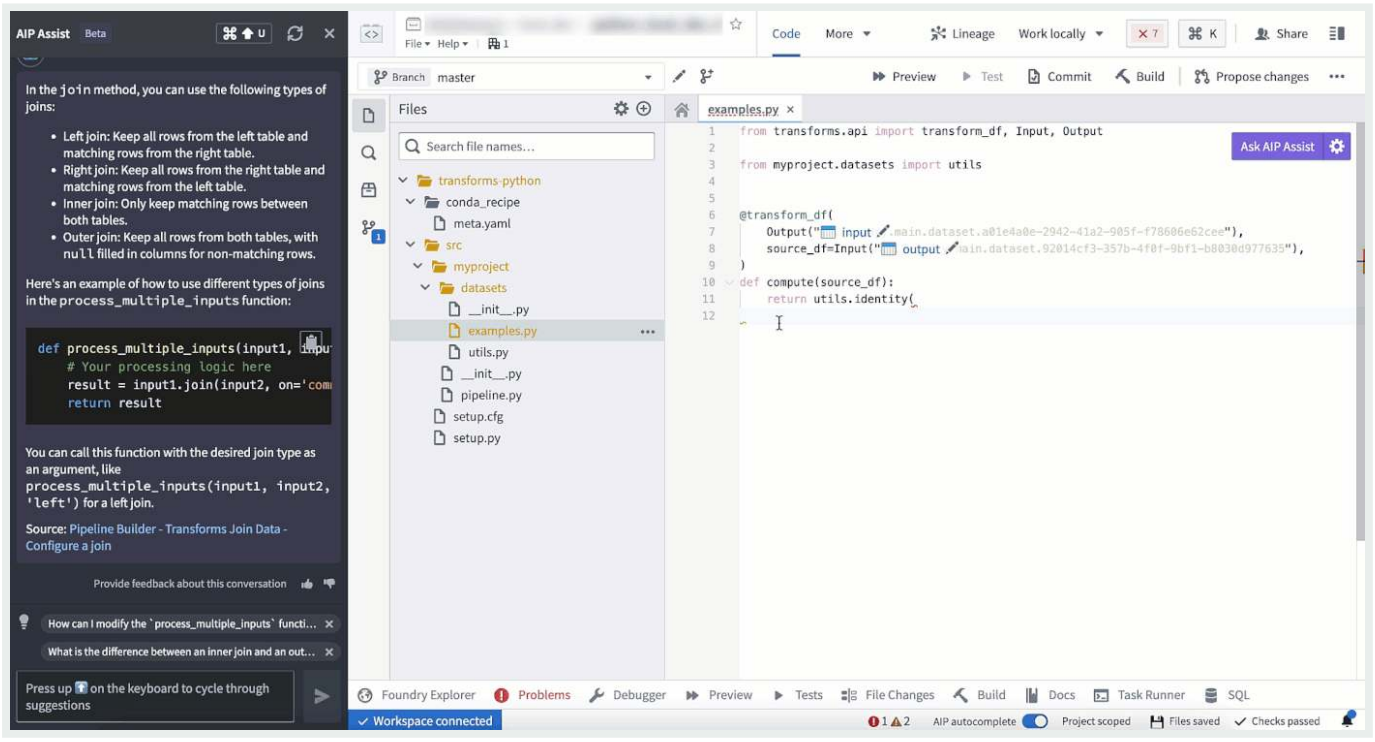
Code explanation

You can now use AIP in Code Repositories to explain the purpose of code snippets or entire files.



Find bugs

Use AIP in Code Repositories to check and debug your code.

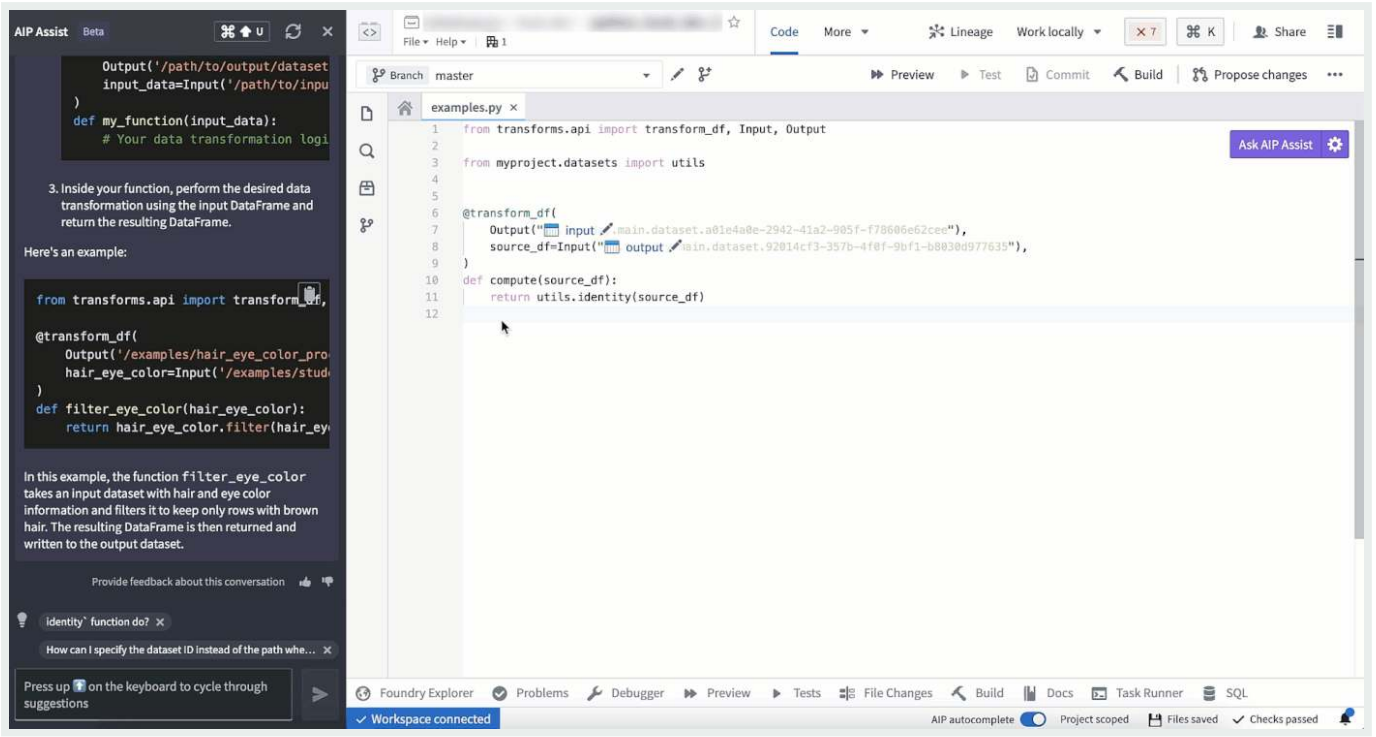


API Reference ↗

Translate

Send feedback

AIP in Code Repositories can convert your code to various languages such as Python, SQL, Mesa, or Java.



Context-aware attachments

Context-aware attachments allow users to attach code snippets, files, and repositories to conversations with AIP Assist. These attachments provide context that enriches AIP Assist knowledge and enables more accurate responses to code-specific questions. You can attach supported resources via the AIP Assist sidebar or from the **Ask AIP Assist** dropdown.

Code snippets can also be attached by highlighting the desired code, right-clicking, and selecting **Attach to AIP Assist** from the context menu or directly by using the inline code assistance options that appear above the selected code.

Inline code assistance [Beta]

> **Beta**
>
> API Reference ↗ istance is in the beta phase of development and may not be available on your enrollment. Functionality may change during a
>
> Send feedback

Code Repositories now features inline code assistance when a code snippet is highlighted. The **Explain**, **Find bugs**, and **Ask a question** options are displayed above the selected code snippet to enable users to access targeted help from AIP Assist directly from code.

This feature can be disabled by opening the **Ask AIP Assist** dropdown menu in the top right corner of the editor and selecting **Configure AIP Settings**. In the **AIP features** section, you can configure the **Show AIP Assist actions above selected code** option and other AIP features to suit your needs.

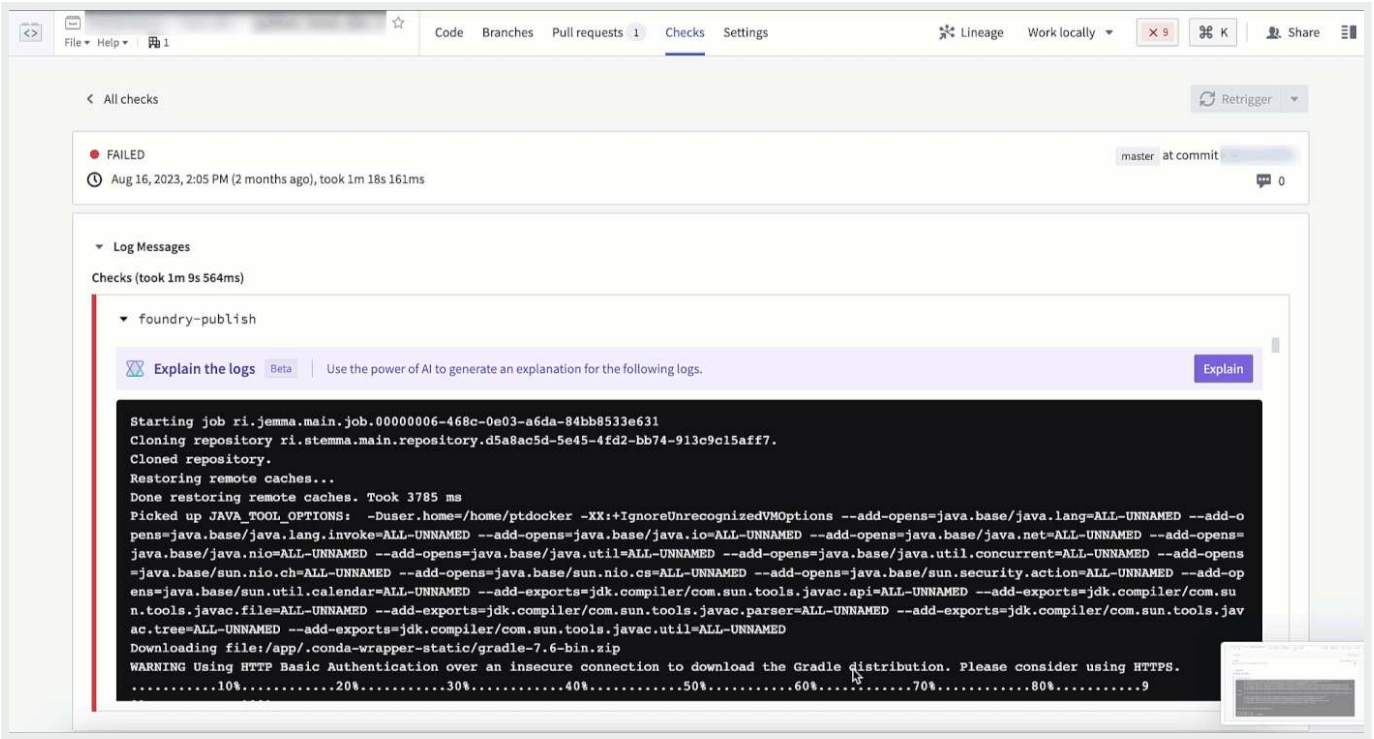
You can also access these AIP Assist features from the **Ask AIP Assist** dropdown menu or by right-clicking a highlighted code snippet and selecting one of the available AIP Assist options from the context menu.

AI error enhancer

The AI error enhancer provides comprehensive error explanations and suggested code changes where available, along with access to source documents for further information. This feature is available in the Foundry apps listed below:

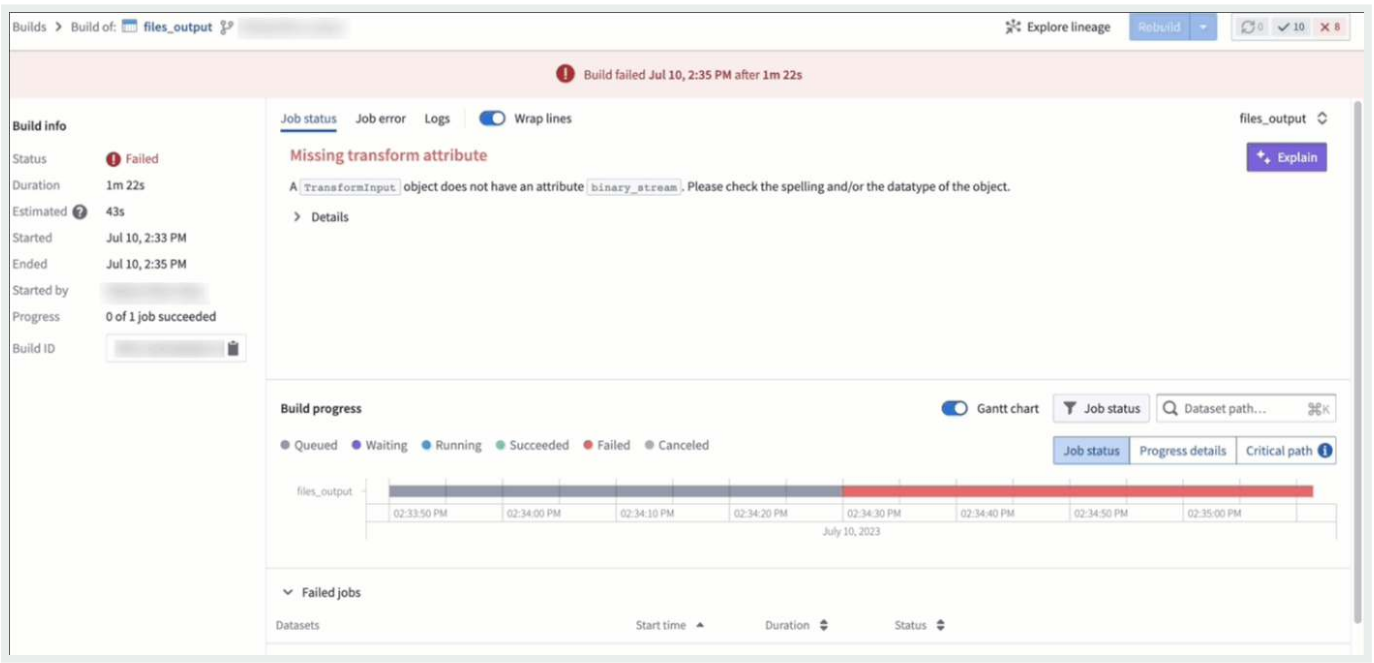
Code Repositories

In Code Repositories, the feature is shown in the **Checks** and **Preview** section.



Builds

In Builds, this feature is found in the **Job status menu**.



C omplete

API Reference ↗

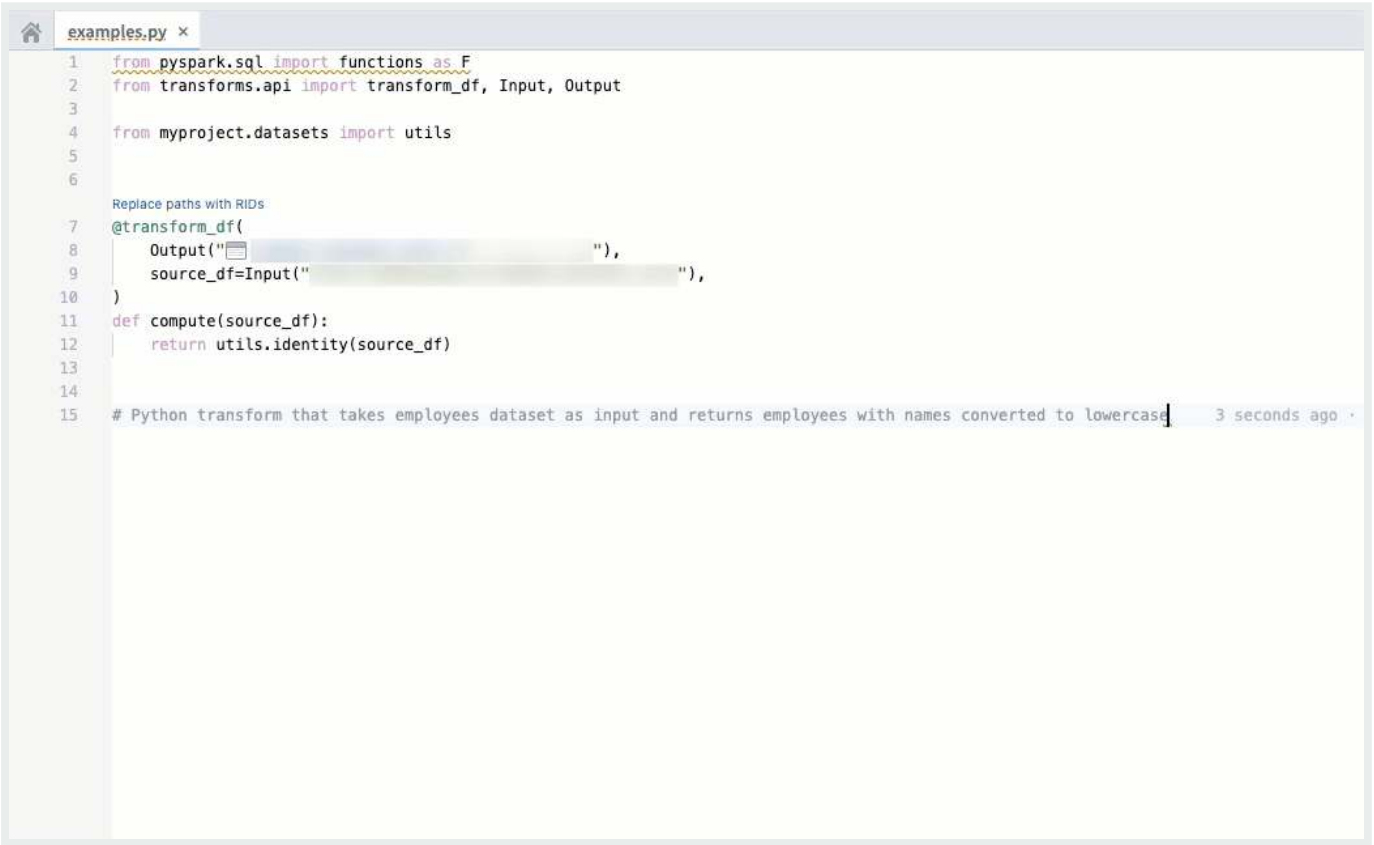
Send feedback

> The feature is currently exclusive to Python repositories.

Code autocomplete can help you generate code toward your objective by parsing your currently-opened file to automatically provide the relevant autogenerated code.

- To accept the suggestion, press `tab`.
- To ignore the suggestion, press `esc`, or start typing something else.

Code autocomplete can be toggled on or off directly in the editor status bar at the bottom of the window or by navigating to **Settings** (**purple cog icon**) **> Preferences > AIP Features**.



Note: AIP feature availability is subject to change and may differ between customers.

← API Reference ↗

NEXT

Add dataset transformation to a Marketplace product →

Send feedback

Add dataset transformation to a Marketplace product

Use Foundry DevOps to include your dataset transformations in Marketplace products for other users to install and reuse. Learn how to create your first product.

Supported features

When packaging a dataset transformation (along with its producing Code Repository), all required dependencies are stored as part of the product; this guarantees that the transformation is self-contained and can run successfully anywhere. Repositories can bring Maven, PyPI, and Conda dependencies.

Python, Java, and SQL transformations are supported. Transformations must be produced from a repository with a recent template, otherwise packaging errors may occur. To debug, upgrade your repository in the Code Repositories application. If a transformation can be successfully packaged, it will not cause any installation or runtime errors.

⚠ **The dataset you provide as an input at the time of installation must have column names and variable types that are identical to the source dataset.**
This is because all dataset columns from the source input datasets (for example, an `airplane` dataset used as an input to a dataset transformation, which is then included in a Marketplace product) will be required inputs when installing, whether mns are referenced in the dataset transformation.

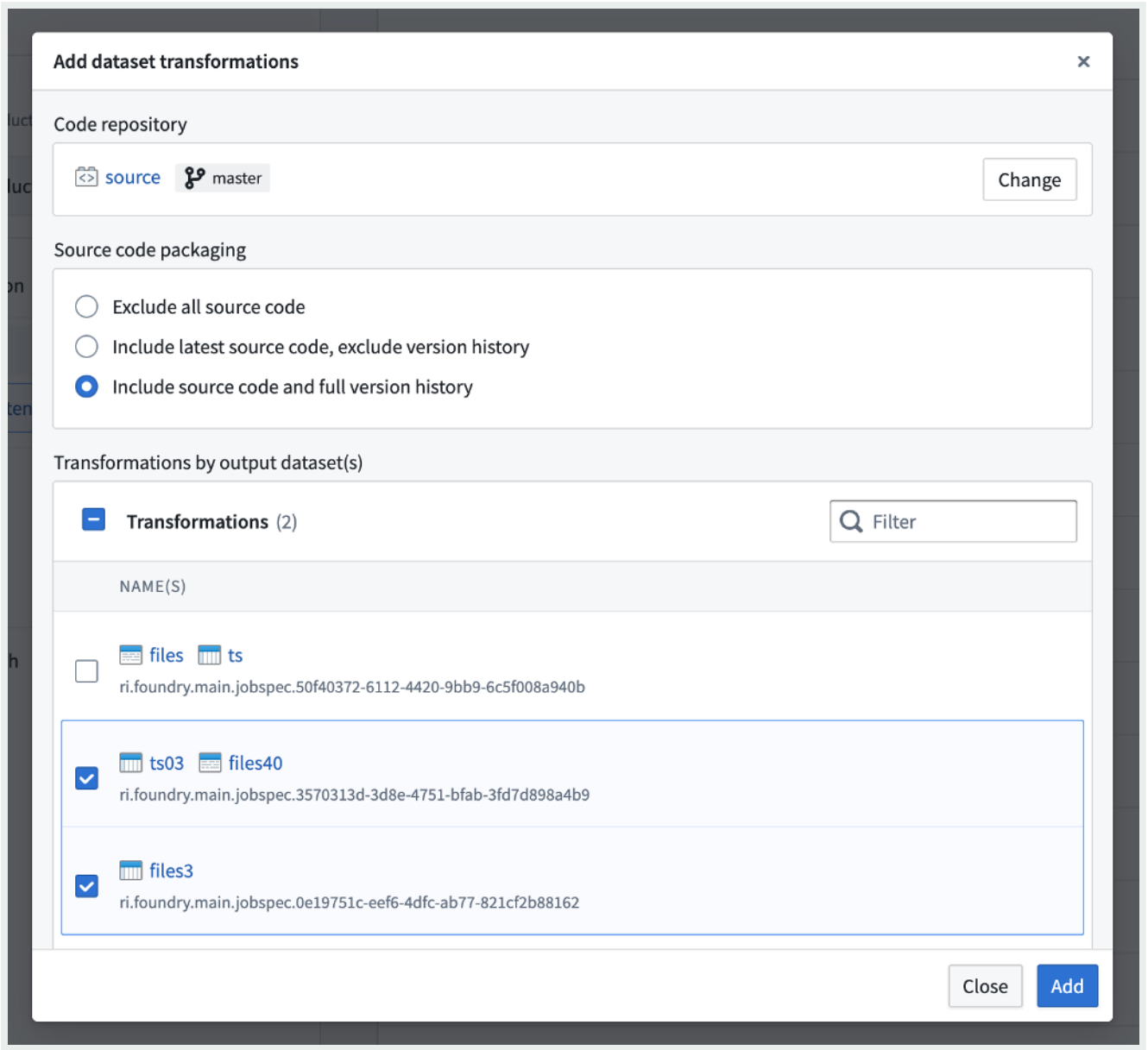
Supported features include:

- Incremental transforms
- Unmarking workflows
- Spark profiles
- Telemetry
- Libraries
- External transforms
- Schemaless datasets

Adding dataset transformations to products

To add a dataset transformation to a product, first create a product, then add outputs. Choose the **Add files** option to navigate to the dataset transformation from within the Compass filesystem and add it to your product.

In some cases, one transformation may produce multiple output datasets. If this is the case, all produced datasets need to be included in the product.



Repository packaging options

There are three ways to package a repository.

- **Exclude all source code:** The repository is packaged without any source code. The only purpose of the repository is to hold the dependencies required when running the transformation. This method includes the compiled user code and all transitive dependencies.
- **Include latest source code, exclude version history:** The repository contains both the the necessary artifacts; however, the Git history (including tags) is not the recommended way to ship repositories as read-only documentation.

- **Include source code and full version history:** The repository is persisted in the product as-is. The entire Git history is saved at packaging time and restored at installation time. This is the only mode which allows you to run checks and rebuild the transformations from within the Code Repositories application after installation.

PREVIOUS
← AIP features

NEXT
Artifact repositories / Overview →

Artifact repositories

Artifact repositories enable users to publish and manage Artifacts, including Conda ↗, Docker ↗, and Maven ↗.

Artifact repositories should be used to upload all Conda, Docker, or Maven Artifacts that are not authored as a library or accessible through an external URL. For example, you may have written a Conda package on your local machine that you wish to access in Code Repositories. By publishing the Conda package to an Artifact repository, you will be able to access it from the **Library** search panel in Code Repositories.

Key features of Artifacts repositories are:

- **Publishing Artifacts:** Generate a token and push an Artifact into an Artifact repository.
- **Searching for Artifacts:** Find Artifacts from the Artifact Repository interface.
- **Recalling Conda Artifacts:** Recall Conda Artifacts to prevent downstream consumers from compiling code with a specific version.

Learn more about the Artifact Repository interface and how to create an Artifact repository.

PREVIOUS
← Add dataset transformation to a Marketplace product

NEXT
Navigation →

Terms of Use ↗

Cookies Settings

API Reference ↗

Send feedback

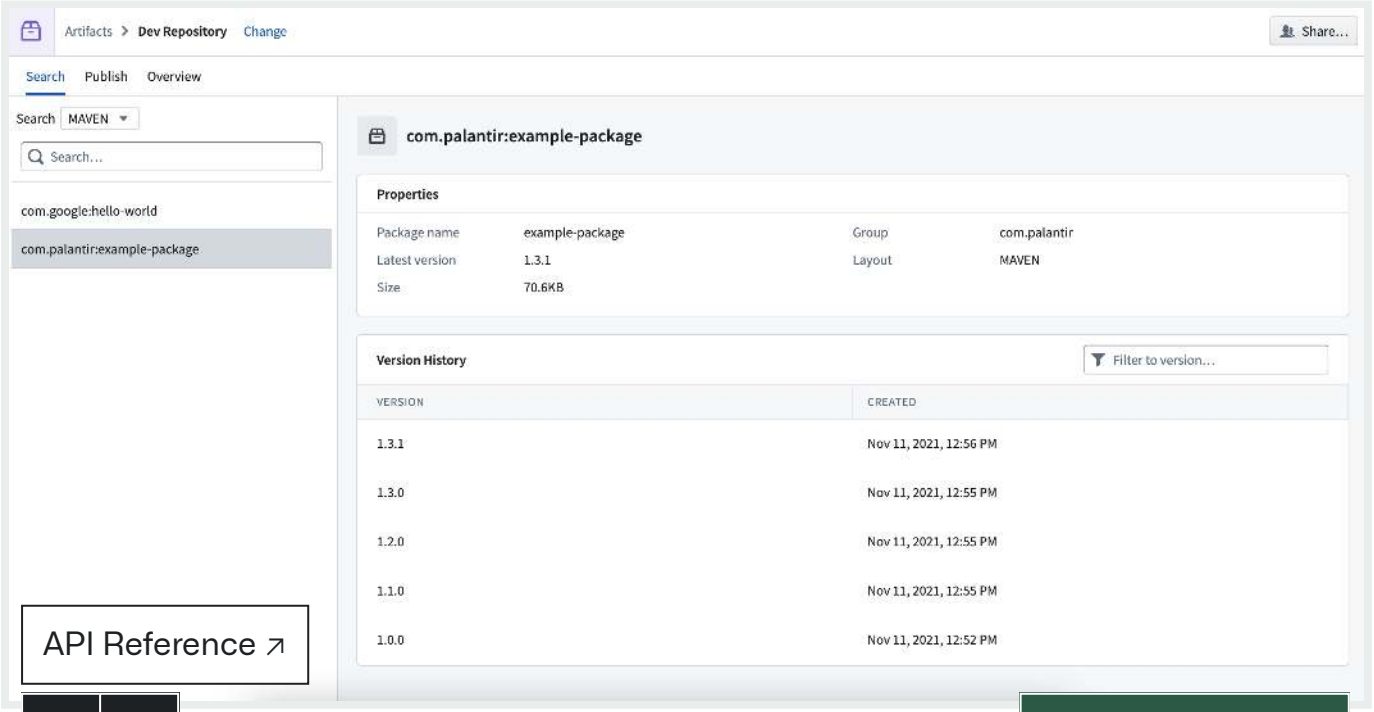
Navigation

Before working with Artifacts, we recommend familiarizing yourself with the Artifact Repository interface.

There are three tabs at the top of the Artifact Repositories interface:

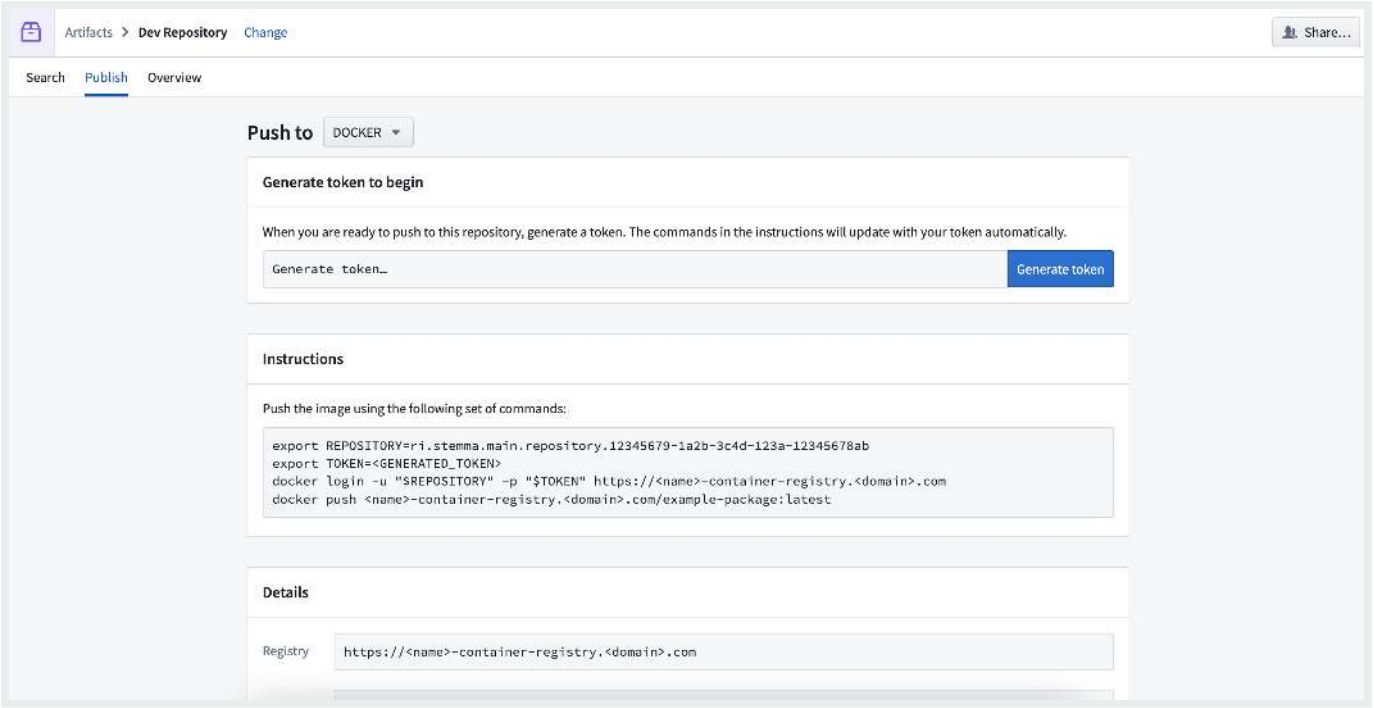
- Search
- Publish
- Overview

Search



Search for Artifacts in your Artifact repository using the dropdown menu and search bar. Select a search result to view metadata such as version history, size, and creation date.

Publish

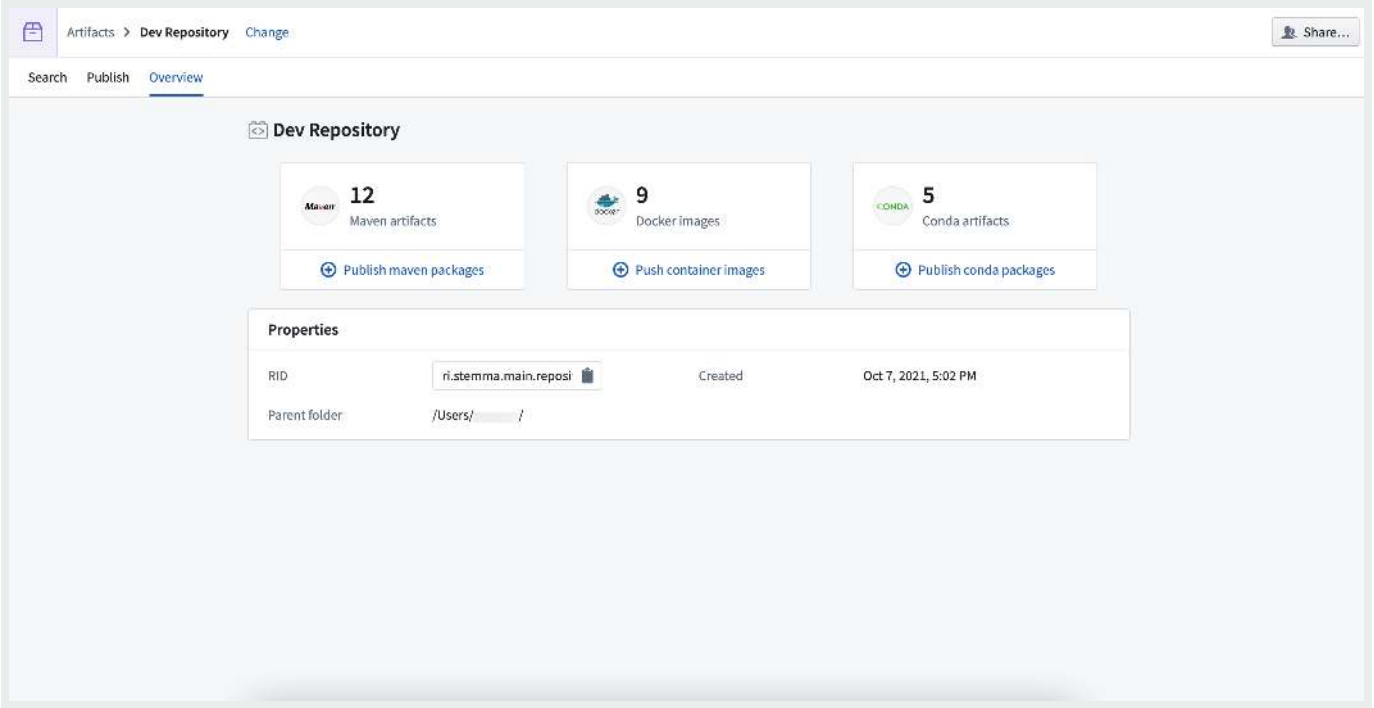


Publish Artifacts to your Artifact repository by generating a token and following step-by-step instructions in the **Instructions** section of the page.

Overview

API Reference ↗

Send feedback



View a summary of all the Artifacts in your Artifact repository, including metadata like its resource ID (RID), creation date, and parent folder in your filesystem.

← PREVIOUS
Overview

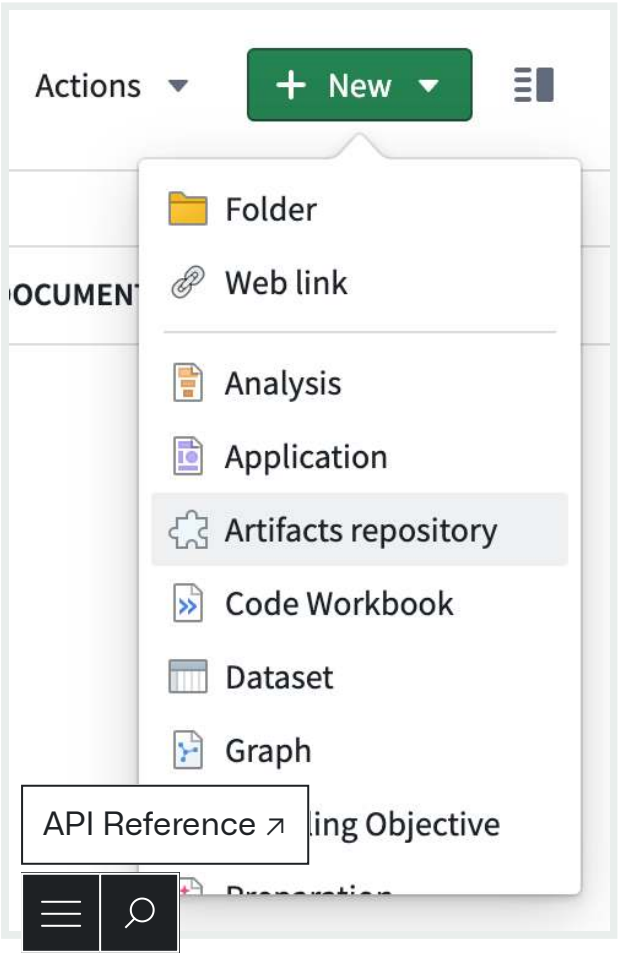
NEXT
Create an Artifact repository →

Create an Artifact repository

Before you can begin publishing and managing Artifacts, you need to create an Artifact repository that you will use to store them.

To create an Artifact repository, first navigate to a Project where you would like the Artifact repository to be saved.

Then, select **New** and choose **Artifact Repository**.



After creating your Artifact repository, start publishing Artifacts.

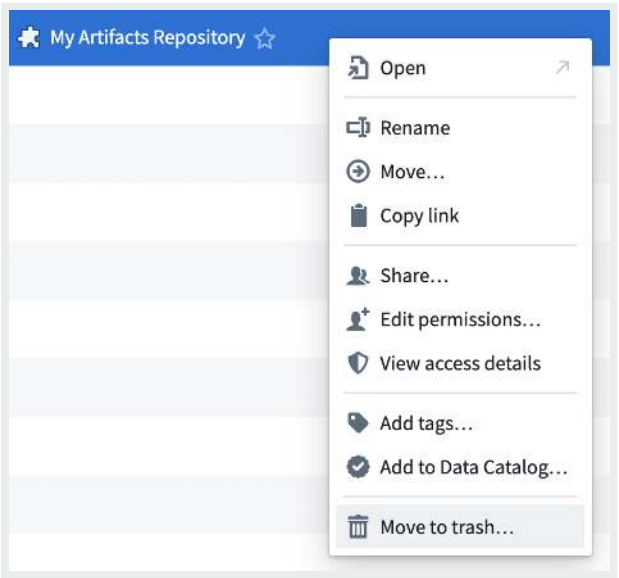
PREVIOUS
← Navigation

NEXT
Delete an Artifact repository →

Delete an Artifact repository

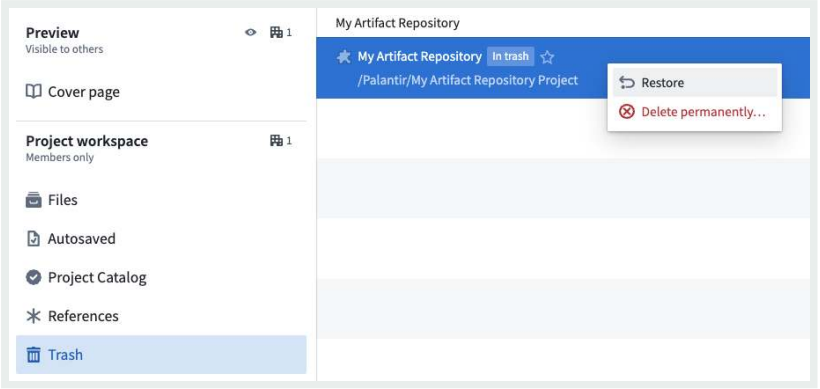
> ⚠ Deleting an Artifact repository will remove all Artifacts contained within the Artifact repository. This is considered a breaking change and all consumers of the deleted Artifacts will be impacted. Take care when deleting Artifact repositories.

To delete an Artifact Repository, first navigate to the Project that contains the Artifact Repository. Then, right-click on the Artifact Repository and choose **Move to trash**.



To permanently delete an Artifact Repository, navigate to the **Trash** tab within the Project. Right-click on the Artifact Repository and select **Delete permanently**. This action cannot be u

It may be possible to restore an Artifact Repository. First, navigate to the **Trash** tab within the Project. Then, right-click on the Artifact Repository and select **Restore**. If you do not see the **Trash** tab, be sure you are in the Project overview rather than a folder within the Project.



Learn more about deleting and restoring files in Foundry.

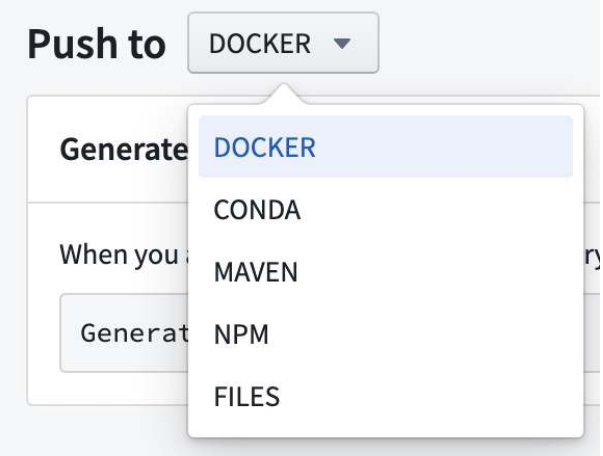
← PREVIOUS
Create an Artifact repository

NEXT
Publish an Artifact →

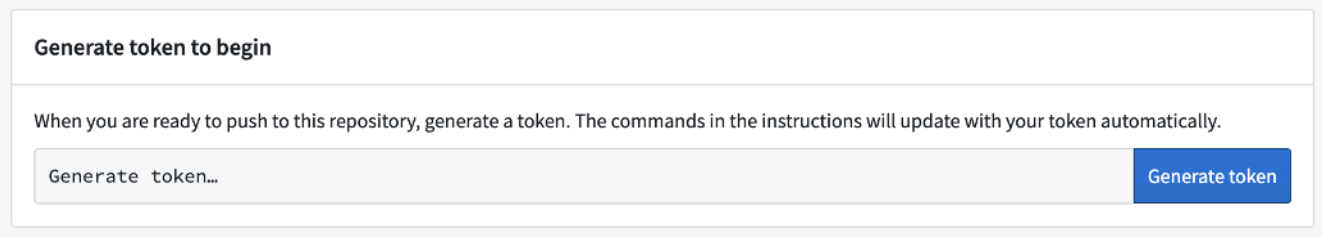
Publish an Artifact

To add an Artifact to an Artifact repository, follow these steps to generate a token and view publish instructions:

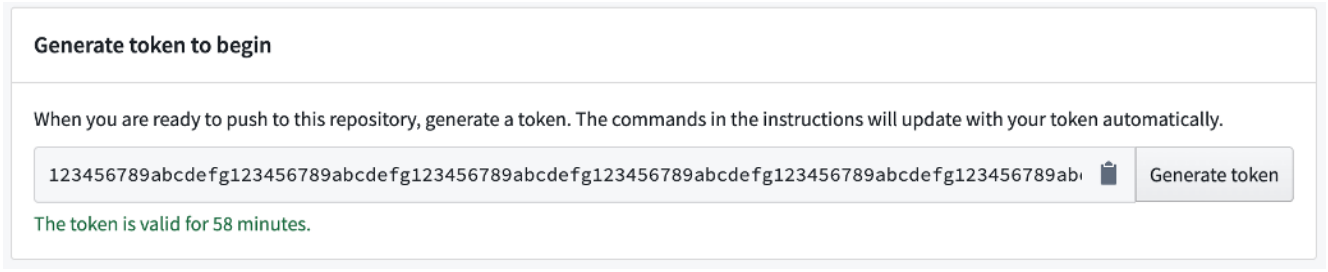
1. Navigate to the **Publish** tab in the interface.
2. Select the type of Artifact to publish from the dropdown menu.



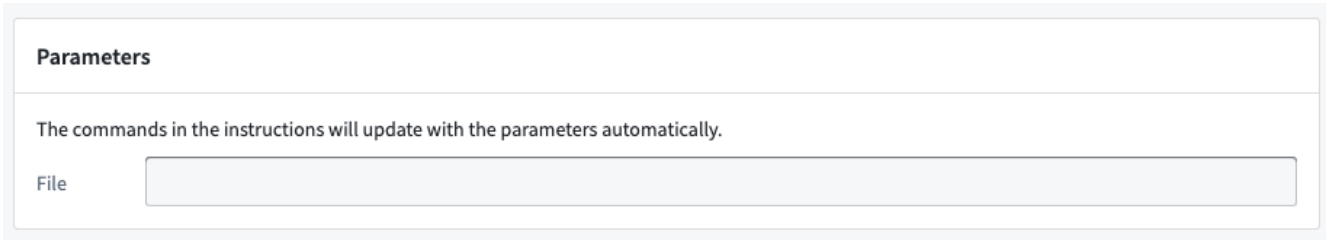
3. Select **Generate token**.



4. View the generated token in the token field. The text under the token field will show for n is valid.



5. (Optional) For some Artifact types, you can update the publishing instructions with parameter commands. Add commands in the field of the **Parameters** section.



6. Run the commands listed in the **Instructions** section to publish your Artifact.

7. When prompted, provide the additional details listed in the **Details** section.

N API Reference ↗ ow to publish an Artifact, learn how to recall Artifacts.

Send feedback

Find consumers of a Conda artifact

To identify which repositories consume a specific Conda artifact:

1. Navigate to the **Search** tab in Artifact Repositories.
2. Set the format dropdown to **Conda**.
3. Use the search bar to locate the artifact by name.
4. Select the artifact from the search results to open its version history.
5. Find the specific version you are interested in.
6. Select the arrow to the right of the **Downloads** column.
7. The **</> Consumers** section lists all repositories consuming that version of the artifact.

> Consumer tracking is only available for Conda artifacts.

PREVIOUS
← Delete an Artifact repository

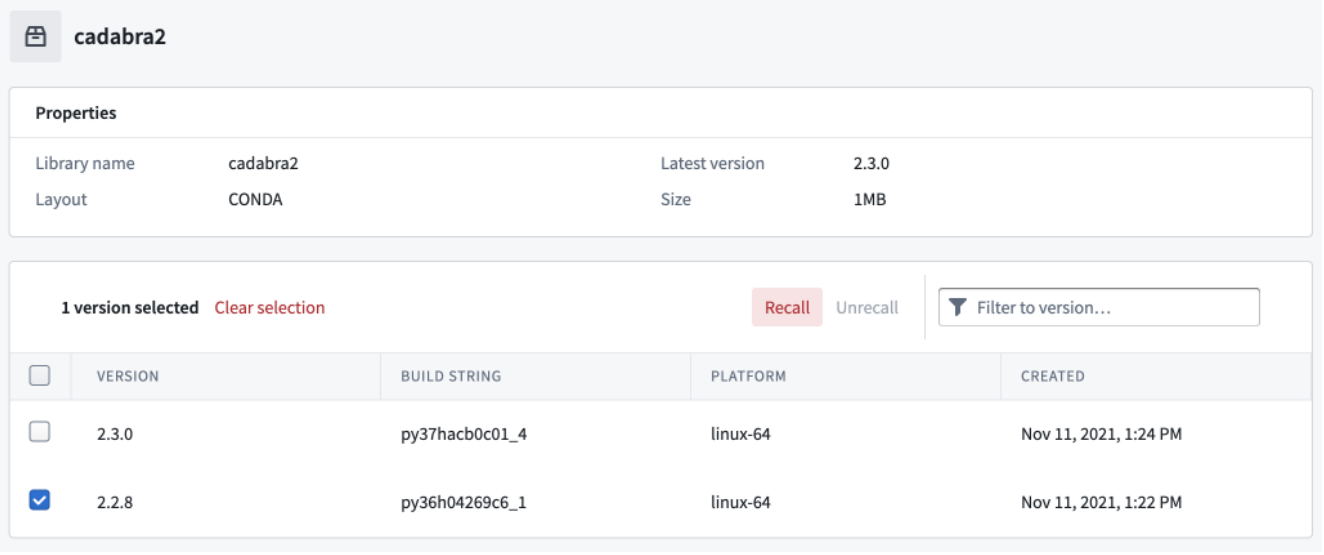
NEXT
Recall an Artifact →

Recall an Artifact

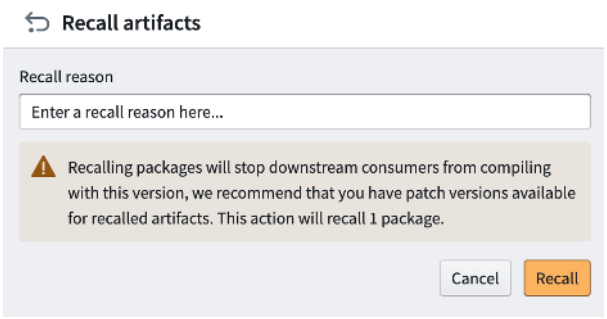
It is possible to recall Conda Artifacts to stop downstream consumers from compiling code with the recalled version. We recommend having patch versions available for recalled Artifacts before starting the recall process.

Follow these steps to recall an Artifact:

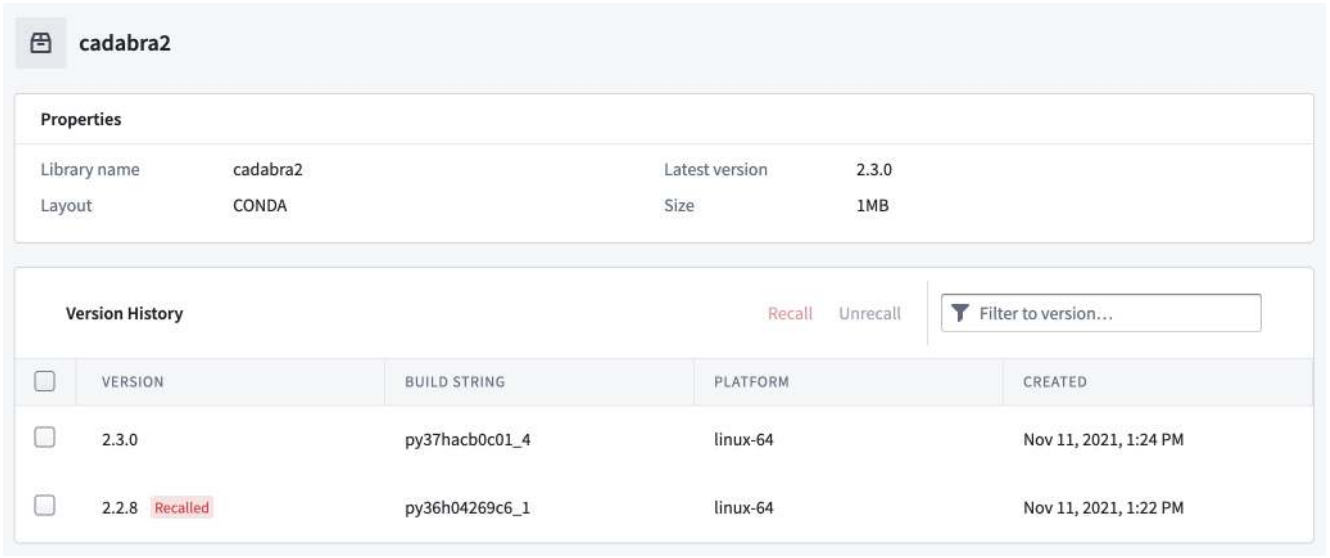
1. Search for the Conda Artifact in your Artifact Repository and select it to view the Version History section in the summary page.
2. Select the version to recall and then choose **Recall**.



3. A **Recall artifacts** pop-up will appear. Enter the reason for recalling the Artifact in the field.



4. View the Version History again to see that the Artifact is now marked as `Recalled`.



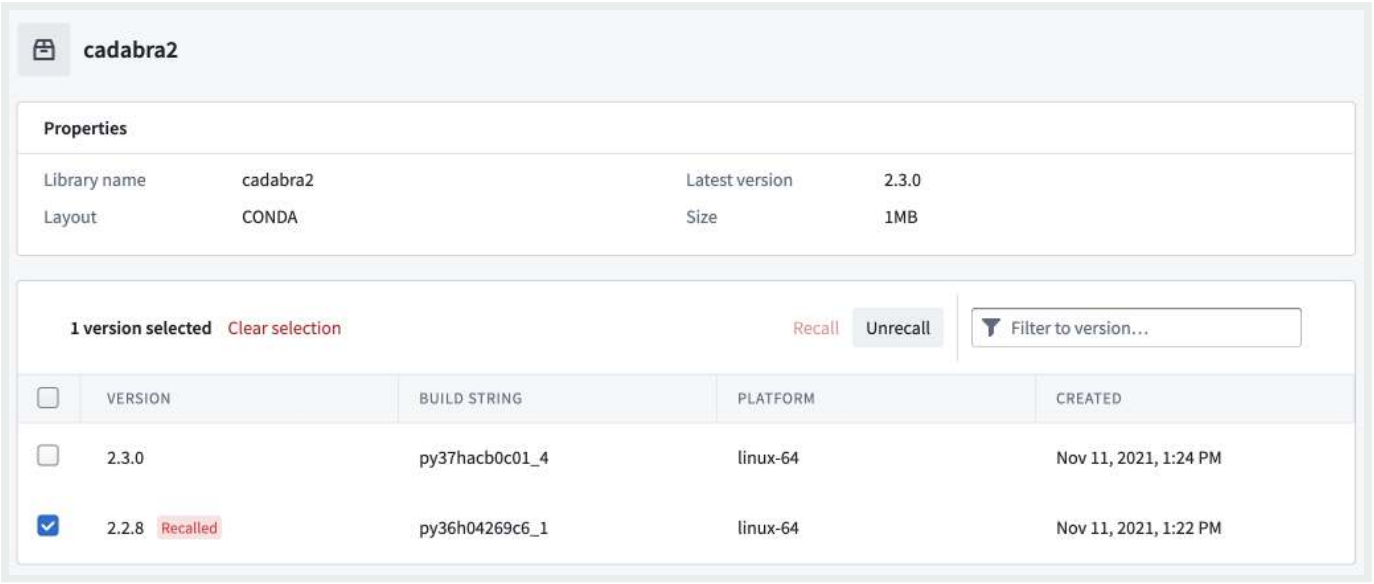
Unrecall

You can unrecall an Artifact.

To unrecall an Artifact, select the version of a recalled Artifact and click **Unrecall**.

API Reference ↗

Send feedback



Delete

Conda Artifacts can be recalled, but it is not possible to delete any Artifacts in an Artifact repository. If you explicitly need to delete an Artifact, you must delete the Artifact repository.

PREVIOUS
← Publish an Artifact

NEXT
Manage permissions →

Manage permissions

When using Artifact repositories, consider the following permissions requirements:

- To use an Artifact, users must have **View** permission on the Artifact repository.
- To publish an Artifact to an Artifact repository, users must have **Edit** permission on the Artifact repository.
- To recall an Artifact, users must have **Edit** permission on the Artifact repository.

> It is not possible to delete an Artifact.

Learn more about permissions and Project access.

PREVIOUS
← Recall an Artifact

NEXT
Advanced workflows / Create custom checks →

Create custom checks

Custom checks can be created as a Gradle ↗ tasks. These tasks should be added to the appropriate inner `build.gradle` files, within the language subfolders. Here is an example of a Gradle task that prints `Hello World` when executed (note that the `doLast` method creates a task action that runs when the task executes, rather than at configuration time):

```
// Define custom task
task customTask {
    doLast {
        println "Hello World"
    }
 }
```

In order for tasks to get executed during CI checks, there must be a CI task that depends on your custom task. This is also defined in the same `build.gradle` file. In the following example, the check task would be executed after `customTask` in CI; this is where the `Hello World` message will appear in the CI logs.

```
// Add CI dependency on custom task
 project.tasks.check.dependsOn customTask
```

In order for tasks to get executed at the end of the task list, you would use the following syntax instead.

```
project.tasks.publish.configure { finalizedBy customTask }
```

To add dependencies, you can use the following CI tasks: `project.tasks.check`, `project.tasks.test`, and `project.tasks.publish`. We strongly recommend you do not use or rely on other CI tasks (for example, internal tasks) as these are not guaranteed and are subject to change.

More documentation of the functionality provided by Gradle build scripts is available in the Gradle docs ↗.

> Adding custom CI checks to the `ci.yml` file is not recommended as this file will be overwritten each time the repo is upgraded.

PREVIOUS
← Artifact repositories / Manage permissions

NEXT
Prepare datasets for download →

Prepare datasets for download

⚠ The following export process is an advanced workflow and should only be performed if you are unable to download data directly from the Foundry interface using the **Actions** menu or export data from another Foundry application.

This guide explains how to prepare a CSV for download using transforms in Code Repositories or Pipeline Builder.

Prepare data

The first step in preparing the CSV for download is to create a filtered and cleaned set of data. We recommend performing the following steps:

1. Ensure the data sample can be exported and follow data export control rules. Specifically, you should verify that the export adheres to the data governance policies of your Organization.
2. Filter the data to be as small as possible to meet the necessary objectives. For optimal performance, the uncompressed size of the data in CSV format should be less than the default HDFS block size (128 MB). To achieve this, you should only select the necessary columns and minimize the number of rows. You can reduce the number of rows by filtering for specific values, or taking a random sample with an arbitrary number of rows (such as 1000). Attempting to create a CSV larger than 128 MB may consume more time additional Spark executor memory to succeed.
3. ype to `string`. Since the CSV format lacks a schema (unenforced d labels for columns), it is recommended to cast all the c s especially important for timestamp columns.

The following sample Python (PySpark) code illustrates some of the above principles as applied to a dataset of taxi trips in New York City.

```
def prepare_input(my_input_df):
    from pyspark.sql import functions as F

    filter_column = "vendor_id"
    filter_value = "CMT"
    df_filtered = my_input_df.filter(filter_value == F.col(filter_column))

    approx_number_of_rows = 1000
    sample_percent = float(approx_number_of_rows) / df_filtered.count()

    df_sampled = df_filtered.sample(False, sample_percent, seed=0)

    important_columns = ["medallion", "tip_amount"]

    return df_sampled.select([F.col(c).cast(F.StringType()).alias(c) for c in im
```

Using similar logic and Spark concepts, you can also implement the preparation in Pipeline Builder or other Spark APIs like SQL or Java.

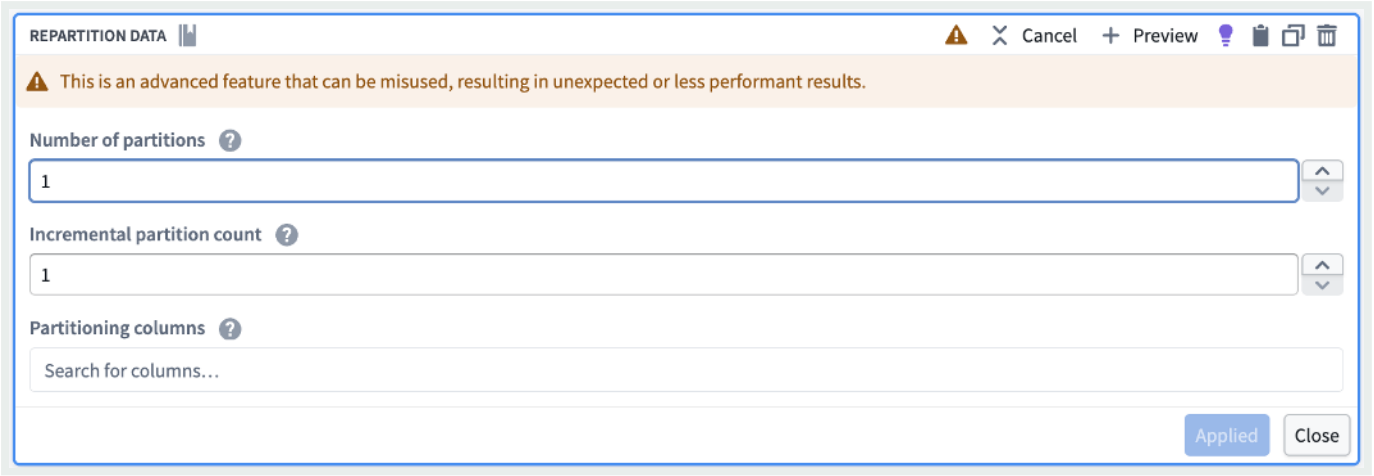
Reduce to one partition and set output format

Once the data is prepared for export, you need to repartition or coalesce the data to a single partition so that the output will have one file for download. Then, set the output format to CSV or another supported output format such as JSON, ORC, Parquet, or Avro. The below examples show how to repartition data and set an output format in Pipeline Builder, Python, and SQL.

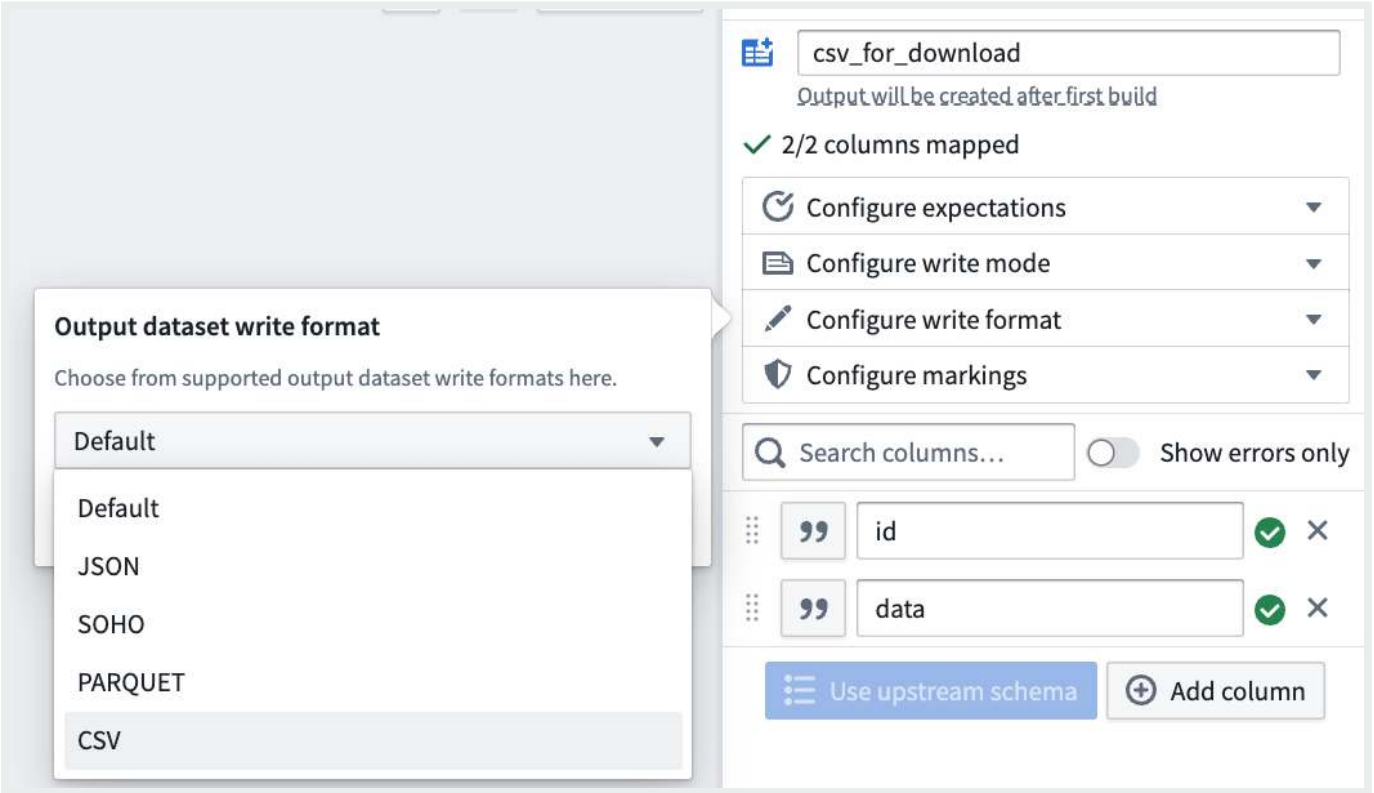
Both `repartition(1)` and `coalesce(1)` reduce data to a single partition, but `coalesce` should be avoided when a filter operation is performed as part of the
API Reference ↗
n because `coalesce` collapses upstream tasks, potentially causing the filter operation to run in a non-distributed way.

Pipeline Builder

First, use the **Repartition data** transform to reduce your data to one partition.



Then, in the pipeline output configuration, configure the write format to CSV (or any other supported format).



API Reference ↗

Send feedback

```python
from transforms.api import transform, Input, Output
@transform(
      output=Output("/path/to/python_csv"),
      my_input=Input("/path/to/input")
)
def my_compute_function(output, my_input):
      output.write_dataframe(my_input.dataframe().repartition(1), output_fo
```

SQL

```
CREATE TABLE `/path/to/sql_csv` USING CSV AS SELECT /*+ REPARTITION(1) */ * FROM
```

Review official Spark documentation ↗ for additional CSV generation options (note that Pipeline Builder only exposes a subset of these options).

> In the above examples, we reduce to one partition so that the output dataset has exactly one CSV file for download. As a result, the entirety of the data must be able to fit in the memory of one Spark executor, which is why we recommend filtering or sampling the data first. If filtering or sampling the data is not an option, but the data is too large to fit in one executor's memory, you can increase executor memory using an appropriate Spark profile. Alternatively, you can use a value other than 1 for the partition count. Using `repartition` with a value greater than 1 will reduce the amount of data that must be held in memory on one executor, but will also entail that the output dataset will have multiple CSV files that need to be downloaded individually instead of one.

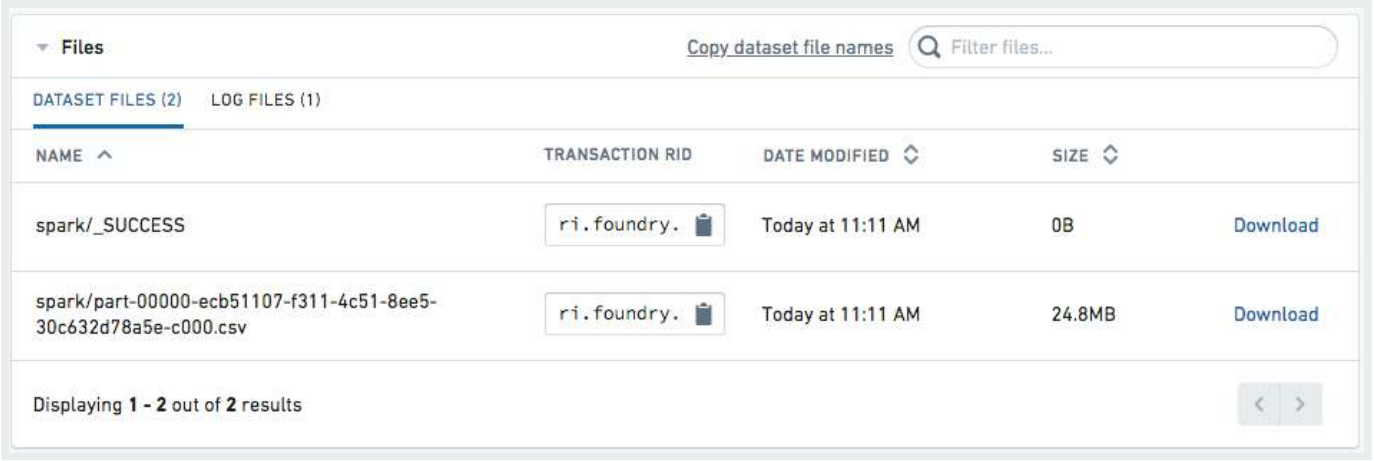
Access the file for download

Once the dataset is built, navigate to the **Details** tab of the dataset page. The CSV should a

API Reference ↗

or download.

Send feedback



PREVIOUS
← Create custom checks

NEXT
Administer repositories / Overview →

Administer repositories

> Administration of code repositories requires the appropriate permissions. Most settings can be adjusted by repository **owners** (by default). Other options, like the code editor preferences, are available to all authors.

To change the settings of your Code Repository, access the repository and select **Settings** in the top navigation bar. The settings tab allows you to set your personal editor preferences as well as configure the behavior of the repository. See additional information on the following settings:

- Branch settings
- Repository settings
- Repository upgrades
- Spark
- Artifacts
- Project scope
- Security Approvals
- Additional repository settings (`repoSettings.json`)

PREVIOUS

← Advanced workflows / Prepare datasets for download

NEXT

Branch settings →

API Reference ↗

Send feedback

Branch settings

This page will walk you through the various branch settings in Code Repositories:

- Setting the default branch
- Selecting pull-request merge modes
- Protecting branches
- Configuring branch fallbacks

Default branch

In Code Repositories with more than one branch, a default branch can be configured as the base branch. By default, all pull requests and commits will be made against that branch unless chosen otherwise. Usually, the default branch is the `master` branch.

You can choose the main branch of your Code Repository by navigating to `Settings > Branches > Default branch`.

Merge modes

There are several strategies to merge code changes proposed in a pull request. In the Settings tab you can select one or more merge-modes to be available to code authors in the pull request page.

ge - Squash-and-merge mode will create a single commit to the target
ncorporating all the changes that the pull request introdu a
ndensed commit-history on the default branch.

- **Merge** - When you use *Merge* in a pull request, all the individual commits created on the branch will be added to the target branch alongside a merge-commit. Note that in the target branch commit history, all the individual commits will appear with their original timestamp, signaling the time they were created on the development branch.
- **Merge with fast-forward** - When there is a direct path from the target branch to your branch (there are no additional changes on the target branch), merge-with-fast-forward advances the target branch to the front of the development branch and combines their commit history.

All the selected merge modes will appear as options in the pull request page. If "Squash-and-merge" mode is selected, it will appear as the main option, with the other selected modes available in a menu.

Protected branches

When there are multiple authors contributing to the same code repository, or when the repository backs critical data assets, you can protect your branch to achieve a greater level of governance and defense against unintentional changes. A protected branch can only be modified via a pull request and must satisfy a pre-defined set of requirements.

> By default, only the Code Repository's owners can change the branch protection settings, while both Owners and Editors can merge pull requests to protected branches. Regardless of permissions, all code authors need to abide to the protected branch policy.

You can set the following requirements in the branch settings panel:

- Require ci/foundry-publish to run successfully
- Require code reviews
- Require specific reviewers
- Require security approval
- Restrict stable version tags (Functions repositories only)

API Reference ↗

Require ci/foundry-publish to run successfull

Send feedback

In order to publish changes to your data, the continuous integration process `ci/foundry-publish` must run and finish successfully. There are no guarantees that changes will take effect if you merge changes before it finishes successfully so it is highly recommended to make this a requirement for your protected branch.

Require code reviews

One benefit of protecting critical branches is the ability to receive a collaborator's review on code changes before merging them to a production environment. Anyone with permissions to merge changes can submit a review on a pull request (by default: Owners and Editors of a repository).

You can enforce the following review policies:

- **Require no rejections before merging** - This will block the pull request from merging if at least one of the reviewers rejected the code changes.
- **Require at least one approval before merging** - This will ensure the code is reviewed and approved before changes are merged.

Require specific reviewers

You can require approval from a specific user or group before pull requests can be merged. To satisfy the Group requirement, at least one member of the group needs to approve the pull request. Note that on its own, this policy allows rejections as long as an approval was received. For example, if one member of a group rejected the changes and another member approved, the approval will supersede the rejection unless "Require no rejections" policy applies.

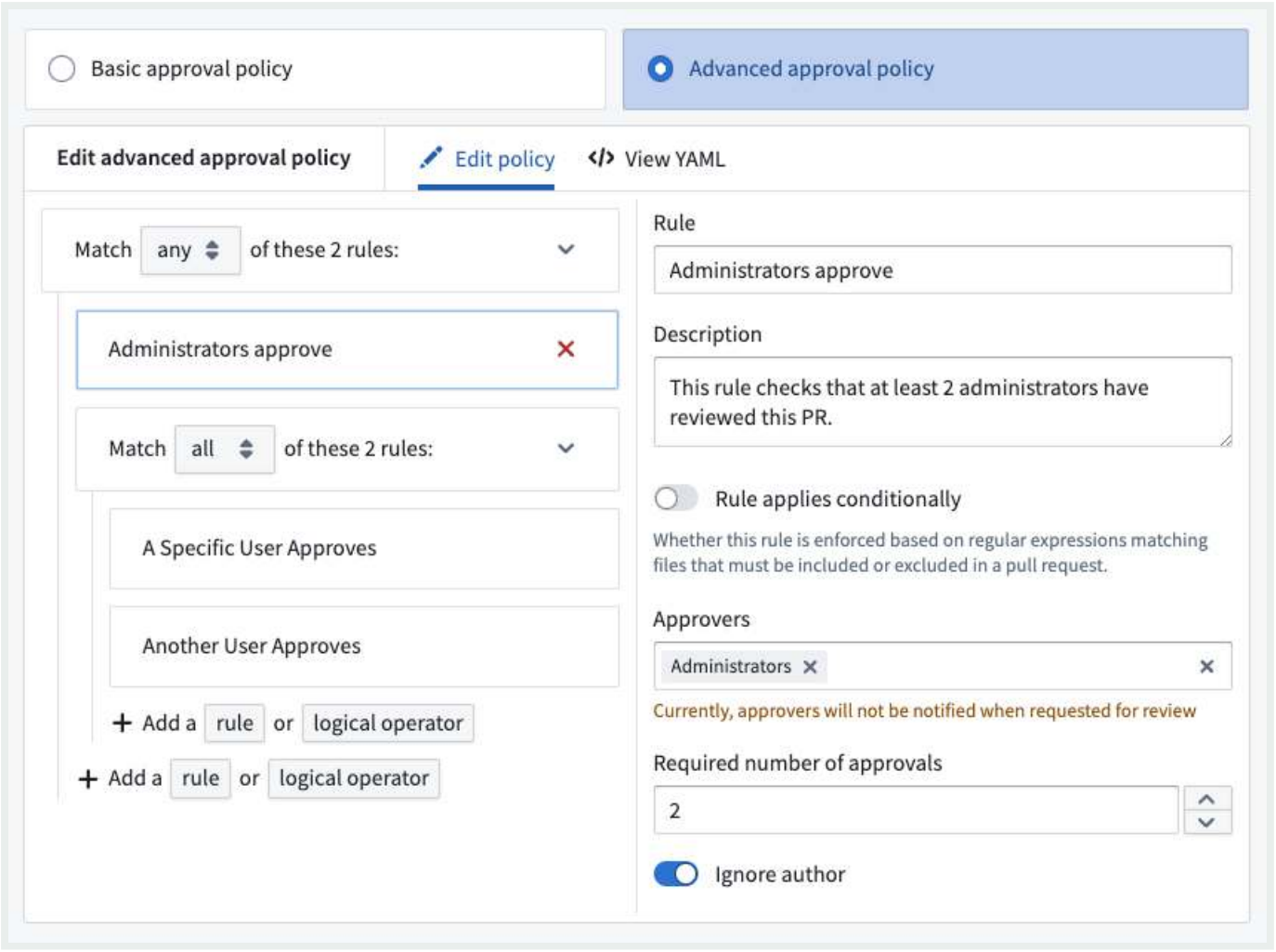
> ⚠ Warning
>
> Requiring users/groups approval does **not** give them permission to review the pull request. Always assure that required reviewers also have access to the Code Repository.

API Reference ↗

Send feedback

Configure advanced approval policies

Advanced pull request approval policies determine the users and groups required for review based on the files modified in a pull request. In the branch settings of a protected branch, select **Edit**, then **Advanced approval policy** to start configuring a policy.



The **Edit policy** tab provides a form-based editor. The advanced approval policy groups logical operators of a rule, such as `ALL` or `ANY`. Select a rule to provide metadata, such as the name or description of the rule. Toggle on **Rule applies conditionally** to only apply this rule if the files modified in a pull request match some regular expression. For example, in the typical folder structure for a Python transform, `["transforms-python/src/myproject/datasets/.*\\.py"]` will match all Python files within the datasets folder.

Within the **View YAML** tab, you can directly modify, copy, and paste the YAML representation of the policy. This can be useful for making bulk edits to a policy, such as finding and re rm.

API Reference ↗

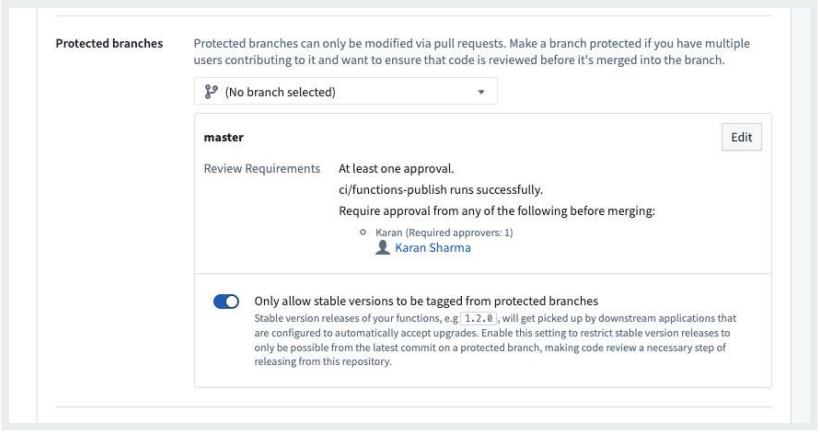
Send feedback

Require security approval

To stop propagating security Markings on a branch, the branch must be protected. Once security changes are enabled on a repository, it will automatically and immutably require security checks and approval (if necessary) before merging a pull request. Read more about removing inherited Markings.

Restrict stable version tags

In Functions repositories, you can enforce restrictions on the release of stable versions of your Functions. Once at least one protected branch has been configured for the repository, an additional toggle will be available to **Only allow stable versions to be tagged from protected branches**.



If **Only allow stable versions to be tagged from protected branches** is enabled, tagging from feature branches (non-protected branches) of the repository will block users from being able to submit a stable Semantic Version string. For example, `1.2.3` is a stable Semantic Version, whereas `1.2.3-rc1` denotes a prerelease version that will be allowed from feature branches.

Applications that invoke your Functions may be configured to reference a *version range*; for example, a Workshop module might call a Function `foo` at versions `>=1.2.3 <2.0.0`, so if you release version `1.3.0`, the Workshop module will now invoke this version (`1.3.0`).

To prevent unexpected behavior from new versions, we recommend that code changes go th API Reference ↗ esting before they are released in a new stable version. The **Only allow stable versions to be tagged from protected branches** toggle ensures that all changes must be reviewed before being released in a stable version, since cha Send feedback code review before they can merge into a protected branch.

Since version ranges ignore prerelease versions, you can publish a prerelease version like `1.3.0-test` to prevent downstream applications from invoking the new release. Enabling the **Only allow stable versions to be tagged from protected branches** toggle allows prerelease versions to be released from feature branches, so that versions can be tested in downstream applications without causing issues in production.

Unprotect branches

Owners can unprotect branches from Code Repository settings. However, branches with active security changes, like stopped propagation of markings, cannot be unprotected until the branch is free of security changes. To unprotect a branch with active security changes, the changes should be removed through a pull request. The branch can then be unprotected from Code Repository settings.

Fallback branches

Code Repositories allow you to build datasets on any branch and view the affect your transforms have on the data. If an input dataset to your transform has not been built on the current branch, an attempt will be made to locate a built version from a list of fallback branches instead. The default branch will automatically be set as the fallback branch unless configured otherwise. You can set different fallback branches to each branch and have more than one fallback if needed.

> This configuration only applies to builds and actions taken in the Code Repository on which it is set. It has no effect on other Foundry applications or builds triggered outside of the repository (for example, using a scheduled build).

You can also configure a Pipeline Builder branch with the same name as a Code Repositories branch so that Pipeline Builder transforms read input datasets from that matching branch. Pipeline Builder supports fallback branch configuration as well, allowing you to control which branch is used when an input dataset has not been built on the current b

API Reference ↗

Send feedback

Repository settings

Repository settings allow you to configure the behavior of your code repository. Changes to these settings will apply to all repository users.

Commit message settings

By default commit messages will be auto-generated when clicking the commit button or when building a dataset. You can encourage more meaningful messages by disabling this option. The commit message dialog will open before each commit and require a message to be submitted.

Repository upgrades

See repository upgrades to learn more.

Dataset aliases

Datasets can be referenced in code by using their exact location in Foundry or by using their unique resource identifier (RID). While both options are valid, it is recommended

to use RIDs where possible, as this allows resources to be moved from one location to another without needing any updates to the code in the repository.

The repository editor type-ahead functionality will allow you to search for data as you type, whether you type in paths, RIDs or free text. When you pick a dataset from the list of results, the editor will insert its path or RID depending on the setting:

- **Automatically change to RIDs when possible** - This option will insert the **RID** of the dataset as well as offer to convert any path in the transform to an RID. The editor will present the dataset name over the RID and allow editing it as needed.
- **Do not automatically change locations to RIDs** - This option will insert the **path** of the dataset and allow converting RIDs to paths when possible.

External systems

Information security officers can enable a repository for interaction with external systems. This is an advanced feature that allows developers to write external transforms, python code that interacts with web services and APIs on the open Internet or on-prem system via agent-proxy. This feature should only be enabled on repositories for highly trusted developers, since interactions with external systems cannot be guaranteed to follow Foundry's strict data provenance and access control practices. Security officers can limit the scope of data that is allowed to interact with external systems by configuring which mandatory control markings are allowed for export.

PREVIOUS
← Branch settings

NEXT
Repository upgrades →

Repository upgrades

Foundry will occasionally generate upgrade pull requests on active repositories. These upgrades contain important updates to the Transforms template as well as runtime improvements. Upgrade Pull Requests will be opened on a dedicated branch and request a merge to the default branch.

Automatic upgrades

When automatic upgrades are enabled in the repository, Foundry will attempt to merge the upgrade PRs automatically. After the required checks finish successfully, the pull request will be merged and a new merge-commit will appear in the commit history of the default branch. Select **Automatic merge of upgrade PRs** in the repository settings to enable this feature.

When this option is disabled, or when the PR fails to merge, the repository will present a message when a new upgrade PR is available and require a user with the appropriate permissions to merge it.

> **Warning**
> Enabling automatic upgrades is strongly recommended in order to keep the repository up-to-date with the latest runtime improvements.

T API Reference ↗ s where automatic repository upgrades will require manual intervention before merging:

- The template type of the repository does not support merging.

- The user committed any changes to the upgrade PR branch after it was opened.
- The upgrade PR overwrites any files that contain user-authored changes (see file overwrites).
- Some datasets directly affected by this pull request have not been built with the latest version of the code (see impact analysis).
- Jobs produced by this repository have not yet been built with the runtime Spark module version introduced in the upgrade pull request (see impact analysis).
- The template version the repository is currently on is too old to support automatic upgrades.

File overwrites

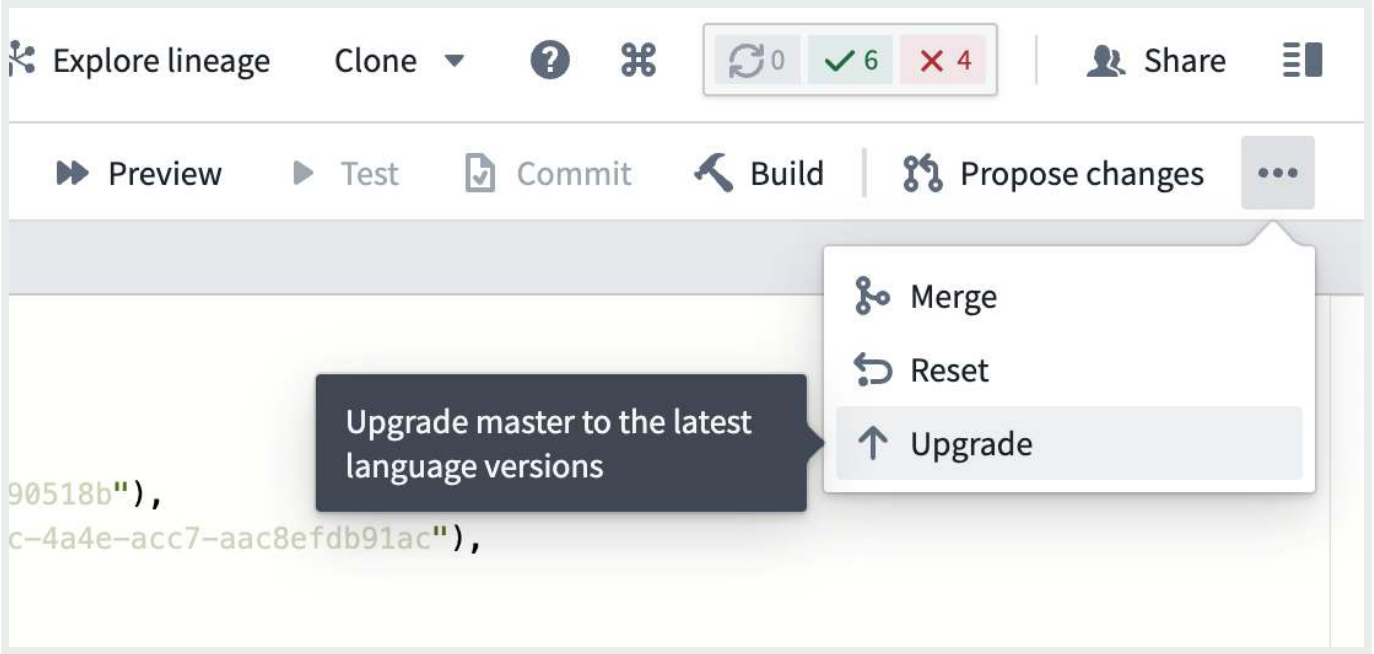
When a repository is created, it is bootstrapped with default template files, which differ depending on the type of template chosen. When a repository is upgraded, some of these default files will be overwritten to match those of the newest template version. This is because the default files are considered to be important for the correct functioning of a repository, and should not be overwritten by users.

Impact analysis

You can use the impact analysis tab of the Repository Upgrade PR to review any potential changes to affected datasets and ensure that the changes are safe before merging. Some upgrades may affect the runtime Spark module version used, which could have an effect on the datasets built by this repository.

Manual branch upgrade

To manually upgrade your branch to the latest language versions, open the **...** menu in Code Repositories and select **Upgrade** as shown in the screenshot below.



If you move a repository across different projects, the repository must be manually upgraded in the new location before any builds or checks are triggered. This is necessary to update relevant project references in the repository.

PREVIOUS
← Repository settings

NEXT
Spark profiles →

Spark profiles

Repository settings are used to specify Spark profiles for use in the repository. After configuring the Spark profile in repository settings, you can use the profile in code.

Before a given profile can be applied, it must be imported into the project. There are two types of profiles:

- *Unrestricted Profiles*: Profiles that can be imported to a repository by all users.
- *Restricted Profiles*: Profiles that can only be imported by administrators.

Spark profiles that are already available for use in a repository are found in the **Spark** section under the **Settings** tab in **Enabled profiles**.

Importing Spark Profiles

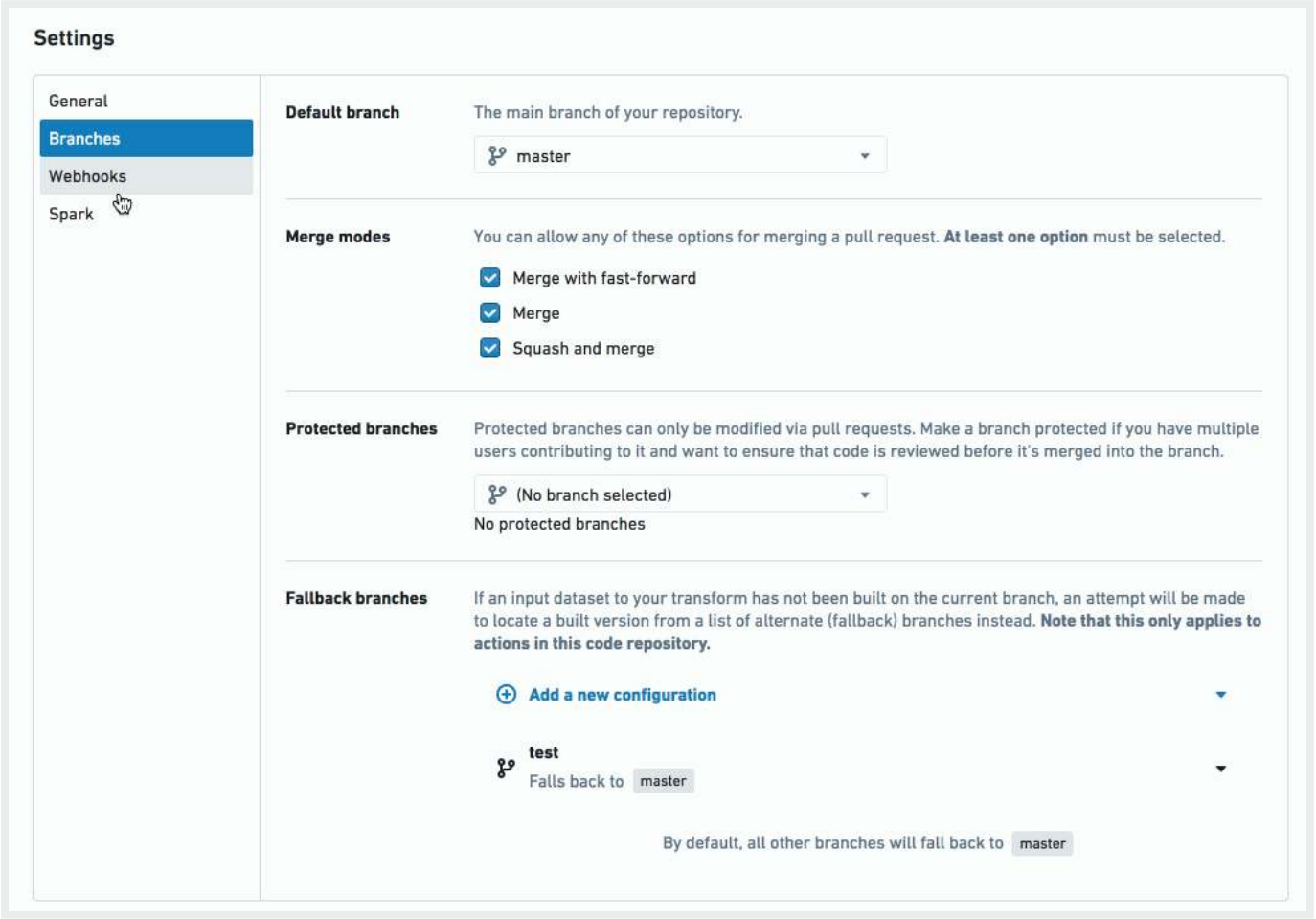
In order to use a Spark profile in a transforms job, the profile must first be imported into the Project containing the job, or else Checks will fail when attempting to publish transforms.

Spark profiles may be browsed and imported to a Project using the Spark configuration tab in the Code Repositories editor.

To import a profile to a project, go to the **Settings** tab and select **Spark**. Click on **Add profiles** to find the desired profile in the dropdown. Hover over the profile that you need:

restricted, you can click on **Import** to import the profile to the Project.
tricted, you may see a lock next to it.

By default, any Resource Management Administrator has the necessary permissions to import restricted Spark profiles. Additionally, there may be a user group named `spark-profile-admins` which also has the necessary permissions. Regular users must ask an administrator to import a profile to their project before they can use it.



Spark Profiles enabled

All profiles already enabled in a Project are discoverable in the **References** panel of the Summary sidebar in the project. When imported to a repository, profiles are automatically added as references for the entire Project and made available as “enabled profiles” for all repos in the Project.

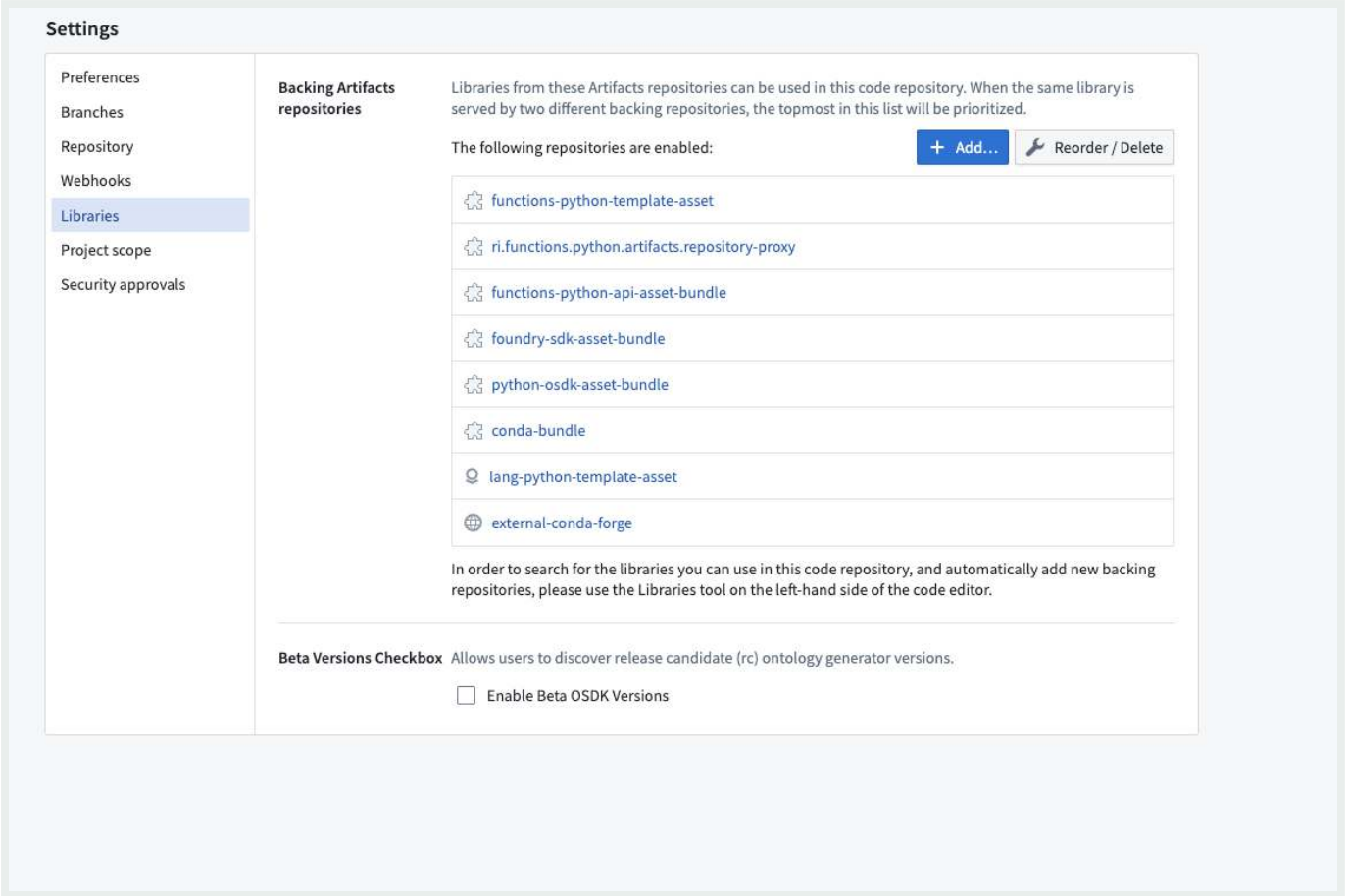
PREVIOUS
← des
NEXT
Artifact settings →
API Reference ↗
Send feedback

Artifact settings

If you are looking to import and use Python libraries, see the section on sharing Python libraries.

The **Libraries** tab contains a list of repositories that can be referenced in your code repository, which we refer to as backing repositories. This is a list of all shared code repositories in your Foundry environment, as well as external or public libraries. You can use the Libraries tab to discover and add backing repositories.

Viewing artifact settings requires the `artifacts:view-repository` permission, and managing artifact settings requires the `artifacts:manage-repository` permission.



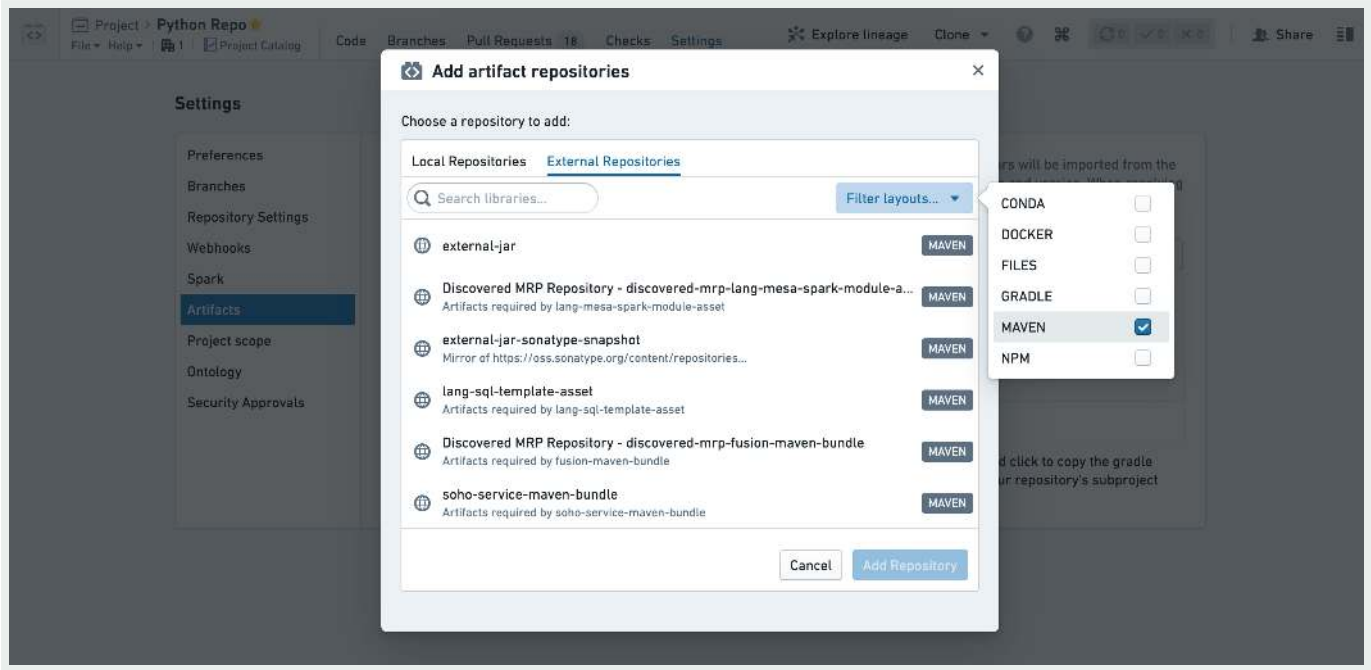
Add a new artifact to your Code Repository

To add a new artifact, select **Add** and choose one of two types of repositories:

1. **Local repositories:** These are other code repositories in your Foundry environment that were configured as a shared library.
2. **External repositories:** Artifact repositories stored out of your Foundry environment. These could be external Foundry repositories or public artifact repositories that are available in your environment.

API Reference ↗

Send feedback



If the added artifact repository contains references to additional repositories, they will be added as well. The same access permissions are required for all the dependencies of the added repository.

> When adding a local repository from a different project, a project reference to that repository will be added. This requires `compass:view-project-imports` and `compass:import-resource-to` on your own code repository and `compass:import-resource-from` permission on the referenced shared repository.

> While it is possible to reorder and delete backing repositories, this might break builds of transforms that use packages from those repositories. Only take this action after considering the possible implications.

Beta OSDK generator versions toggle

When the **Enable beta OSDK versions** option is enabled, users can generate beta (release candidate) ontology generator versions. The toggle is off by default.

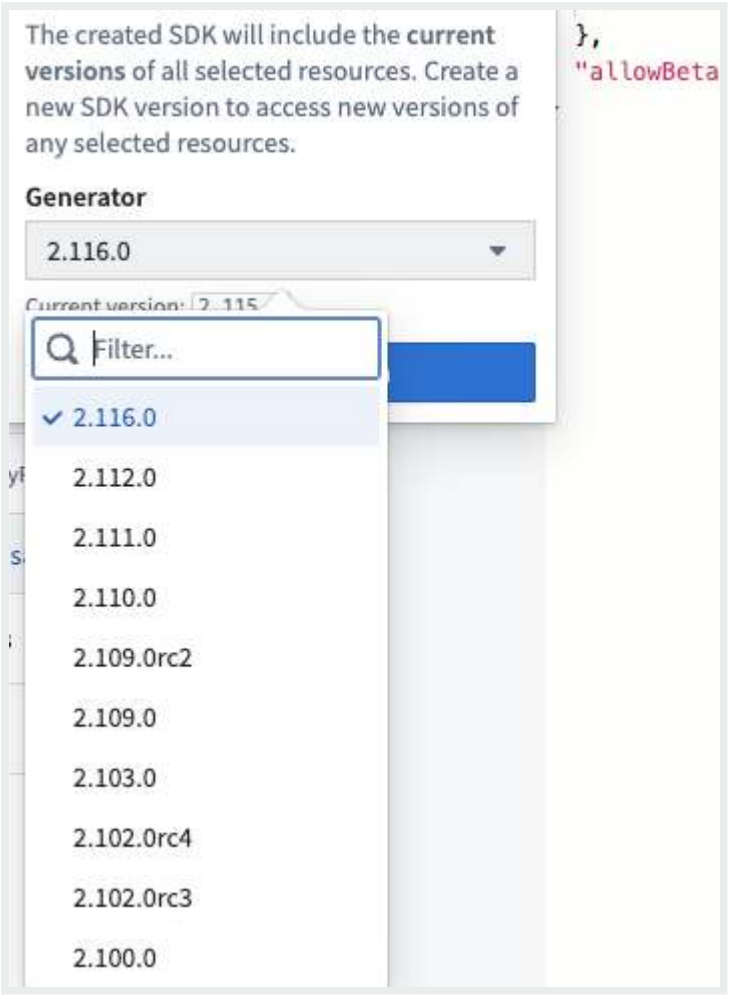
API Reference ↗

Send feedback

To enable beta OSDK versions:

1. Navigate to the **Branches** tab and ensure you are on the default branch.
2. Inside the repository's functions.json file, add `"allowBetaGeneratorVersions": true` to the file. To discover this file, the **Show hidden files** toggle may have to be enabled by selecting the cogwheel at the top near the Files header.

3. Make sure to commit this change in the **Source Control** tab; otherwise, it will not be reflected in the settings.
4. Navigate to the **Libraries Settings** enable the **Beta OSDK Versions** toggle.
5. You can now discover beta generator versions in the **Generator** dropdown when creating a new version in the **Functions Imports** tab.



PREVIOUS
← Spark profiles

NEXT
Ontology imports →

Ontology imports

The **Ontology** tab in Code Repositories is used to import object and link types for use in Functions on Objects. Refer to the Functions documentation on ontology imports to learn more.

PREVIOUS
← Artifact settings

NEXT
Advanced repository settings →

Advanced repository settings

Most Code Repositories settings can be found in the **Settings** tab. Some additional values are configured using the `repoSettings.json` file at the repository root. If the file does not exist, you can create a file named `repoSettings.json` at the repository root.

Custom tag name validation

To enforce that all newly created tags in a repository follow a specific naming scheme in addition to the default constraints enforced by Code Repositories, you may configure a custom regular expression and an error message that is displayed to users if the regex is not satisfied. For example:

```
"tagNameValidation": {
    "regex": "^(0|[1-9]\\d*)\\.(0|[1-9]\\d*)\\.(0|[1-9]\\d*)(-rc\\d+)?$",
    "errorMessage": "Tag name must have the format x.x.x or x.x.x-rcx"
}
```

If you set a value for `tagNameValidation`, you must configure **both** a `regex` and an `errorMessage`.

Pull request description template

To maintain best practices when opening new pull requests, you can configure a description template that will be used to pre-populate the `Description` field when creating

a pull request:

```
"prDescriptionTemplate": "First line of the description template\nAnd anot
```

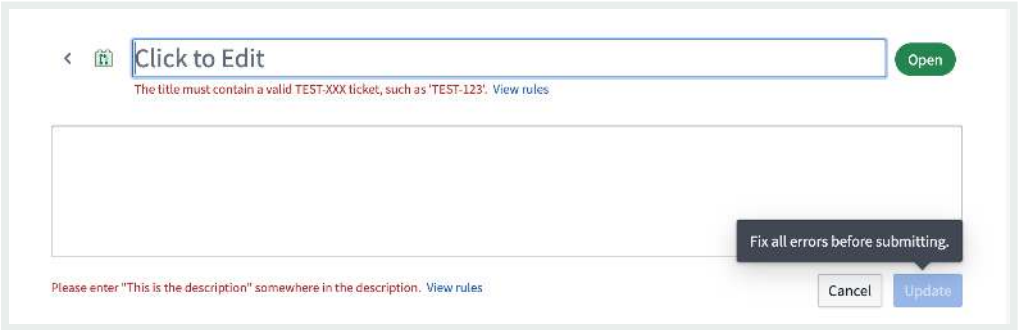
Output dataset path for new transforms

By default, the templates for new transforms files use the file path as the initial output dataset path. For example, for a Python transform `/path/to/your/repository/transforms-python/src/myproject/datasets/name_of_file`. You can change this to a more convenient location, such as a specific folder in your project, by configuring an `outputPathPrefix`:

```
"outputPathPrefix": "/My/Custom/Prefix"
```

With the above setting, the output dataset path will be set to `/My/Custom/Prefix/name_of_file` instead.

Pull request validation rules



You can enforce rules for pull requests within a repository by adding `prValidation` entries to your `repoSettings.json` file. When a pull request is created, it will be validated against th `son` file on the base branch.

API Reference ↗

Send feedback

A pull request validation rule consists of a regular expression, a list of match the expression against, and an error message to render when a field does not compiy

with the rule.

Example: `repoSettings.json`

```
{
    "tagNameValidation": {
        "regex": "^(0|[1-9]\\d*)\\.(0|[1-9]\\d*)\\.(0|[1-9]\\d*)(-rc\\d+)
        "errorMessage": "Tag name must have the format x.x.x or x.x.x-rcx
    },
    "prDescriptionTemplate": "First line of the description template\nAnd
    "outputPathPrefix": "/My/Custom/Prefix",
    "prValidation": [
        {
            "regex": "TEST-\\d{3}",
            "fields": ["title", "branchName"],
            "errorMessage": "This field must contain a valid TEST-XXX tic
        },
        {
            "regex": "This is the description",
            "fields": ["description"],
            "errorMessage": "Please enter 'This is the description' somew
        }
    ]
}
```

PREVIOUS
← Ontology imports

NEXT
Compute Usage →

Compute usage with Code Repositories

Running builds in Code Repositories requires the use of Foundry compute, a resource measured in compute-seconds. This documentation details how builds use compute and provides information about investigating and managing compute usage in the product.

When running a transforms build of one or more datasets, Foundry pulls the transform logic into its serverless compute cluster and executes the code. The length and size of the build depends on the complexity of the code, the size of the input and output datasets, and the Spark computation profile set on the code.

The execution of code on input datasets requires Foundry compute (measured in Foundry compute-seconds) when running the parallelized compute and Foundry storage when the outputs of the transformation are written to Foundry Storage. The act of writing code does not incur compute usage; only the building of datasets incurs compute usage.

Measuring Foundry Compute

The transforms engine powering Code Repositories uses parallel compute on the backend, most commonly in the Spark scalable computing framework. Transformations in Code Repositories are measured in the total number of Foundry *compute-seconds* that are used by the job during its runtime. These compute-seconds are measured during the entire duration of the job, which includes the time taken to read from the input datasets, execute th operations such as I/O waits), and writing the output datasets back to F

You can configure transformations to make use of parallel computation. Compute-seconds are a measure of compute runtime, not wall clock time, and therefore parallel transforms will incur multiple compute-seconds per wall clock second. For a detailed breakdown on how parallelized compute is measured for Foundry compute-seconds for jobs in Code Repositories, review the examples below.

When paying for Foundry usage, the default usage rates are the following:

vCPU / GPU	Usage Rate
vCPU	1
T4 GPU	1.2
V100 GPU	3
A100 GPU	1.3
A10G GPU	1.5
L4 GPU	2.1
H100 GPU	4.7

If you have an enterprise contract with Palantir, contact your Palantir representative before proceeding with compute usage calculations.

Investigating Foundry Compute usage from Code Repositories

Usage information can be found in the Resource Management Application, which allows drill-downs on usage metrics.

While builds are the drivers of Foundry Compute usage, that usage is recorded against the long-lived resource to which it is associated. In the case of dataset transformations, the resource is the dataset (or set of datasets) materialized by the job. You can view the timeline of usage on a dataset in the dataset details tab under **Resource usage metrics**.

API Reference ↗

N rmations that produce multiple output datasets, the compute usage is equally distributed across all the datasets. For example, if a transform datasets, one of which has five rows and the other one five million ro Foundry compute-seconds will be equally distributed between the two.

Send feedback

Understanding drivers of Foundry Compute usage

Unless canceled early, transformations in Code Repositories will run until all logic is run on all data and the outputs are written back to Foundry. The two main factors that affect this runtime are (1) the size of the input data and (2) the complexity of computational operations performed by the transformation logic.

- Jobs with larger input data sizes will require more compute than jobs with smaller input data sizes if they have the same logic. For example, a job that performs column processing on 100GB of data will use more Foundry compute-seconds than a job performing the same processing on 10GB of data.
- Jobs with that perform more complex operations on data will require more compute than jobs that perform comparatively fewer operations. This is sometimes known as "job complexity."
  - As a basic example, consider the number of operations between two mathematical operations, `5 * 5` and `5!`. `5 * 5` is a single multiplication operation. `5!` is equivalent to `5 * 4 * 3 * 2 *1` (four multiplication operations), which is double the complexity of the `5 * 5` example. As jobs get more complex with tasks such as aggregations, joins, or machine learning algorithms, the number of operations a job must complete on the data can grow.

Managing Foundry Compute usage with Code Repositories

For each job, you can review the underlying compute metrics that drove the performance and compute usage of the job. For more details, review Understanding Spark Details.

In a job, Foundry compute-seconds are driven by the size and number of parallelized executors. Both of these settings are fully configurable per job. Review the Spark computation profile documentation for details on how this is set per job. The size of executors is governed by their memory and vCPU counts. Increased vCPU and increased memory per executor will increase the compute-seconds incurred by that executor.

T API Reference ↗ taneous tasks is driven by the configured executor counts and their corresponding number of vCPUs. If no configuration overrides are sp

transformation will use a default Spark profile. Foundry storage for resulting datasets is driven by the size of the dataset being created.

In the end, jobs with different logic can accomplish the same outcome with very different numbers of operations.

Optimizing code to manage usage

There are a number of ways to optimize your code and manage the compute-seconds your jobs use. This section provides links to more information about commonly-used optimization techniques.

- Spark, the distributed cluster-computing framework adopted by Foundry for most types of code repository batch compute, allows for a variety of optimization techniques. Learn more about optimizing Spark.
- In Spark, you can optimize partitioning to expedite builds. The optimal number of partitions depends on the number of rows, the number of columns, the columns' type, and the content. We recommend an approximate ratio of one partition for every 128 MB of dataset size.
- Incremental computation is an efficient method of performing transforms to generate an output dataset. By leveraging the build history of a transform, incremental computation avoids the need to recompute the entire output dataset every time a transform is run.
  - Learn more about incremental transforms in Python.
  - Learn more about incremental transforms in Java.
- Especially for small-to-medium-sized datasets, there are several compute engines other than Spark that consistently surpass Spark in performance in benchmarks for single-node applications. Consequently, using these alternatives for running your pipelines can lead to increased processing speed and reduced compute consumption. To fully capitalize on these options, we advise becoming acquainted with Lightweight transforms.
- If your builds are orchestrated using Schedules, we recommend reading our scheduling best practices to optimize costs.

API Reference ↗

Foundry Compute usage

Send feedback

Usage example 1: Standard memory

This example demonstrates how Foundry Compute is measured for a statically-allocated job with a standard memory request.

```
Driver profile:
    vCPUs: 1
    GiB_RAM: 6
Executor profile:
    vCPUs: 1
    GiB_RAM: 6
    Count: 4
Total Job Wall-Clock Runtime:
    120 seconds



Calculation
driver_compute_seconds = max(num_vcpu, GiB_RAM/7.5) * num_seconds
                       = max(1vcpu, 6gib/7.5gib) * 120sec
                       = 120 compute-seconds

executor_compute_seconds = num_executors * max(num_vcpu, GiB_RAM/7.5) * num_secc

                         = 4 * max(1, 6/7.5) * 120sec
                         = 480 compute-seconds

total_compute_seconds = 120 + 480 = 600 compute-seconds
```

Usage example 2: Large memory

This example demonstrates how Foundry Compute is measured for a statically-allocated job with a large memory request.

```
Driver Profile:
    vCPUs: 2
```

```
    GiB_RAM: 6
Executor profile:
    vCPUs: 1
    GiB_RAM: 15
    Count: 4
Total Job Wall-Clock Runtime:
    120 seconds



Calculation:
driver_compute_seconds = max(num_vcpu, GiB_RAM/7.5) * num_seconds
                       = max(2vcpu, 6gib/7.5gib) * 120sec
                       = 240 compute-seconds

executor_compute_seconds = num_executors * max(num_vcpu, GiB_RAM/7.5) * num_secc
                         = 4 * max(1, 15/7.5) * 120sec
                         = 960 compute-seconds

total_compute_seconds = driver_compute_seconds + executor_compute_seconds
                      = 240 + 960 = 1200 compute-seconds
```

Usage example 3: Dynamic executor counts

This example demonstrates how Foundry Compute is measured for a dynamically-allocated job, where some job execution time is performed with two executors, and the rest of the job time is performed with four executors.

```
Driver Profile:
    vCPUs: 2
    GiB_RAM: 6
Executor profile:
    vCPUs: 1
    Count:
        min: 2
        max: 4
```

```
Total Job Wall-Clock Runtime:
    120 seconds:
        2 executors: 60 seconds
        4 executors: 60 seconds



Calculation:
driver_compute_seconds = max(num_vcpu, GiB_RAM/7.5) * num_seconds
                       = max(2vcpu, 6gib/7.5gib) * 120sec
                       = 240 compute-seconds

# Calculate compute seconds for job time with 2 executors
2_executor_compute_seconds = num_executors * max(num_vcpu, GiB_RAM/7.5) * num_se
                           = 2 * max(1, 6/7.5) * 60sec
                           = 120 compute-seconds

# Calculate compute seconds for job time with 4 executors
4_executor_compute_seconds = num_executors * max(num_vcpu, GiB_RAM/7.5) * num_se
                           = 4 * max(1, 6/7.5) * 60sec
                           = 240 compute-seconds



total_compute_seconds = driver_compute_seconds + 2_executor_compute_seconds + 4_
                      = 240 + 120 + 240 = 600 compute-seconds
```

Usage example 4: GPU compute

This example demonstrates how Foundry GPU Compute is measured for a statically-allocated job.

```
Driver profile:
    T4 GPU: 1
    T4 GPU: 1
    Count: 4
Total Job Wall-Clock Runtime:
```

```
    120 seconds


Calculation:
driver_compute_seconds = num_gpu * gpu_usage_rate * num_seconds
                       = 1gpu * 1.2 * 120sec
                       = 144 compute-seconds

executor_compute_seconds = num_executors * num_gpu * gpu_usage_rate * num_second
                         = 4 * 1 * 1.2 * 120sec
                         = 576 compute-seconds

total_compute_seconds = 144 + 576 = 720 compute-seconds
```

← PREVIOUS
Advanced repository settings

NEXT
Data Lineage / Overview →